The Agentic Developer

Stop Writing Code. Start Directing AI.

A comprehensive guide to building software with AI agents. 29 chapters covering mindset, context engineering, workflows, tools, scaling, .NET patterns, and the discipline to do it right.

By Julio Casal

Software Engineer, Ex-Microsoft, Founder of .NET Academy

Table of Contents

Introduction

Part I: The Shift

Chapter 1: Software Development Has Changed

Chapter 2: The METR Paradox

Chapter 3: Your New Job Description

Chapter 4: The Agentic Engineering Mindset

Chapter 5: From Vibe Coding to Vibe Engineering

Part II: Context Engineering

Chapter 6: Writing Effective Context Files

Chapter 7: Path-Scoped Rules

Chapter 8: Skills: Teaching Your Agent New Tricks

Chapter 9: MCP Servers and External Tool Integration

Chapter 10: Designing an AI-Friendly Codebase

Part III: The Tools

Chapter 11: IDE Agents

Chapter 12: CLI and Cloud Agents

Part IV: The Workflows

Chapter 13: The TDD Agent Loop

Chapter 14: Spec-First Development

Chapter 15: Error-Message-Driven Debugging

Chapter 16: Codebase Archaeology

Chapter 17: Migration Generation

Part V: Scaling Up

Chapter 18: The Parallel Agent Lifestyle

Chapter 19: Background Agents

Chapter 20: Multi-Agent Coordination

Chapter 21: Agent + CI/CD Integration

Chapter 22: Cost Management

Part VI: .NET-Specific Patterns

Chapter 23: Context Files for .NET

Chapter 24: Agents and .NET Aspire

Chapter 25: EF Core Migrations with Agents

Chapter 26: Microsoft Agent Framework

Part VII: The Discipline

Chapter 27: Security Review with Agents

Chapter 28: When NOT to Use Agents

Chapter 29: The 30-Day Agentic Transformation

Introduction

About the Author

Julio Casal is a software engineer, ex-Microsoft, and founder of .NET Academy (dotnetacademy.io). He builds real systems, teaches thousands of developers, and has spent the last year going deep on agentic workflows so you don't have to learn the hard way.

This is not a hype piece. This is not "AI will replace developers" clickbait.

This is a field guide. Twenty-nine chapters covering everything from the mindset shift to concrete workflows, from context engineering to multi-agent coordination, from IDE tools to CI/CD integration.

Developers are adopting AI tools at a staggering rate. Anthropic's 2026 report says developers now use AI in 60% of their work. GitHub Copilot has generated over a million pull requests. Every IDE ships an "agent mode." The tools are everywhere.

But a landmark study from METR found that experienced open-source developers were actually 19% *slower* when using AI tools on familiar codebases. And here's the kicker: those same developers *believed* they were 20% faster.

That gap between perception and reality is where most developers live right now. They're using AI. They think it's helping. And in many cases, it's making them worse.

This book exists because the difference between those two outcomes (slower versus faster) isn't the tool. It's the technique. The developers who are genuinely more productive with AI agents aren't the ones who type the best prompts. They're the ones who provide the best context, write the best specs, and review the output with the sharpest eye.

They've become agentic developers.

A Note on Language

The code examples in this book use C# because that's what I write every day. But every pattern, every workflow, every principle applies to whatever language you use. If you write Python, TypeScript, Go, Java, or Rust, this book is for you. The agent doesn't care about your language. It cares about your context.

Let's Go

You can read this book in a weekend. By the end, you'll have a fundamentally different understanding of how to work with AI agents, and concrete workflows you can start using on Monday.

PART I

The Shift

CHAPTER 1

Software Development Has Changed

There's a moment most developers remember. Maybe it was the first time GitHub Copilot completed an entire function for you. Maybe it was the first time you described a feature in plain English and watched an agent write, test, and commit the code. Maybe it was the moment you realized you'd spent the last hour *reviewing* code instead of *writing* it.

Whatever the moment was, it split your career into before and after.

Software development has changed. Not in the way pundits predicted (no, developers are not obsolete), and not in the way skeptics hoped (no, this isn't just better autocomplete). It changed in a way that's both more subtle and more profound: the act of building software has shifted from writing code to directing the writing of code.

This chapter traces that shift. Not to rehash the history of AI for its own sake, but because understanding the trajectory tells you where to aim.

The Three Eras

Era 1: Autocomplete (2021-2023)

GitHub Copilot launched in June 2021, and at first, it was exactly what it looked like: fancy autocomplete. You'd type a function signature, and it would suggest a body. You'd write a comment, and it would generate matching code. Useful? Absolutely. Transformative? Not yet.

The autocomplete era had a clear interaction model. You drove. The AI rode shotgun. It could suggest the next few lines, but it had no understanding of your project, your architecture, or your intent beyond the immediate file. It operated on a single-file context window, and that context window was small.

GitHub's own research during this period showed that developers using Copilot completed 26% more tasks per week. Meaningful, but incremental. You were still writing code. You just had a fast typist looking over your shoulder.

The mental model was simple: AI helps you type faster. Your job didn't change. Your skills didn't change. You just had a productivity boost on the mechanical parts of coding.

Era 2: Chat and Inline Assistance (2023-2025)

Then came ChatGPT, and with it, a different interaction model. Instead of autocomplete, you had a conversation. You could describe a problem and get a solution. You could paste an error message and get a diagnosis. You could ask "how does OAuth 2.0 work?" and get an explanation tailored to your codebase.

This era introduced chat panels in IDEs. Copilot Chat. Cursor's chat interface. Cody. The interaction model shifted from "complete my line" to "answer my question" and eventually to "edit this code for me."

The productivity gains were real but uneven. Developers who were already strong could use chat as a high-bandwidth rubber duck: a thinking partner that happened to know every framework and library. But developers who were struggling often got worse. They'd accept generated code they didn't understand, introducing bugs they couldn't debug.

This era also surfaced a critical insight that would define everything that followed: the quality of the output was directly proportional to the quality of the input. Vague questions got vague answers. Specific, well-structured prompts got excellent code. The skill wasn't "using AI." The skill was communicating precisely.

Era 3: Agents (2025-Present)

In 2025, the interaction model changed again, and this time the change was fundamental. The AI stopped being a tool you used and started being an agent you directed.

The difference matters. A tool does what you tell it, one step at a time. An agent takes a goal, breaks it into steps, executes them, observes the results, and adjusts. Agent mode in Copilot, Cursor, and Windsurf didn't just suggest code. It could:

- Read multiple files across your codebase
- Run your tests and see the output
- Execute terminal commands
- Create and delete files
- Iterate on its own work based on errors

Then came the CLI agents: Claude Code, Codex CLI. These operated directly in your terminal, with access to your full development environment. They could grep your codebase, run your build, execute your test suite, and iterate until things worked.

Then came background agents: GitHub Copilot Coding Agent, Codex Cloud, Google Jules. These didn't even need your IDE open. You could assign a task from your phone, and the agent would create a branch, write the code, run the tests, and open a pull request for your review.

Andrej Karpathy, who coined "vibe coding" in February 2025, evolved his terminology by February 2026 to "agentic engineering," calling it "a serious engineering discipline involving autonomous agents." The casual, throw-a-prompt-and-see-what-happens era was over.

What Actually Changed

The shift from autocomplete to agents isn't just a feature upgrade. It changed the fundamental economics of software development.

Writing Code Became Cheap

This is the hardest thing for experienced developers to internalize. For your entire career, the scarce resource was the ability to translate ideas into working code. That's what companies paid for. That's what made you valuable.

Writing code is no longer the bottleneck. An agent can generate hundreds of lines of working code in seconds. The GitHub/Microsoft study found that Copilot users completed 26% more tasks, effectively turning an 8-hour day into 10 hours of output. At scale, GitHub reported up to 81% productivity improvement among active Copilot users. Over a million pull requests were created by agents between May and September 2025 alone.

The raw production of code is cheap and getting cheaper.

Reading Code Became Expensive

Here's what nobody talks about enough: as code generation gets cheaper, code *review* gets more expensive. Not in dollars, but in cognitive load.

When an agent generates 200 lines of code in 30 seconds, someone has to verify that those 200 lines are correct, secure, performant, and maintainable. That someone is you. And reviewing code you didn't write is harder than reviewing code you did write, because you don't have the mental model of the decisions that were made along the way.

Addy Osmani, engineering lead on Google Chrome and author of *Beyond Vibe Coding*, put it precisely: "You're trading typing time for review time, implementation effort for orchestration skill, writing code for reading and evaluating code."

This is the new trade. And it's not a bad one. But it requires different skills than most developers have spent their careers building.

Specification Became the Product

In the agent era, the specification IS the work. A vague request like "build me a REST API for user management" will produce code. It will compile. It might even pass basic

tests. But it will be riddled with assumptions you didn't intend, patterns you didn't want, and edge cases you didn't consider.

A precise specification, one that defines the endpoints, the data model, the error handling behavior, the authentication scheme, the test scenarios, and the architectural constraints, will produce code that's genuinely useful. Often better than what you'd have written by hand, because the agent can apply patterns consistently across an entire codebase without fatigue or distraction.

Simon Willison, one of the most prolific practitioners of agentic development, made this observation: "Code that started from your own specification is a lot less effort to review." That's the key insight. The spec isn't just input to the agent. It's input to your future self, the reviewer.

Context Became King

The single biggest differentiator between developers who thrive with agents and those who struggle is context engineering. Not prompting. Context engineering.

Prompting is what you type into the chat box. Context engineering is everything else: the files that tell the agent about your project, the rules that guide its behavior, the architectural decisions documented in markdown, the test suite that validates its output, and the codebase itself.

Bharani Subramaniam from ThoughtWorks defined it simply: "Context engineering is curating what the model sees so that you get a better result."

The developers who are genuinely faster with AI agents have invested in making their codebases legible to agents. They have context files that explain their conventions. They have comprehensive test suites that catch mistakes. They have clear architectural boundaries that prevent agents from making cross-cutting changes they shouldn't. They haven't just adopted tools. They've adapted their entire development environment.

The Uncomfortable Truth

Let's be honest about something. Most developers are in a weird middle ground right now. They're using AI tools. They have Copilot or Cursor or Claude Code installed. They use them daily. And many of them are getting worse, not better.

The METR study (which we'll dig into deeply in the next chapter) found that experienced developers were 19% slower with AI tools on familiar codebases. But they *perceived* themselves to be 20% faster. That's not a small discrepancy. That's a 39-percentage-point gap between perception and reality.

This happens because AI tools are seductive in a very specific way. They remove the *feeling* of effort. When an agent writes 200 lines of code for you, it feels like you accomplished something. When you then spend 45 minutes debugging a subtle issue the agent introduced, that feels like a separate problem, not an AI problem. The human brain doesn't naturally attribute the debugging time back to the generation that caused it.

The result is that most developers are in what I call the "productivity illusion": using AI tools, feeling productive, and actually being less effective than they would be without them.

This book exists to get you out of that illusion and into genuine, measurable productivity gains.

What the Best Developers Are Doing Differently

The developers who are genuinely thriving with agents, the ones who are 2-3x more productive, not 19% slower, share a set of practices. These practices are the subject of this entire book, but here's the preview:

They write specs before prompts. They don't start with "build me a thing." They start with a design document, a set of requirements, or at minimum a detailed description of what they want. The spec becomes the prompt.

They invest in context files. Their projects have AGENTS.md or CLAUDE.md files that tell agents how the project works. Coding conventions, architecture decisions, build commands, testing frameworks. This context makes every agent interaction better.

They write tests first. This is the single highest-leverage practice. When you have a comprehensive test suite, you can let an agent iterate freely because the tests catch mistakes automatically. The agent writes code, runs the tests, fixes failures, and repeats. You review the final result, not every intermediate step.

They review ruthlessly. They don't accept generated code at face value. They read it with the same (or greater) scrutiny they'd apply to a pull request from a junior developer. They question architectural decisions. They check edge cases. They verify security implications.

They work in parallel. Following Simon Willison's "parallel coding agent lifestyle," they run multiple agents simultaneously on different tasks. One agent researches a library, another fixes a bug, a third implements a feature. The developer orchestrates and reviews.

They know when to stop. They recognize when an agent is going in circles, when the task is better done by hand, and when the cost of context-switching to agent mode exceeds the benefit. Not every task should be agentic.

The Anthropic Forecast

Anthropic's 2026 Agentic Coding Trends Report identified eight trends reshaping software development. The first is the most relevant: a "tectonic shift in the SDLC" where engineers move from writing code to agent supervision, system design, and output review.

This isn't a future prediction. This is a description of what's already happening at companies that have adopted agentic workflows seriously. Engineers are spending

less time in editors and more time in specifications and reviews. They're writing less code and reading more code. They're making fewer keystrokes and more decisions.

The report also notes that agents are becoming "team players," coordinating with each other in multi-agent systems. One agent handles the API layer, another handles the frontend, a third writes tests, and an orchestrator manages the coordination. This is no longer science fiction. It's available today in tools like Claude Code's sub-agent system and the ComposioHQ agent-orchestrator.

What This Means For You

If you're reading this, you're probably in one of three positions:

You're skeptical. You've tried AI tools, found them overhyped, and gone back to writing code the old way. The METR study probably validates your experience. And you're right: naive AI tool use *does* hurt. But disciplined agentic development is a different thing entirely. Give this playbook a chance. The workflows here are nothing like "ask ChatGPT to write your code."

You're enthusiastic but frustrated. You use AI tools every day but the results are inconsistent. Sometimes the agent nails it. Sometimes it produces garbage. You can't figure out the pattern. The pattern is context. Chapter 4 of this playbook will change how you think about it.

You're already in. You've adopted agentic workflows and you're looking to sharpen your technique. You want the specific patterns, the templates, the workflows that separate good from great. That's what the TDD Agent Loop chapter (and the full book) deliver.

Wherever you're starting from, here's the promise: by the end of this playbook, you'll understand the shift, you'll know why disciplined technique matters, and you'll have a concrete, battle-tested workflow you can start using immediately.

Software development has changed. The question isn't whether to adapt. It's how fast.

CHAPTER 2

The METR Paradox

In July 2025, a research organization called METR published a study that should have been a wake-up call for every developer using AI tools. It wasn't. Most people either ignored it, dismissed it, or cherry-picked the parts that confirmed what they already believed.

Here's what the study found: experienced open-source developers working on codebases they knew well were **19% slower** when using AI tools compared to working without them.

Let that land for a second. Not beginners. Experienced developers. Not unfamiliar codebases. Repositories they had contributed to, understood deeply, and navigated daily. The AI tools didn't help them. The tools actively slowed them down.

But here's the part that turns this from an interesting finding into something genuinely unsettling: those same developers **believed they were 20% faster**.

That's a 39-percentage-point gap between perception and reality. And it tells us something critical about the current state of AI-assisted development that no amount of marketing from tool vendors will address.

The Study

The METR study (published as a preprint on arXiv) was rigorous in ways that most AI productivity claims are not. They didn't survey developers about how they felt. They measured actual task completion times. Experienced open-source contributors worked on real issues in repositories they maintained, with and without AI assistance, in a controlled setting.

The tasks were representative: bug fixes, feature implementations, refactoring. Real work, not coding challenges or toy problems. The kind of tasks that make up a developer's actual week.

And the result was clear. AI tools slowed these developers down.

Why AI Makes Experts Slower (When Used Naively)

The finding sounds counterintuitive until you think about what actually happens when an experienced developer uses an AI tool on familiar code.

The Context Switch Tax

When you know a codebase deeply, you have a rich mental model. You know where things are. You know why decisions were made. You know the hidden constraints that aren't documented anywhere. You can hold the relevant context in your head and write code that fits the system like a puzzle piece.

When you introduce an AI agent into this workflow, you're adding a context switch. Instead of writing code directly from your mental model, you now have to:

1. Translate your mental model into a prompt
2. Wait for the agent to generate code
3. Read the generated code
4. Compare it against your mental model
5. Identify discrepancies
6. Decide whether to fix the code or regenerate
7. Often, just rewrite it yourself anyway

For an expert working on familiar code, steps 1-6 take longer than just writing the code. The agent doesn't have your mental model. It doesn't know about the conversation you had with a teammate three months ago that explains why the authentication

middleware is structured that way. It doesn't know that the seemingly redundant null check exists because of a specific edge case in production.

The Illusion of Progress

So why did these developers *think* they were faster? Because AI tools create a compelling illusion of productivity.

When an agent generates 50 lines of code in two seconds, your brain registers that as "I just did 50 lines of work." It feels fast because the *visible output* appeared fast. The invisible work, reading it, verifying it, fixing the three subtle issues, adjusting the approach when it doesn't quite fit, all gets mentally filed under "normal development work," not "overhead from using AI."

This is a well-documented cognitive bias. Humans are bad at attributing effort to the right cause, especially when the cause and the effort are separated in time. The generation feels instant. The debugging happens ten minutes later. Your brain doesn't connect them.

The Sunk Cost Trap

There's another dynamic at play. When an agent generates a large block of code, developers feel reluctant to throw it away. They've invested time in crafting the prompt, waiting for the response, and reading the output. Even when the code isn't quite right, there's a pull to "fix it up" rather than start fresh.

This is almost always the wrong choice. Fixing AI-generated code that doesn't match your mental model takes longer than writing correct code from scratch. But the sunk cost makes it feel wasteful to discard it.

The Other Side of the Data

Now, the METR study has important boundary conditions that matter enormously.

The study measured experienced developers on *familiar* codebases. This is a very specific scenario. And it's not the only scenario that matters.

Other studies paint a different picture for different contexts:

GitHub's research found that developers using Copilot completed 26% more tasks. The key difference: this included unfamiliar tasks and greenfield work, exactly the contexts where you *don't* have a deep mental model to draw on.

Individual developer output studies show 20-40% productivity increases when AI tools are used across a mix of familiar and unfamiliar work.

GitHub's scale data showed up to 81% productivity improvement among active Copilot users, measured across over a million pull requests created between May and September 2025.

So the full picture isn't "AI tools don't work." It's more nuanced and more useful:

- **AI tools hurt experts on familiar code** (when used naively)
- **AI tools help everyone on unfamiliar code** and greenfield work
- **AI tools help most on boilerplate, repetitive, and mechanical tasks**
- **The gap between "helps" and "hurts" is technique**

That last point is the one that matters for this book.

The Discipline Gap

Here's the paradox, stated cleanly: the same tools that make naive users slower make disciplined users dramatically faster. The variable isn't the tool. It's the technique.

What does disciplined use look like? It looks like the practices we'll cover throughout this book:

1. Knowing When to Use Agents (And When Not To)

The developers who are genuinely more productive have developed an intuition for when an agent will help and when it will hurt. Rules of thumb:

Use an agent when: - You're working in an unfamiliar part of the codebase - The task is boilerplate-heavy (CRUD endpoints, data models, configuration) - You need to implement a well-known pattern (repository pattern, middleware, API versioning) - You're exploring a new library or framework - The task is well-specified and test-coverable

Don't use an agent when: - You can write the code faster than you can describe it - The task requires deep knowledge of hidden constraints - You're in a flow state and the code is flowing naturally - The change is tiny (a one-line fix where you already know the line)

The METR developers didn't make this distinction. They used AI tools across the board, including on tasks where their expertise was the fastest path.

2. Investing in Context

The METR study participants used AI tools with minimal context engineering. They didn't have AGENTS.md files explaining their project conventions. They didn't have comprehensive test suites that could validate agent output automatically. They didn't provide architectural constraints or coding standards.

They did what most developers do: opened the agent, typed a prompt, and hoped for the best.

Disciplined agentic development looks nothing like that. It looks like a carefully engineered context stack (which we cover in Chapter 4) that gives the agent enough information to produce useful output on the first try.

3. Writing Tests First

This is the single biggest differentiator. When you have tests, you can let an agent iterate autonomously. It writes code, runs the tests, sees failures, and fixes them. The feedback loop is tight and automatic.

Without tests, you're the feedback loop. You read every line, evaluate every decision, and catch every bug manually. That's slow. That's what the METR developers were doing.

The TDD Agent Loop (Chapter 5 in this playbook) is designed specifically to address this. It's the workflow that turns the METR paradox on its head.

4. Reviewing Like an Architect

Disciplined developers don't just check if the code "works." They review agent output the way a senior architect reviews a pull request:

- Does this fit the existing patterns in the codebase?
- Are there security implications?
- How will this perform at scale?
- Is this maintainable by a human who hasn't seen the prompt?
- What's the test coverage?

This kind of review is a skill. It's learnable. And it's far more valuable than the ability to type code fast.

The GitClear Warning

While we're looking at data, one more study deserves attention. GitClear's 2025 analysis found a **4x growth in code clones** in repositories using AI assistants. That is, copied or near-identical code blocks appearing throughout codebases.

This is the quality side of the equation. Even when AI tools make you *faster* (in terms of task completion), they can make your codebase *worse*. Agents don't refactor. They don't say "this pattern already exists in your utility library, let me reuse it." They generate fresh code for every request, and that code often duplicates existing functionality.

Disciplined developers catch this during review. They look for duplication, unnecessary abstractions, and code that should have been a function call instead of a reimplementation. Undisciplined developers accept the output and move on, accumulating technical debt at 4x the normal rate.

The Real Lesson

The METR study isn't an argument against AI tools. It's an argument against naive AI tool use. And the distinction matters enormously.

Naive use: open the agent, type a prompt, accept the output, move on.

Disciplined use: engineer the context, write the tests, craft the specification, let the agent work within constraints, review the output thoroughly, iterate when needed, and know when to do the work by hand.

The gap between these two approaches is the gap between being 19% slower and being 2-3x faster. It's the gap between thinking you're productive and actually being productive.

This is why Addy Osmani observed that "AI-assisted development actually rewards good engineering practices MORE than traditional coding does." The better your specs, the better the AI's output. The more comprehensive your tests, the more confidently you can delegate. The cleaner your architecture, the less likely the agent is to introduce inconsistencies.

Good engineering practices were always valuable. In the agent era, they're *essential*.

Your Calibration Check

Before moving on, try this exercise. Think about the last week of your development work. For each task where you used an AI tool, ask:

1. Was I actually faster, or did it just *feel* faster?
2. How much time did I spend debugging or fixing AI-generated code?
3. Did I accept any code I didn't fully understand?
4. Would I have been faster writing it by hand?

Be honest. The METR developers weren't, and the 39-percentage-point gap between their perception and reality is the proof.

If you find that AI tools are genuinely saving you time, great. You're probably already practicing some of the discipline this book teaches. Sharpen it further.

If you find that the tools might be creating a productivity illusion, that's not a failure. It's awareness. And awareness is the first step to the transformation the rest of this book provides.

The paradox is real. But it's solvable. That's what the next chapters are for.

CHAPTER 3

Your New Job Description

If you've internalized the first two chapters, you're sitting with an uncomfortable realization: the thing you've spent years getting good at (writing code) is no longer the primary thing your job demands.

That doesn't mean your skills are useless. It means they've been *promoted*. Everything you know about software architecture, system design, debugging, testing, and code quality is more valuable than ever. You just apply it differently now.

Addy Osmani captured this precisely: "Agentic engineering disproportionately benefits senior engineers." The more you know about building software, the better you are at directing agents to build it. Your experience isn't obsolete. It's your competitive advantage.

But the day-to-day work looks different. Let's define what the job actually is now.

The Old Job vs. The New Job

What You Used to Do

A typical day for a senior developer in 2023 looked something like this:

- **Morning:** Read through open PRs, leave comments, merge what's ready
- **Mid-morning:** Pick up a ticket, read the requirements, start coding
- **Afternoon:** Write implementation code, debug issues, write tests
- **Late afternoon:** More implementation, maybe a design discussion
- **End of day:** Push code, update the ticket, plan tomorrow

The core activity: translating requirements into code. You spent maybe 60-70% of your time writing or editing code, 20% reading code, and 10% on everything else.

What You Do Now

A typical day for an effective agentic developer in 2026:

- **Morning:** Review PRs from overnight background agents, approve or send back with notes
- **Mid-morning:** Write specs for the day's work, break features into agent-sized tasks
- **Late morning:** Launch agents on 3-4 tasks in parallel, review first results
- **Afternoon:** Architecture decisions on edge cases agents flagged, review more output, refine context files based on patterns you see
- **Late afternoon:** Final reviews, integration testing, context file updates for tomorrow
- **End of day:** Queue up background agent tasks for overnight work

The core activity: specification, review, and orchestration. You spend maybe 20% of your time writing code by hand, 40% reviewing agent output, and 40% on specs, architecture, and context engineering.

That's not a small shift. That's a fundamentally different job.

The Five New Skills

The transition from code-writer to agentic developer requires five skills that most developers haven't deliberately practiced. They're all learnable. Some of them you already have in embryonic form.

1. Context Engineering

This is the big one. Chapter 4 of this playbook goes deep on it, so here's the short version: context engineering is the skill of curating what the agent sees so that it produces better results.

It includes:

- Writing and maintaining AGENTS.md or CLAUDE.md files
- Creating path-scoped rules for different parts of your codebase
- Designing your codebase to be AI-readable (clear naming, good structure, documented decisions)
- Knowing what context to include and, crucially, what to exclude
- Building and maintaining a test suite that serves as automated verification

This skill didn't exist two years ago. It's now the most important skill in your toolkit. The developers who are 2-3x more productive with agents are, almost without exception, excellent context engineers.

2. Specification Writing

Remember when "writing good requirements" was something PMs did and developers complained about? Now it's your core deliverable.

When you direct an agent, your specification IS the work. The more precise your spec, the better the output. The more ambiguous your spec, the more time you spend on review and rework.

Good specs for agents look like this:

```
## Task: Add rate limiting to the API

### Requirements
- Apply rate limiting to all endpoints under /api/v1/
- Use a sliding window algorithm with a 60-second window
- Default limit: 100 requests per window per API key
- Return 429 Too Many Requests with a Retry-After header when exceeded
- Rate limit state stored in Redis (use the existing IDistributedCache)
- Bypass rate limiting for requests with the X-Internal-Service header

### Constraints
- Do NOT modify the existing middleware pipeline ordering
- Use the existing ApiKey from the request context (HttpContext.Items["ApiKey"])
- Follow the existing middleware pattern in src/Api/Middleware/

### Test Scenarios
- Request within limit returns 200
- Request exceeding limit returns 429 with correct Retry-After value
- Rate limit resets after window expires
- Internal service header bypasses rate limit
- Different API keys have independent limits
```

Compare that to: “Add rate limiting to the API.” Same task. Vastly different output quality.

The spec isn’t just for the agent. As Simon Willison observed, “Code that started from your own specification is a lot less effort to review.” When you wrote the spec, you already know what the code *should* do. Reviewing becomes verification, not exploration.

3. Architectural Thinking

When agents write most of the code, the decisions that remain with you are the ones that matter most: the architectural decisions.

Which patterns to use. How to structure the project. Where to draw service boundaries. How data flows through the system. What trade-offs to make between complexity and flexibility. These are the decisions agents can't make well because they require understanding the full business context, the team's capabilities, the deployment constraints, and the long-term evolution of the system.

This is why Osmani says agentic engineering benefits senior engineers disproportionately. A junior developer can write a prompt. A senior developer can write a prompt *and* evaluate whether the resulting architecture will hold up under load, scale with the team, and survive the next three requirements changes.

If you've been coding for years and feel like your "just typing code" skills are becoming less valuable, here's the reframe: your architectural judgment is becoming *more* valuable. It's the thing agents can't replicate.

4. Code Review (At Scale)

You've done code review before. But you probably haven't done it at the volume and velocity that agentic development demands.

When you're running 3-5 agents in parallel (which is the workflow Simon Willison describes and we cover in the full book), you might review 10-20 substantial code changes in a single day. That's more review than most developers do in a week.

This requires a different approach than the leisurely PR review you're used to. You need:

A triage system. Not every agent output needs the same level of scrutiny. A boilerplate CRUD endpoint needs a quick scan. A security-sensitive authentication change needs line-by-line review. Develop a sense for what's high-risk and what's routine.

Pattern recognition. After reviewing hundreds of agent outputs, you'll start recognizing common failure modes: unnecessary abstractions, duplicated logic, missing edge cases, overly complex solutions to simple problems. Your review speed increases as your pattern library grows.

Automated guardrails. You shouldn't be the only line of defense. Comprehensive test suites, linting rules, static analysis, and CI pipelines catch the mechanical issues. Your

review focuses on the things machines can't catch: architectural fit, business logic correctness, and long-term maintainability.

The "would I approve this PR from a junior developer?" test. If the answer is "I'd send it back with comments," don't accept it from the agent either. Your quality bar doesn't change just because the code was free to produce.

5. Agent Orchestration

This is the meta-skill: knowing how to break work into pieces that agents can handle effectively, when to run agents in parallel versus sequentially, when to intervene versus let the agent iterate, and when to do the work yourself.

Good orchestration looks like:

- **Decomposing a feature** into 4-5 independent tasks that agents can work on simultaneously
- **Ordering dependent work** so that agent A's output feeds into agent B's context
- **Recognizing stuck agents** early and redirecting instead of letting them burn tokens going in circles
- **Switching tools** based on the task (CLI agent for terminal work, IDE agent for multi-file edits, background agent for overnight tasks)

Bad orchestration looks like:

- Giving an agent a vague, multi-day task and hoping for the best
- Running agents sequentially when they could work in parallel
- Spending 30 minutes crafting a prompt for a task you could code in 10 minutes
- Accepting agent output without review because "it generated fast, it must be right"

The Emotional Shift

Let's talk about something the technical literature ignores: this transition can feel terrible.

If you've spent 10+ years becoming an excellent code-writer, it can feel like the ground shifted under you. The thing you're great at is still useful, but it's no longer the main thing. That's disorienting. It can feel like a demotion even though it's objectively a promotion.

Here's the reframe that helped me: you're not losing the coding skill. You're adding skills on top of it. An architect who can also code is more valuable than an architect who can't. A reviewer who understands implementation deeply catches things a non-coding reviewer misses. Your coding skill makes you a *better* agentic developer, not an obsolete one.

The developers who struggle most with this transition are the ones who define their identity by their typing speed, their ability to hold an entire system in their head, or their pride in "doing it myself." These are all legitimate sources of professional pride. They're just no longer sufficient on their own.

The developers who thrive are the ones who redefine their identity around *outcomes*: shipping working software, making good architectural decisions, maintaining high code quality, and enabling their team (human and AI) to do their best work.

The 80/20 of Agentic Development

If you do nothing else after reading this playbook, do these three things. They account for roughly 80% of the productivity gains:

1. Create a Context File for Your Project

Even a simple AGENTS.md with your build commands, testing framework, coding conventions, and architectural decisions will dramatically improve agent output. Chapter 4 shows you exactly how. This takes 30 minutes and pays off on every single agent interaction going forward.

2. Write Tests Before Prompting

When you have tests, agents can self-correct. Without tests, you're the test suite. Chapter 5 walks through the TDD Agent Loop in detail. This single workflow change is responsible for the majority of productivity gains in my experience.

3. Spec Your Tasks

Before you prompt an agent, spend 5 minutes writing a clear specification. What should this code do? What are the constraints? What are the test scenarios? This small upfront investment saves 30+ minutes of review and rework on the other end.

These three practices, context files, tests first, and specs, are the foundation. Everything else in this book builds on them.

The 30-Day Horizon

Anthropic's report says developers now use AI in 60% of their work. That number is climbing. Within a year, developers who haven't adopted agentic workflows will be the exception, not the norm.

But here's the good news: this isn't a years-long transformation. The full book includes a 30-day plan (Chapter 29) that takes you from wherever you are today to a fully operational agentic workflow. The core skills, context engineering, specification writing, and the TDD agent loop, can be learned and practiced in weeks, not months.

You don't need to change everything at once. Start with the three practices above. Add context engineering depth as you go. Experiment with parallel agents when you're comfortable with single-agent workflows. Build toward the full agentic development lifecycle at your own pace.

The job has changed. But the new job is learnable, valuable, and honestly, more interesting than the old one. You make more decisions, solve harder problems, and ship more software. You just do less typing.

That's a trade worth making.

CHAPTER 4

The Agentic Engineering Mindset

Every developer who's spent more than a week with coding agents has had the same experience. You fire up Claude Code or Copilot agent mode, give it a task, and watch it produce something impressive. Then you give it a slightly different task, and it produces something terrible. Same tool, same model, same you. Different result.

The variable isn't the agent. It's the mental model you bring to the interaction.

This chapter is about the mental models that separate productive agentic developers from frustrated ones. Not tips and tricks. Not prompt templates. The underlying way you think about what an agent is, what it can do, and what your role becomes when you stop typing code and start directing its creation.

Get the mindset right, and the techniques in the rest of this book will click. Get it wrong, and no amount of tooling will save you.

The Junior Developer with Perfect Memory and Zero Judgment

Here's the most useful mental model for working with coding agents: think of them as a junior developer who has read every programming book, every Stack Overflow answer, every documentation page, every open-source repository on GitHub, and has perfect recall of all of it. But who has absolutely no judgment about what matters in your specific situation.

This isn't a metaphor. It's a description of what's actually happening. Large language models have been trained on essentially the entire public corpus of programming knowledge. They know every pattern, every framework, every idiom. They can write a binary search in 47 languages. They can implement OAuth 2.0 from memory. They can set up an ASP.NET Core Web API with Entity Framework, dependency injection, and middleware in seconds.

What they cannot do is understand why your team chose a specific architecture, why that particular database column is nullable, why the test suite skips integration tests in CI, or why the previous developer left that cryptic comment about not touching the payment service on Thursdays.

This distinction, between knowledge and judgment, is the foundation of everything else. Let's make it concrete.

What "Perfect Memory, Zero Judgment" Looks Like in Practice

Say you ask an agent to add a new endpoint to your ASP.NET Core API. It will produce something like this:

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;

    public OrdersController(IOrderService orderService)
    {
        _orderService = orderService;
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<OrderDto>> GetOrder(int id)
    {
        var order = await _orderService.GetByIdAsync(id);
        if (order is null)
            return NotFound();

        return Ok(order);
    }
}
```

That's clean, conventional, correct C#. The pattern is universal. But is it right for *your* project?

Maybe your team uses `Guid` for IDs, not `int`. Maybe you use the result pattern instead of returning `ActionResult` directly. Maybe your convention wraps responses in an envelope. Maybe you use `[ProducesResponseType]` attributes. Maybe you have a base controller that handles null checks. The agent doesn't know any of this unless you tell it.

The junior developer analogy holds perfectly here. A good junior developer fresh out of a bootcamp would write exactly this code. It's textbook correct and project-wrong. Your job is the same in both cases: provide context, set constraints, review the output.

The difference is that the agent is faster than any junior developer who ever lived, never gets tired, never gets defensive when you ask for changes, and can run your tests 50 times without complaining. That's a hell of a trade-off, if you know how to manage it.

The Director Mindset

If the agent is the junior developer, what are you? You're the director.

Not "director" as in corporate hierarchy. Director as in filmmaking. A film director doesn't operate the camera, design the sets, or act the scenes. A director sets the vision, establishes the constraints, defines what "good" looks like, and makes the calls that hold the whole thing together. The director is the person who ensures that a hundred talented specialists produce a coherent whole instead of a disjointed mess.

That's your new job. You set the vision (what we're building and why). You establish constraints (architecture, patterns, performance budgets, security requirements). You define the quality bar (what "done" means). And you make the judgment calls that the agent can't make for itself.

Vision: What and Why

Before you type a single prompt, you should be able to answer: what am I trying to build, and why does it matter? This sounds obvious, but watch how most developers interact with agents. They jump straight to "build me a REST API for orders." That's a what without a why.

The why matters because it constrains the design space. "Build me a REST API for orders" could mean a hundred different things. "We need an orders API that supports our mobile app's checkout flow, handles partial fulfillment, and needs to scale to 10,000 orders per hour during flash sales" narrows it to maybe three. The agent can't narrow the design space for you. That's a judgment call.

Constraints: The Guardrails

Here's a counterintuitive truth: agents produce better work with more constraints, not fewer. Giving an agent total freedom is like telling a junior developer "just build something." You'll get *something*. It probably won't be what you wanted.

Effective constraints for agents include:

- **Architectural boundaries.** "This is a vertical slice. Don't modify shared infrastructure."
- **Pattern requirements.** "Use the CQRS pattern with MediatR. Here's an existing handler as reference."
- **Performance budgets.** "This endpoint must respond in under 200ms at p99."
- **Security requirements.** "All inputs must be validated. Use FluentValidation. Here's our standard validator as an example."
- **What NOT to do.** "Don't add new NuGet packages without asking. Don't modify the database schema. Don't change the authentication middleware."

That last category, the "don'ts," is arguably the most important. Agents are enthusiastic. They'll install packages you don't want, refactor code you didn't ask them to touch, and make architectural decisions they're not qualified to make. Explicit boundaries prevent this.

This is why context files (AGENTS.md, CLAUDE.md, .cursorrules) matter so much. They're your standing constraints. Instead of repeating "use our repository pattern, not raw DbContext" in every prompt, you write it once in a context file and it applies to every interaction.

Quality Bar: What "Done" Means

The most underrated part of the director mindset is defining what "done" looks like before the work starts. For an agent, "done" should be specific and verifiable:

- All existing tests still pass
- New tests cover the happy path and at least two error cases
- The code follows existing patterns in the codebase

- No new compiler warnings
- No new security vulnerabilities (no hardcoded secrets, no SQL injection, no unvalidated input)

If you can't define "done" in concrete terms, the agent can't hit the target. And neither could a human, frankly. The director mindset forces a discipline that benefits your entire engineering practice, not just your agent interactions.

The Five Mental Traps

Knowing the right mindset is half the battle. The other half is recognizing the wrong ones. Here are the five mental traps that consistently derail developers working with agents.

Trap 1: Over-Trusting

This is the big one. The METR study found that experienced developers were 19% slower with AI tools but perceived themselves to be 20% faster. That 39-point gap between perception and reality is almost entirely explained by over-trust.

Over-trusting looks like this: the agent produces code that compiles, you glance at it, it looks reasonable, you accept it. An hour later you're debugging a subtle null reference exception because the agent didn't handle a case you assumed it would.

The fix isn't suspicion. It's verification. Treat agent output exactly like a pull request from a developer you've never worked with before. Read it. Question it. Run it. The test suite is your verification layer (which is why Chapter 7 on TDD with agents is the most important chapter in this book). But tests only catch what they test for. You still need to read the code.

A practical rule: the more complex the task, the more carefully you review. Agent-generated boilerplate? Skim it. Agent-generated business logic? Read every line. Agent-generated security code? Read it twice, then write a test for each edge case.

Trap 2: Under-Specifying

Under-specifying is the mirror of over-trusting. Where over-trusting is accepting bad output, under-specifying is providing bad input.

“Add pagination to the products endpoint” is under-specified. Which pagination strategy? Offset-based or cursor-based? What’s the default page size? What’s the maximum? What happens when someone requests page 10,000? What does the response envelope look like? Should it include total count (expensive for large datasets)?

Every question you don’t answer is an assumption the agent makes for you. Sometimes those assumptions are fine. Sometimes they’re catastrophic. The ratio depends on how much context the agent has about your project, but even with perfect context files, ambiguity in the task itself leads to ambiguity in the output.

The fix is simple in concept: write specs before prompts. A two-paragraph description of what you want, with explicit requirements and explicit non-requirements, will save you more time than any prompting technique. Simon Willison nailed this insight: “Code that started from your own specification is a lot less effort to review.” The spec does double duty. It guides the agent AND it gives you a checklist to review against.

Trap 3: Anthropomorphizing

“The agent understood what I wanted.” “The agent decided to refactor the service layer.” “The agent thought it would be better to use a different pattern.”

No. The agent didn’t understand, decide, or think anything. It generated the next most probable sequence of tokens given its training data and your input. This isn’t pedantry. It’s a practical distinction that changes how you work.

When you anthropomorphize the agent, you start treating it like a colleague with opinions and reasoning. You defer to its “decisions.” You assume its “choices” have rationale. You negotiate with it instead of instructing it.

Agents don’t have preferences. They have probability distributions. When an agent “chooses” to use one pattern over another, it’s because that pattern was more strongly associated with the context you provided. If you want a different pattern, don’t argue

with it. Change the context. Provide an example. Put it in your context file. The agent will follow whatever pattern has the strongest signal in its input.

This is liberating once you internalize it. You're not managing a personality. You're tuning a function. If the output is wrong, the input is wrong. Fix the input.

Trap 4: All-or-Nothing Thinking

Some developers treat agents as either magical or useless. The agent either handles the entire task autonomously or it's not worth using. This binary thinking misses the massive middle ground where agents deliver the most value.

The reality is a spectrum:

- **Fully autonomous:** Setting up boilerplate, writing tests for existing code, generating CRUD endpoints, formatting, linting fixes. Low risk, high confidence. Let the agent run.
- **Collaborative:** Implementing business logic, designing data models, writing complex queries. Medium risk, medium confidence. Work iteratively, review each step.
- **Human-led, agent-assisted:** Architecture decisions, security-critical code, performance-critical algorithms, cross-cutting concerns. High risk, low agent confidence. You drive, agent suggests.
- **Human-only:** Production incident triage, interpersonal decisions about code review feedback, understanding why a system was built a certain way. No agent involvement.

The best developers fluidly move between these modes, sometimes within a single task. They'll let the agent scaffold a new service (fully autonomous), then collaborate on the business logic (iterative), then take the wheel for the authentication layer (human-led), then hand back to the agent for writing tests (autonomous again).

This fluidity comes from accurate calibration, which brings us to the last trap.

Trap 5: Static Calibration

Your expectations for an agent should change based on the task, the domain, and the context available. Most developers develop a single mental model of “how good” agents are and apply it everywhere. This leads to over-trusting in hard domains and under-using in easy ones.

Calibrate dynamically:

High agent confidence (trust more): - Well-defined, pattern-matching tasks (CRUD, DTOs, mappers) - Tasks with strong conventions (REST endpoints, EF Core configurations) - Code with comprehensive tests as a safety net - Widely-documented technologies (ASP.NET Core, Entity Framework, xUnit)

Low agent confidence (verify more): - Novel business logic specific to your domain - Performance-sensitive code - Security-critical flows (authentication, authorization, payment processing) - Anything requiring understanding of undocumented system behavior - Tasks involving coordination across many services

Near-zero agent confidence (do it yourself or supervise closely): - Distributed system debugging - Data migration scripts for production - Anything where “close enough” has real consequences

This calibration should be a conscious, explicit step before you start any agent interaction. Ask yourself: “On a scale from boilerplate to brain surgery, where does this task fall?” Then set your review intensity accordingly.

Calibrating Expectations by Task Type

Let’s make this concrete with common development tasks and realistic expectations for what agents can and can’t do well.

Greenfield Scaffolding

Agent capability: Excellent. This is where agents shine brightest. Creating a new project, setting up folder structure, wiring up dependency injection, configuring

middleware, adding initial endpoints. An agent can do in 2 minutes what would take you 30.

Your role: Provide the architectural blueprint. What patterns, what folder structure, what packages, what configuration. Review the output for convention violations.

Watch out for: Agents will happily generate project structures that don't match your team's conventions. Always provide a reference (existing project, context file, or explicit spec).

Bug Fixes

Agent capability: Good, with caveats. Agents excel at "here's the error message, find and fix the bug" scenarios, especially when the error is clear and the fix is localized. They struggle with bugs that involve subtle state interactions, timing issues, or require understanding of runtime behavior that isn't visible in the code.

Your role: Reproduce the bug first. Give the agent the error message, the steps to reproduce, and the relevant code. Verify the fix doesn't introduce regressions.

Watch out for: Agents sometimes fix the symptom, not the cause. They'll add a null check instead of fixing why the value is null. Always ask "why was this null in the first place?" before accepting a null check as a fix.

Refactoring

Agent capability: Good for mechanical refactoring, poor for architectural refactoring. Renaming, extracting methods, converting to async/await, updating to new API versions: agents handle these well. Redesigning a service boundary, splitting a monolith, changing data flow patterns: these require judgment the agent doesn't have.

Your role: For mechanical refactoring, define the transformation clearly and let the agent run. For architectural refactoring, break it into mechanical steps yourself, then let the agent execute each step.

Test Writing

Agent capability: Excellent. This might be the single highest-value use case. Agents are remarkably good at generating test cases, especially when they can see the implementation and your existing test patterns. They're particularly good at generating edge cases you wouldn't have thought of.

Your role: Provide examples of your test style. Specify what testing framework you use. Review the generated tests for meaningful assertions (agents sometimes write tests that pass but don't actually verify anything useful).

Performance Optimization

Agent capability: Poor to moderate. Agents can suggest common optimizations (caching, async I/O, reducing allocations), but they can't profile your system, understand your actual bottlenecks, or reason about the performance characteristics of your specific deployment. They optimize by pattern matching against known optimization strategies, not by analysis.

Your role: Profile first. Identify the bottleneck. Then ask the agent to implement a specific optimization strategy you've chosen. Don't ask "make this faster." Ask "convert this to use `ArrayPool<byte>` to reduce GC pressure."

Putting It Together: The Pre-Task Checklist

Before starting any agent-assisted task, run through this mental checklist:

1. **What am I building?** Can I describe it in two sentences?
2. **What does "done" look like?** What tests should pass? What behavior should change?
3. **Where does this task fall on the autonomy spectrum?** Boilerplate? Business logic? Security-critical?

4. **What constraints apply?** Architectural boundaries, patterns, security requirements, performance budgets?
5. **What context does the agent need?** Existing code examples, documentation, specifications?
6. **What should the agent NOT do?** What files shouldn't it touch? What packages shouldn't it add?

This takes maybe 90 seconds. It will save you 30 minutes of debugging bad output or reworking code that went in the wrong direction.

The Mindset Is the Multiplier

Here's the thing that surprised me most about writing this chapter: almost none of this is specific to AI agents. Writing clear specs, defining "done" criteria, calibrating review intensity based on risk, establishing constraints before work begins: these are just good engineering practices.

Addy Osmani made this observation and it's worth repeating: "AI-assisted development actually rewards good engineering practices MORE than traditional coding does." The agentic engineering mindset isn't a new discipline bolted onto software engineering. It's software engineering with the feedback loops tightened and the rewards for discipline amplified.

The developers who struggle with agents aren't struggling because agents are bad. They're struggling because agents expose sloppy habits that were previously hidden by the slow pace of manual coding. When you write code by hand, you can hold a vague spec in your head and make judgment calls as you go. When an agent writes code, those judgment calls happen silently, based on probability distributions, not understanding. The vague spec that "worked fine" when you were coding manually produces garbage when an agent interprets it.

The mindset shift isn't about learning to work with a new tool. It's about becoming a better engineer. The agents just make the difference between good and sloppy engineering impossible to ignore.

Think of it this way: agents are a 10x amplifier. They amplify good process into great results. They amplify bad process into spectacular failure. The developers in the METR study who got 19% slower? They were amplifying habits that worked at human speed but collapsed at agent speed. The developers who report 2-3x productivity gains? They were already disciplined. The agent just removed the bottleneck of manual typing from their workflow.

The rest of this book gives you the specific techniques. But those techniques only work if you bring the right mindset to them. Be the director, not the typist. Treat agents as tireless junior developers, not as omniscient colleagues. Define what “good” looks like before you start. Verify everything. And calibrate your trust to the task at hand.

The tools will keep getting better. Models will improve. Context windows will grow. New agent frameworks will emerge. But the mindset, the fundamental way you think about directing autonomous systems, will remain the foundation of everything.

Build that foundation solid, and everything else gets easier.

CHAPTER 5

From Vibe Coding to Vibe Engineering

In February 2025, Andrej Karpathy posted a tweet that named something everyone was already doing: “I just mass mass mass mass prompt, write barely any code. Sometimes I don’t even read the diffs. I just accept them.” He called it “vibe coding.” The term went viral because it was honest. Millions of developers were doing exactly this and nobody had a word for it.

One year later, in February 2026, Karpathy posted again. This time, the term was “agentic engineering.” He described it as “a serious engineering discipline involving autonomous agents.” Same person, same field, twelve months apart. The progression from “fully give in to the vibes” to “serious engineering discipline” tells you everything about how fast this field matured.

This chapter traces that evolution. Not as history for its own sake, but because understanding the spectrum from vibe coding to agentic engineering tells you which mode to use when. And the answer is: all of them, depending on what you’re building.

What Karpathy Actually Said

Let’s be precise about vibe coding, because the term got distorted as it spread. Karpathy’s original description was specific: “I just see things, say things, run things, and copy-paste things, and it mostly works.” The key phrase was “forget that the code even exists.”

This wasn’t laziness. Karpathy is one of the most technically capable people on the planet, founding member of OpenAI, former head of AI at Tesla. He was describing a

genuinely new interaction model. For certain tasks (quick scripts, prototypes, one-off tools), you could describe what you wanted in natural language, let the AI generate code, run it, see if it worked, and iterate. You didn't need to understand the code at a line-by-line level. You just needed to evaluate the output.

The context matters too. In early 2025, the tools were chat-based. You described what you wanted. The AI generated code. You pasted it into your editor. You ran it. If it failed, you pasted the error back. This loop was clunky but surprisingly effective for small, self-contained tasks.

Karpathy was explicit about the scope: "It's not really coding. I just see stuff, say stuff, run stuff." He positioned it as something different from professional software engineering. A new way of interacting with computers that happened to produce code as a side effect.

What Vibe Coding Got Right

Let's give vibe coding its due. It identified something genuinely important that the professional software engineering community was slow to acknowledge.

The Accessibility Revolution

Before vibe coding, if you wanted a custom script to rename 10,000 files according to a specific pattern, you needed to know Python or Bash or PowerShell. After vibe coding, you needed to describe the pattern in English. This is a real, meaningful expansion of who can create software. Researchers, designers, marketers, small business owners: people who were previously locked out of automation gained access to it.

This wasn't a toy. Non-engineers started building real tools. Internal dashboards. Data processing pipelines. Workflow automations. Anthropic's 2026 report identifies "non-engineers embrace agentic coding" as one of its eight major trends, noting that sales, legal, and marketing teams are now building their own tools.

The Prototyping Speed

For exploratory work, vibe coding was and remains blazingly fast. When you're trying to figure out if an idea even works, the overhead of writing specs, setting up project structure, configuring CI/CD, and writing tests is counterproductive. You need a quick-and-dirty prototype that lets you evaluate the concept. Vibe coding delivers that in minutes instead of days.

I still use vibe coding for this exact purpose. When I want to test whether a particular API integration is feasible, I don't need production-quality code. I need code that runs, calls the API, and shows me the response shape. Vibe coding gets me there in five minutes.

The Creative Energy

This is the most underappreciated contribution of vibe coding. It reintroduced *play* into programming. The rapid feedback loop of "describe, generate, run, see" has an almost game-like quality. You're experimenting, iterating, exploring possibilities at a pace that traditional development doesn't allow. That creative energy is valuable. It shouldn't be discarded just because it doesn't produce production-quality code.

What Vibe Coding Got Wrong

Vibe coding had a fatal flaw, and it was right there in Karpathy's original description: "forget that the code even exists."

For prototypes, that's fine. For anything that persists, that's a disaster.

No Verification

The core of vibe coding was "run it and see if it works." But "works" meant "produces visually correct output for the one case I tested." It didn't mean correct, secure, performant, or maintainable. It meant "doesn't crash on the happy path."

This is how vibe-coded projects accumulate debt at staggering speed. The code works for the cases you tried. The cases you didn't try are time bombs. And because you

“forgot the code exists,” you don’t even have a mental model of where the bombs are planted.

GitClear’s research found a 4x growth in code clones with AI assistants. Code clones are a proxy for copy-paste development, taking generated code without understanding it. That’s vibe coding at scale.

No Structure

Vibe-coded projects tend to grow into unmaintainable tangles because nobody made architectural decisions. The AI generated code that worked, and that code accumulated. There’s no separation of concerns because nobody specified one. There’s no consistent error handling because nobody defined a strategy. There are three different ways to call the database because each prompt got a different response from the model.

This is fine for a 200-line script. It’s catastrophic for a 20,000-line application. And the transition from 200 lines to 20,000 lines happens faster than you’d think when an agent is generating code.

No Sustainability

The final failure of pure vibe coding is that it produces code that only the original prompter can work with, and even they can’t work with it a month later. There are no comments (because the human didn’t write the code and didn’t review it closely enough to add comments). There are no tests (because tests are overhead that slows down the vibes). There’s no documentation (because the prompts that generated the code aren’t saved alongside it).

Try handing a vibe-coded project to a new developer. Or try coming back to your own vibe-coded project after three months. You’ll find yourself re-prompting from scratch because the existing code is inscrutable, even though (especially because) you didn’t read it carefully the first time.

Simon Willison’s Intervention: “Vibe Engineering”

In October 2025, Simon Willison published a piece that reframed the entire conversation. He’d been one of the most prolific users of AI coding tools, running multiple agents in parallel daily, and he had strong opinions about what was working and what wasn’t.

His key move was reclaiming the word “vibe” while adding engineering discipline to it. Vibe engineering kept the creative energy, the rapid iteration, the natural language interaction model. But it added the things that pure vibe coding lacked: verification, structure, and sustainability.

The Willison Method

Willison’s workflow, as described across his blog posts and conference talks, looked like this:

1. **Start with a spec.** Not a formal PRD. A description of what you want, written in your own words, specific enough that you could review the output against it. “Code that started from your own specification is a lot less effort to review.”

2. **Use agents, not chat.** By late 2025, Willison had moved from chat-based coding to Claude Code and Codex CLI: agents that could read his codebase, run tests, and iterate. The agent didn’t just generate code. It operated within the project.

3. **Run agents in parallel.** His “parallel coding agent lifestyle” involved 3-5 agents simultaneously, each on a different task in a different git worktree. One researching a library, one implementing a feature, one fixing bugs. He orchestrated and reviewed.

4. **Review everything.** Not at the diff level for every change, but at the outcome level for every task. Does it work? Does it match the spec? Does it follow conventions? Are there tests?

5. **Maintain context.** His projects had context files that told agents about conventions, patterns, and constraints. Each agent interaction benefited from this accumulated context.

The “vibe” in vibe engineering was the creative flow, the willingness to explore, the natural language interaction. The “engineering” was everything else: specs, tests, reviews, structure, context management.

Why This Mattered

Willison’s framing mattered because it created permission to enjoy the process while being disciplined about the output. The vibe coding camp said “just let go and trust the AI.” The traditional engineering camp said “AI-generated code is dangerous, review everything manually.” Vibe engineering said “both of you are half right.”

You can enjoy the creative flow of rapid AI-assisted development AND maintain engineering standards. These aren’t in tension. The standards actually enable the flow by catching mistakes early and giving you confidence to move fast.

The Spectrum

By early 2026, the field had settled into a spectrum with four distinct modes. Each has legitimate uses. The skill isn’t committing to one mode. It’s knowing which mode fits the situation.

Mode 1: Pure Vibe Coding

What it is: Prompt, generate, run, iterate. Don’t read the code. Evaluate the output.

When it’s appropriate: - One-off scripts you’ll use once and delete - Rapid prototyping to test if an idea is feasible - Personal tools that only you will use and that don’t handle sensitive data - Learning and exploration (“show me how webhooks work in .NET”) - Hackathons and time-boxed experiments

When it’s dangerous: - Anything that goes to production - Anything that handles user data - Anything other people will maintain - Anything that needs to work in six months

Realistic assessment: Maybe 10-15% of your development work can safely be pure vibe coding. That's higher than the traditional engineering camp would admit and lower than the vibe coding evangelists claim.

Mode 2: Vibe Engineering

What it is: The Willison method. Creative flow plus engineering discipline. Write specs, use agents, review outcomes, maintain context. You still use natural language as your primary interface, but you verify the results against defined criteria.

When it's appropriate: - Feature development where you have clear requirements - Building new services or modules within an existing architecture - Refactoring with defined before/after states - Any task where you can write a spec and define "done"

When it's not enough: - Performance-critical systems where you need to understand every instruction - Security-sensitive code that requires formal review - Distributed systems debugging - Anything where the cost of a subtle bug is very high

Realistic assessment: This is the sweet spot for maybe 50-60% of professional development work. Most feature development, most bug fixes, most incremental work fits comfortably in this mode.

Mode 3: Agentic Engineering

What it is: The full discipline. Context engineering, TDD agent loops, comprehensive test suites, formal review processes, architectural specifications, multi-agent orchestration. You're directing a team of agents the way a senior engineer directs a team of developers.

When it's appropriate: - Production systems - Anything that needs to scale - Security-sensitive features - Performance-sensitive features - Projects with multiple contributors - Long-lived codebases

When it's overkill: - Quick experiments - Internal tools with a single user - Throwaway prototypes

Realistic assessment: About 25-30% of your work warrants full agentic engineering discipline. But that 25-30% is the work that matters most: the production code, the systems that users depend on, the features that generate revenue.

Mode 4: Traditional Engineering

What it is: You write the code. By hand. With full understanding of every line.

When it's still necessary: - Novel algorithms where correctness is paramount - Performance-critical hot paths (the inner 1% of code that handles 99% of load) - Code that must be formally verifiable - Debugging distributed system failures where you need to reason about state across time - Extremely domain-specific logic where the agent doesn't have relevant training data

Realistic assessment: Probably 5-10% of your work. Shrinking, but not zero. And that's fine. The goal isn't to eliminate manual coding. It's to reserve your manual coding effort for the tasks where human judgment is genuinely irreplaceable.

Knowing Which Mode to Use

The choice of mode isn't a personality preference. It's a risk/value calculation. Here's the decision framework:

What's the blast radius if this code is wrong? - Crash in production affecting users? Agentic engineering or traditional. - Bug in an internal tool that someone might notice? Vibe engineering. - Wrong output in a script you'll run once? Vibe coding.

How long will this code live? - Months or years? Agentic engineering. - Weeks? Vibe engineering. - Hours? Vibe coding.

Who else will touch this code? - Your team? Agentic engineering. - Just you? Vibe engineering or vibe coding. - Nobody (throwaway)? Vibe coding.

How well-defined is the task? - Clear spec with acceptance criteria? Agentic engineering. - Rough idea that needs exploration? Vibe engineering. - "I wonder if this is possible"? Vibe coding.

Most developers should be spending the majority of their time in the vibe engineering and agentic engineering modes. Pure vibe coding is for exploration. Traditional hand-coded engineering is for the critical paths. The middle two modes are where the bulk of productive work happens.

The Maturation of a Field

The progression from vibe coding to agentic engineering mirrors a pattern we've seen before in technology. Every significant shift goes through the same phases: novelty, hype, disillusionment, and maturation.

Phase 1: Novelty (Early 2025)

"Look what this can do!" Developers shared screenshots of AI-generated apps. Karpathy named the vibe. Everyone was amazed that you could describe a feature and get working code. The focus was on the capability, not the quality.

Phase 2: Hype (Mid 2025)

"This changes everything!" Breathless predictions that developers would be obsolete. LinkedIn posts about "10x" productivity. Companies mandating AI tool adoption. The discourse outpaced the reality.

Phase 3: Disillusionment (Late 2025)

The METR study dropped in July 2025 like a bomb: experienced developers were 19% slower with AI tools. GitClear published data showing 4x growth in code clones. Production incidents caused by unreviewed AI-generated code made the news. The backlash began. "AI coding tools are overrated." "They make developers sloppy." "The productivity gains are illusory."

The disillusionment was healthy. It forced honesty about what the tools could and couldn't do. It separated the "AI will write all our code" fantasy from the reality that AI tools require discipline to use effectively.

Phase 4: Maturation (2026)

Now we're here. The hype has burned off. The skepticism has been addressed. What's left is a genuine engineering discipline. The tools are better (agents, not just chat). The practices are defined (context engineering, spec-first development, TDD agent loops). The expectations are calibrated (real productivity gains, but only with disciplined use).

Karpathy's terminology evolution captured this perfectly. "Vibe coding" was Phase 1/2. "Agentic engineering" is Phase 4. Same person, same tools, different understanding of how to use them well.

The METR Study: What It Really Tells Us

The METR study deserves deeper attention because it's been cited by both sides (AI skeptics and AI advocates) to support their positions. Both readings are wrong.

The study found that experienced open-source developers were 19% slower using AI coding tools on tasks in codebases they were already deeply familiar with. The developers believed they were 20% faster. That's a 39-percentage-point gap between perception and reality.

The skeptic reading: "See? AI tools make you slower. Stop using them."

The advocate reading: "That study was flawed. The tasks were unusual. The developers didn't use the tools properly."

The accurate reading: AI tools make you slower when you use them naively on tasks where your existing expertise already makes you fast. The overhead of prompting, reviewing, and correcting AI output exceeds the time savings from generating code when you could have written the code faster yourself.

This is the argument for everything in this book. Naive AI tool use hurts. Disciplined agentic development helps. The gap between the two is the gap between the METR

study's 19% slowdown and the reports of 2-3x productivity gains from practitioners like Willison who use structured workflows.

The METR finding isn't an argument against AI tools. It's an argument against using AI tools without a methodology. It's an argument for this book.

What Changes Next

The spectrum from vibe coding to agentic engineering isn't static. It's shifting toward more agent autonomy as models improve. Tasks that required full agentic engineering discipline in 2025 (comprehensive context files, heavy review, tight test loops) might need only vibe engineering by 2027 as models get better at inference, context management, and self-verification.

But the fundamental dynamic won't change: there will always be tasks where you can trust agents more and tasks where you need to verify more. The spectrum won't collapse. It will shift.

What will change:

The "vibe coding" zone will expand. As models improve at self-verification and error detection, more tasks will be safe to handle with minimal review. Internal tools, scripts, prototypes, these will increasingly be fire-and-forget.

The "agentic engineering" zone will move up the complexity stack. Full agent orchestration with comprehensive testing and review will handle increasingly complex tasks. Things that require traditional hand-coding today (performance-critical paths, security-sensitive features) will move into the agentic engineering zone.

The "traditional" zone will shrink but never disappear. Some tasks will always require human judgment applied directly to code. Novel algorithm design. Debugging emergent behavior in distributed systems. Making the call about whether to ship or hold.

The developers who thrive will be the ones who can move fluidly across the spectrum, matching their approach to the task. Not zealots for any one mode. Pragmatists who

use every tool available, including the oldest tool of all: their own brain, their own keyboard, their own judgment.

The Choice

Here's where this gets personal. You're reading this book because you want to get better at building software with AI agents. The spectrum I've described isn't just a framework for choosing tools. It's a mirror for choosing who you want to be as an engineer.

Pure vibe coders are already hitting a ceiling. The prototypes are impressive. The production code is a mess. The technical debt accumulates silently until it doesn't.

Traditional holdouts are being outpaced. They write better code line-by-line, but they ship less of it. In a world where an agent can produce a decent first draft in seconds, spending an hour hand-crafting the same code is a competitive disadvantage for tasks that don't require that level of care.

The sweet spot, the place where careers are being built and companies are gaining advantage, is in the middle of the spectrum. Vibe engineering for the bulk of your work. Full agentic engineering for the critical paths. Vibe coding for exploration. Traditional engineering for the irreducible core that requires human judgment.

This is what the rest of the book teaches you to do. Not just which tools to use, but how to think about the work so that you're applying the right level of discipline to each task. The mindset from Chapter 4, the spectrum from this chapter: together they form the foundation for everything that follows.

The field matured fast. Twelve months from "forget the code exists" to "serious engineering discipline." The developers who matured with it are building things that weren't possible two years ago, faster than was possible one year ago, at a quality level that matches or exceeds what they could achieve by hand.

That's not hype. That's the new baseline. Let's build on it.

PART II

Context Engineering

CHAPTER 6

Writing Effective Context Files

If you take one thing from this entire playbook, make it this: **context engineering is the single most important skill in agentic development.** Not prompting. Not tool selection. Not model choice. Context.

Bharani Subramaniam from ThoughtWorks defined it simply: “Context engineering is curating what the model sees so that you get a better result.” That sounds abstract. In practice, it’s concrete and mechanical. It’s the set of files, rules, and conventions that tell your agent how to work with your specific project.

The developers who are 2-3x more productive with agents all have one thing in common: they’ve invested in context. They didn’t just install the tool and start prompting. They built an information layer that makes every agent interaction better.

This chapter gives you the system. Templates, examples, and the reasoning behind every decision.

The Context Stack

Modern AI coding agents load context from multiple layers. Understanding these layers tells you where to invest your effort.

Layer	What It Does	When It Loads	Your Time Investment
AGENTS.md / CLAUDE.md	Always-on project guidance	Automatically, every session	High (create once, maintain ongoing)
Path-scoped rules	Modular guidance for specific file types	When matching files are open	Medium (add as patterns emerge)
Skills	On-demand instructions and scripts	When the agent decides they're relevant	Low to medium
MCP servers	External tool and data access	When the agent needs external info	Setup cost, then low
The codebase itself	Your existing code IS context	Always	Ongoing (it's your normal work)
Test suite	Automated verification of agent output	When agent runs tests	High (but you should have this anyway)

The highest-leverage investment is the first row: your main context file. Everything else builds on it.

The Main Context File: AGENTS.md

There's an emerging standard called AGENTS.md (documented at agents.md). GitHub Copilot uses `copilot-instructions.md`. Cursor uses `.cursorrules`. Claude Code uses `CLAUDE.md`. The format varies but the concept is identical: a markdown file at the root of your project that tells agents how to work here.

I recommend using AGENTS.md because it’s the most tool-agnostic, but the content matters more than the filename. Write it once and symlink or copy to tool-specific filenames if you need to.

What Goes In It

Your context file answers one question: “What does an agent need to know to write good code in this project?” Not everything about the project. Just the things that affect code quality.

Here’s the structure I use, section by section.

Template: The Complete AGENTS.md

```
# AGENTS.md

## Project Overview
Brief description of what this project does. 2-3 sentences max.
Not a README. Just enough for the agent to understand scope.

## Tech Stack
- Runtime: .NET 9 / ASP.NET Core
- Database: PostgreSQL via EF Core
- Cache: Redis via IDistributedCache
- Message queue: RabbitMQ via MassTransit
- Auth: JWT bearer tokens with ASP.NET Identity
- Testing: xUnit + FluentAssertions + NSubstitute
- CI: GitHub Actions

## Build & Run
```bash
# Restore and build
dotnet build

# Run tests
dotnet test

# Run the API (requires Docker for dependencies)
docker compose up -d
dotnet run --project src/Api
```
```

Project Structure

```
src/
  Api/              # ASP.NET Core Web API (entry point)
  Domain/           # Domain entities and interfaces
  Infrastructure/   # EF Core, external services, repositories
tests/
  Api.Tests/        # Integration tests
  Domain.Tests/     # Unit tests
```

Coding Conventions

- Use file-scoped namespaces
- Use primary constructors for DI
- Records for DTOs, classes for entities
- No regions. Ever.
- Keep methods under 30 lines. Extract if longer.
- Async all the way down. No .Result or .Wait().

Architecture Patterns

- Vertical slice architecture: each feature is a self-contained folder
- CQRS with MediatR: commands and queries as separate types
- Repository pattern for data access (no direct DbContext in handlers)
- Result pattern for error handling (no exceptions for business logic)

Error Handling

- Return Result from handlers, not exceptions
- Map Result failures to HTTP status codes in the controller
- Use ProblemDetails for all error responses
- Log errors in middleware, not in handlers

Testing Conventions

- One test class per feature/handler
- Use the builder pattern for test data (see tests/Builders/)
- Integration tests use WebApplicationFactory with a real database
- Name tests: MethodName_Scenario_ExpectedResult

What NOT to Do

- Do NOT use AutoMapper (we use manual mapping methods)
- Do NOT add NuGet packages without asking
- Do NOT modify the middleware pipeline in Program.cs without asking
- Do NOT use static methods for business logic
- Do NOT write console output (use ILogger)

API Conventions

- RESTful routes: /api/v1/{resource}
- Use [FromRoute], [FromQuery], [FromBody] explicitly

- Return ActionResult from controllers
- POST returns 201 with Location header
- All endpoints require [Authorize] unless explicitly noted

```
That's a real context file. It's opinionated, specific, and actionable. An agent reading this f

### Why Each Section Matters

**Project Overview:** Agents make better decisions when they understand what they're building. 

**Tech Stack:** Prevents the agent from suggesting libraries you don't use. Without this, agent

**Build & Run:** Agents (especially CLI agents like Claude Code) can execute these commands. If

**Project Structure:** Agents need to know where to put new files. Without this, they'll create

**Coding Conventions:** This is where your context file pays the highest dividends. Every conve

**Architecture Patterns:** Prevents the agent from fighting your architecture. If you use CQRS 

**What NOT to Do:** Arguably the most important section. Agents are biased toward the most comm

## Path-Scoped Rules

Your main context file handles project-wide conventions. Path-scoped rules handle the specifics

In Claude Code, these go in `.claude/rules/` with glob patterns. In Cursor, they go in `.cursor

### Example: Controller Rules

File: `.claude/rules/controllers.md`
Applies to: `src/Api/Controllers/**`

```markdown
# Controller Conventions

Controllers are thin. They do three things:
1. Validate the request (model binding + FluentValidation)
2. Send a command/query via MediatR
```
```

```
3. Map the result to an HTTP response

No business logic in controllers. No direct database access.
No try/catch (the global exception middleware handles errors).

## Template
```csharp
[ApiController]
[Route("api/v1/[controller]")]
[Authorize]
public class OrdersController(ISender sender) : ControllerBase
{
    [HttpGet("{id:guid}")]
    public async Task<ActionResult<OrderResponse>> Get(Guid id)
    {
        var result = await sender.Send(new GetOrderQuery(id));
        return result.Match<ActionResult<OrderResponse>>(
            success => Ok(success),
            notFound => NotFound()
        );
    }
}
```
```

Example: Test Rules

File: `.claude/rules/tests.md`
Applies to: `tests/**`

```markdown
# Testing Conventions

Use xUnit, FluentAssertions, NSubstitute. No other test libraries.

## Unit Tests
- Test one behavior per test method
- Use Arrange/Act/Assert structure (with blank line separators)
- Use NSubstitute for mocking interfaces, never concrete classes
- Test name format: MethodName_Scenario_ExpectedResult

## Integration Tests
- Use WebApplicationFactory<Program>
- Each test class creates its own test database
- Seed data using builder pattern (see tests/Builders/)
- Always dispose/clean up database after test class

## Example
```csharp
public class CreateOrderHandlerTests
{
    private readonly IOrderRepository _repository = Substitute.For<IOrderRepository>();
    private readonly CreateOrderHandler _handler;

    public CreateOrderHandlerTests()
    {
        _handler = new CreateOrderHandler(_repository);
    }

    [Fact]
```
```

```
    public async Task Handle_ValidOrder_ReturnsCreatedResult()
    {
        // Arrange
        var command = new CreateOrderCommand("SKU-001", 2);

        // Act
        var result = await _handler.Handle(command, CancellationToken.None);

        // Assert
        result.IsSuccess.Should().BeTrue();
        await _repository.Received(1).AddAsync(Arg.Any<Order>());
    }
}
```

```
### When to Add Path-Scoped Rules

Don't create them all upfront. Add them reactively:

1. Agent writes code in a specific area that doesn't follow your conventions
2. You correct it during review
3. You add a rule so it doesn't happen again

This is the "build context files gradually" principle that experts recommend. Start with the ma

## Context File Anti-Patterns

I've seen developers torpedo their own context engineering in a few predictable ways.

### The Novel

A 2,000-line context file that documents every class, every method, every database table. This

**Rule of thumb:** Your AGENTS.md should be under 200 lines. If it's longer, you're documenting

### The Museum

A context file that was written once and never updated. Your project evolves. Your conventions

**Practice:** Review your context file every two weeks. Does it still reflect how you actually

### The Wishlist

A context file full of aspirational conventions that the codebase doesn't actually follow. "All

**Rule:** Your context file should document what IS, not what you wish it were. If you want to

### The Secret Keeper

A context file that omits the negative constraints. What the agent should NOT do is often more
```

```
## Making Your Codebase AI-Readable

Your context file is only part of the equation. Your codebase itself is context, and how you st

### Clear Naming

Agents don't have the team knowledge that tells you "OrderProcessor" actually handles payments.

This was always a best practice. With agents, it's a force multiplier.

### Small, Focused Files

Agents work better with small files that do one thing. A 2,000-line God class is hard for an ag

Again, this was always good engineering. Agents just make the payoff more obvious.

### Documented Decisions

When you make a non-obvious architectural decision, document it. Not in a wiki that nobody read

```csharp
// DECISION: We use raw SQL for this query instead of LINQ because
// EF Core generates a suboptimal execution plan for this specific
// join pattern. See ADR-007 for benchmarks.
```

That comment saves the agent from “helpfully” converting your raw SQL to LINQ.

Comprehensive Test Suite

Your test suite is the most powerful form of context you have. It tells the agent:

- What the code is supposed to do (specification)
- What edge cases matter (boundary conditions)
- What patterns to follow (test structure and conventions)
- Whether its changes work (automated verification)
```

A codebase with 80%+ test coverage is dramatically easier for agents to work with than one with 10% coverage. The full book includes a chapter on testing strategy for agentic development, but the short version is: invest in tests now. They pay for themselves many times over in the agent era.

The Context Engineering Workflow

Here's the practical workflow for building and maintaining your context layer:

Week 1: The Foundation

1. Create AGENTS.md with the template above, adapted to your project
2. Fill in every section honestly (what IS, not what you wish)
3. Start your next coding task using an agent with this context file
4. Note where the agent still gets it wrong

Week 2: The Refinement

1. Add path-scoped rules for the areas where the agent struggled
2. Add "What NOT to Do" entries for every convention the agent violated
3. Review and tighten your AGENTS.md based on the first week's experience
4. Start writing test-first for agent tasks (preview of Chapter 5)

Week 3: The Expansion

1. Add rules for remaining code areas (frontend, infrastructure, etc.)
2. Document architectural decisions that agents keep getting wrong
3. Add examples (like the controller template above) for complex patterns
4. Share the context files with your team for feedback

Week 4: The Habit

1. Context file updates become part of your PR process
2. When you change a convention, update the context file in the same PR
3. Review the full context layer: is anything stale? Redundant? Missing?
4. Measure: are agent outputs getting better over time?

By the end of this process, you'll have a context layer that makes agent interactions productive by default. New team members (human or AI) can read your context files and immediately produce code that fits.

A Real Example

Here's a condensed example of how context engineering transforms agent output. Same task, same agent, different context.

Without context file:

Prompt: "Add an endpoint to get a user's order history"

Agent produces: a controller with direct DbContext access, AutoMapper for mapping, synchronous calls, no authentication, returns a List instead of paginated results.

You spend 20 minutes rewriting it to fit your architecture.

With context file:

Same prompt: "Add an endpoint to get a user's order history"

Agent produces: a controller that sends a GetOrderHistoryQuery via MediatR, a handler that uses the repository pattern, manual mapping, async throughout, [Authorize] attribute, paginated response with proper HTTP headers.

You spend 3 minutes reviewing and approve it.

The difference is 17 minutes on a single task. Multiply that by every agent interaction across a day, a week, a month. Context engineering doesn't just help. It compounds.

Key Takeaways

1. **Create an AGENTS.md now.** Use the template. Adapt it. It takes 30 minutes and pays off immediately.
2. **Document conventions, not code.** Your context file tells the agent how to work, not what the code does.
3. **Negative constraints are critical.** What NOT to do is as important as what to do.
4. **Build context gradually.** Start with the main file. Add scoped rules as patterns emerge.
5. **Your codebase is context.** Clean naming, small files, documented decisions, and comprehensive tests make everything better.
6. **Maintain it.** A stale context file is worse than no context file because it produces confidently wrong output.

Context engineering is the skill that separates developers who are slower with AI from developers who are dramatically faster. Invest in it. Everything else in this book builds on this foundation.

CHAPTER 7

Path-Scoped Rules

Your main context file handles the broad strokes. It tells agents how your project works, what conventions to follow, what patterns to use. But projects aren't monoliths. Your controllers have different conventions than your domain models. Your test files follow different rules than your migration scripts. Your frontend components live in a different world than your backend services.

Path-scoped rules solve this. They're modular, targeted instructions that only activate when an agent is working with matching files. Think of them as surgical context: the right guidance, at the right time, for the right code.

Beyond the Main Context File

Chapter 6 gave you the AGENTS.md foundation. It works. But there's a ceiling to what a single file can do effectively.

Consider a typical .NET project. Your controllers should be thin wrappers around MediatR. Your domain entities should never reference infrastructure concerns. Your EF Core migrations should follow specific naming and ordering conventions. Your React components (if you have a frontend) use a completely different set of conventions.

Putting all of this in AGENTS.md creates the "Novel" anti-pattern from Chapter 6: a bloated file that degrades model performance through sheer volume. Path-scoped rules let you keep each concern separate, loading only what's relevant.

The principle is simple: **give the agent the minimum context it needs for the specific file it's editing.**

How Path-Scoped Rules Work

Claude Code: .claude/rules/

Claude Code stores path-scoped rules in the `.claude/rules/` directory. Each rule is a markdown file with a description and glob patterns that determine when it activates.

The structure looks like this:

```
.claude/
  rules/
    controllers.md
    tests.md
    migrations.md
    frontend.md
    domain.md
```

Each file contains a YAML frontmatter block specifying when it applies, followed by the actual instructions:

```
---
description: Conventions for API controllers
globs: src/Api/Controllers/**/*.cs
---

Controllers are thin. Three responsibilities only:
1. Validate the request
2. Send a command/query via MediatR
3. Map the result to an HTTP response

No business logic. No direct database access. No try/catch blocks.
```

When an agent opens or modifies a file matching `src/Api/Controllers/**/*.cs`, these rules load automatically. When it's editing a test file, the controller rules stay dormant. The agent sees only what it needs.

Cursor: .cursor/rules/

Cursor uses the same concept with `.cursor/rules/` directory. The format is slightly different but the idea is identical: markdown files with glob patterns that scope when rules activate.

```
---
description: Testing conventions
globs: tests/**/*.cs
---

Use xUnit, FluentAssertions, NSubstitute. No other test libraries.
```

The Universal Pattern

Regardless of tool, the pattern is the same:

1. A directory of markdown files
2. Each file targets specific paths using glob patterns
3. Rules load automatically when the agent touches matching files
4. Rules supplement (don't replace) the main context file

If you're using a tool that doesn't support path-scoped rules natively, you can approximate them by including relevant instructions in your prompts when working on specific areas. But native support is better because it's automatic: you don't have to remember to include the right context.

Glob Patterns and File-Type Matching

Glob patterns determine when your rules fire. Getting them right matters. Too broad and you're loading irrelevant context. Too narrow and the rules miss files they should cover.

Common Patterns

```
src/Api/Controllers/**/*.cs     # All C# files under Controllers, any depth
tests/**                         # Everything in the tests directory
**/*.test.cs                     # Any test file anywhere in the project
src/Infrastructure/Data/*.cs     # Migration and DbContext files
src/**/Models/*.cs               # Model files in any project
*.proto                          # Protocol buffer definitions
docker-compose*.yml              # All docker-compose variants
```

Pattern Tips

Use `**` for recursive matching. `src/Api/*.cs` only matches files directly in the Api folder. `src/Api/**/*.cs` matches everything in Api and all subdirectories.

Be specific about file extensions. `tests/**` matches everything, including JSON fixtures, text files, and markdown. `tests/**/*.cs` matches only C# files, which is usually what you want for coding conventions.

Match by convention, not just location. If your test files follow a naming pattern like `*.Tests.cs`, you can use `**/*.Tests.cs` to catch them regardless of directory structure. This is more resilient to project restructuring.

Avoid overlapping patterns. If `src/**/*.cs` loads one set of rules and `src/Api/**/*.cs` loads another, files in `src/Api/` get both. This isn't necessarily a problem, but conflicting instructions between the two will confuse the agent. Be intentional about overlap.

Real Examples

Here's a complete set of path-scoped rules for a typical .NET project. Each one addresses real problems I've seen agents get wrong repeatedly.

Controller Rules

File: `.claude/rules/controllers.md`

```
---
description: API controller conventions
globs: src/Api/Controllers/**/*.cs
---

# Controller Conventions

Controllers are thin orchestrators. They do NOT contain business logic.

## Structure
Every action method follows this pattern:
1. Accept the request (model binding handles validation)
2. Send a command or query via MediatR
3. Map the Result<T> to an ActionResult

## Template
```csharp
[HttpPost]
public async Task<ActionResult<OrderResponse>> Create(
    [FromBody] CreateOrderRequest request)
{
    var result = await sender.Send(
        new CreateOrderCommand(request.Sku, request.Quantity));

    return result.Match<ActionResult<OrderResponse>>(
        order => CreatedAtAction(nameof(Get), new { id = order.Id }, order),
        error => error.ToProblemDetails()
    );
}
```
```

Rules

- Use primary constructors for dependency injection
- Always include [Authorize] unless the endpoint is explicitly public
- POST returns 201 with Location header
- PUT and PATCH return 204 (no content)
- DELETE returns 204
- Use [FromRoute], [FromQuery], [FromBody] explicitly
- No try/catch (global exception middleware handles it)
- No logging in controllers (middleware handles request logging)

Test Rules

File: `.claude/rules/tests.md`

```markdown
---
description: Testing patterns and conventions
globs: tests/**/*.cs
---

# Testing Conventions

## Libraries
- xUnit for test framework
- FluentAssertions for assertions
- NSubstitute for mocking
- Do NOT use Moq, MSTest, or any other test library

## Unit Tests
- One test class per handler/service
- Test name format: MethodName_Scenario_ExpectedResult
- Use Arrange/Act/Assert with blank line separators
- Mock interfaces only, never concrete classes
- Use test builders from tests/Builders/ for complex objects

## Integration Tests
- Inherit from IntegrationTestBase (sets up WebApplicationFactory)
- Each test class gets its own database (see TestDatabaseFixture)
- Seed data in constructor, clean up in Dispose
- Test real HTTP calls through the pipeline
- Do NOT mock repositories in integration tests

## Example
```csharp
[Fact]
```
```

```
public async Task Handle_ValidOrder_ReturnsCreatedResult()
{
    // Arrange
    var command = new CreateOrderCommand("SKU-001", 2);

    // Act
    var result = await _handler.Handle(command, CancellationToken.None);

    // Assert
    result.IsSuccess.Should().BeTrue();
    result.Value.Sku.Should().Be("SKU-001");
    await _repository.Received(1).AddAsync(Arg.Any<Order>());
}
```

```
### Migration Rules

File: `.claude/rules/migrations.md`

```markdown
---
description: EF Core migration conventions
globs: src/Infrastructure/Data/Migrations/**/*.cs
---

# Migration Conventions

## CRITICAL: Never edit existing migrations
Existing migration files are immutable. If a migration has been applied
to any environment, it cannot be modified. Create a new migration instead.

## Creating Migrations
Always use the CLI:
```bash
dotnet ef migrations add <MigrationName> --project src/Infrastructure --startup-project src/Api
```

Naming

- Use descriptive PascalCase: AddOrderStatusColumn, CreatePaymentsTable
- Prefix breaking changes: Breaking_RemoveObsoleteColumns
- Never use generic names like “Update1” or “Changes”

Rules

- Always include a Down() method that cleanly reverses the migration
- Add indexes for any foreign key columns
- Add indexes for columns used in WHERE clauses
- Use string max lengths explicitly (no unbounded nvarchar(max))
- Include data migrations in separate migration files from schema changes
- Add comments explaining WHY for non-obvious schema decisions

Domain Entity Rules

File: `.claude/rules/domain.md`

```markdown
---
description: Domain layer conventions
globs: src/Domain/**/*.cs
---

# Domain Conventions

## Entities
- Entities have private setters; modify state through methods
- Use factory methods (static Create()) instead of public constructors
- Raise domain events for significant state changes
- No infrastructure references (no EF Core attributes, no JSON attributes)
- Validation in factory methods and state-changing methods

## Value Objects
- Implement as records
- Include validation in constructor
- Override ToString() for logging

## Example
```csharp
public class Order : Entity
{
    public Guid CustomerId { get; private set; }
    public OrderStatus Status { get; private set; }
    public Money Total { get; private set; }

    private Order() { } // EF Core

    public static Result<Order> Create(Guid customerId, Money total)
```
```

```
    {
        if (total.Amount ≤ 0)
            return Result.Failure<Order>("Order total must be positive");

        var order = new Order
        {
            Id = Guid.NewGuid(),
            CustomerId = customerId,
            Status = OrderStatus.Pending,
            Total = total
        };

        order.AddDomainEvent(new OrderCreatedEvent(order.Id));
        return Result.Success(order);
    }
}
```

Frontend Rules (if applicable)

File: `.claude/rules/frontend.md`

```markdown
---
description: Frontend component conventions
globs: src/Web/Components/**/*.razor, src/Web/Pages/**/*.razor
---

# Blazor Component Conventions

## Structure
- One component per file
- Component parameters at the top, then inject directives, then fields
- Event handlers as private methods at the bottom
- Extract components when they exceed 100 lines

## Naming
- Pages end with "Page": OrderListPage.razor
- Reusable components are descriptive: OrderStatusBadge.razor
- Parameters use PascalCase with [Parameter] attribute

## Rules
- Use CascadingAuthenticationState, not manual auth checks
- Call StateHasChanged() only when necessary (not after every operation)
- Dispose event handlers and subscriptions in IDisposable
- Use @key for list rendering performance
- No inline styles; use CSS isolation (Component.razor.css)
```

When to Add Path-Scoped Rules

This is where most developers get it wrong. The instinct is to sit down, think of every possible convention, and create rules for everything. Resist that instinct.

Add rules reactively, not proactively.

The workflow:

1. You give the agent a task in a specific area of the codebase
2. The agent produces code that doesn't follow your conventions
3. You correct it during review
4. You ask yourself: "Will this keep happening?"
5. If yes, create a path-scoped rule

This approach has three advantages:

You only create rules you actually need. Hypothetical rules for hypothetical problems waste your time and the model's context window.

The rules reflect real pain points. A rule born from an actual mistake is more valuable than one born from speculation. It addresses a specific, demonstrated failure mode.

You don't over-constrain the agent. Too many rules limit the agent's ability to make good decisions. Some things are better left to the model's judgment. Only constrain the things where the model's default judgment consistently fails for your project.

The Trigger Threshold

Not every agent mistake warrants a rule. Here's the filter:

Create a rule when: - The agent makes the same mistake twice in the same area - The convention is non-obvious (the agent can't infer it from existing code) - The convention contradicts common patterns (your project does it differently than most) - Getting it wrong has significant consequences (security, data integrity, architecture violations)

Don’t create a rule when: - The mistake was a one-off caused by a vague prompt - The convention is obvious from existing code (agents learn from surrounding files) - A quick prompt correction is sufficient (“use NSubstitute, not Moq”) - The “rule” is actually a code review preference, not a convention

Anti-Patterns

Path-scoped rules are powerful. They’re also easy to misuse.

Too Many Rules

I’ve seen projects with 30+ rule files. At that point, you’ve recreated the “Novel” problem from Chapter 6, just distributed across more files. The agent is drowning in instructions.

Guideline: Start with 3-5 rule files for distinct areas of your codebase. Add more only when you have demonstrated need. If you’re above 10, audit whether some rules should be consolidated or removed.

Conflicting Rules

When rules overlap in scope, they can contradict each other. Your general `src/**` rules say “use constructor injection.” Your controller rules say “use primary constructors.” These aren’t contradictory to a human, but an agent might struggle with the nuance.

Fix: When rules overlap, make the more specific rule self-contained. Don’t assume the agent will correctly synthesize instructions from multiple rule files. If controller rules need to mention injection, include the full instruction rather than assuming the agent will reference the general rules.

Overly Broad Rules

A rule file matching `**/*.cs` (every C# file in the project) is barely better than putting everything in AGENTS.md. The whole point of path-scoped rules is specificity.

Fix: Scope rules to the narrowest useful path. `src/Api/Controllers/**/*.cs` is better than `src/**/*.cs`. If a convention truly applies everywhere, it belongs in AGENTS.md, not in a path-scoped rule.

Rules That Duplicate the Context File

If your AGENTS.md says "use xUnit and FluentAssertions" and your test rules also say "use xUnit and FluentAssertions," you've wasted context. Path-scoped rules should extend the main context file, not repeat it.

Fix: AGENTS.md covers project-wide conventions. Path-scoped rules cover area-specific conventions that don't apply globally. There should be minimal overlap.

Stale Rules

A rule that references patterns, classes, or conventions that no longer exist actively misleads the agent. This is worse than having no rule at all.

Fix: When you refactor an area of the codebase, update (or delete) the corresponding rules. Include rule files in your code review process: when a PR changes conventions in an area, the reviewer should check whether the rules still match.

Managing Rules as a Team

Path-scoped rules are code. Treat them that way.

Commit them to the repo. Rules in `.claude/rules/` or `.cursor/rules/` should be version-controlled. They're part of your project's development infrastructure, just like your CI configuration or linting rules.

Review them in PRs. When someone adds or changes a rule, the team should review it. A bad rule affects everyone's agent interactions.

Document the intent. The `description` field in the frontmatter isn't just metadata. It helps team members understand why the rule exists without reading the entire file.

Delete aggressively. If a rule hasn't been useful in a month, delete it. You can always recreate it. Stale rules cost more than missing rules.

A Complete Example: Rule Set Evolution

Here's how a typical project's rules evolve over the first month of agentic development.

Week 1: You create AGENTS.md. No path-scoped rules yet. The agent writes decent code but keeps using AutoMapper in new controllers (you don't use it) and keeps writing synchronous test setup code.

Week 2: You add two rules: `controllers.md` (with the "no AutoMapper, use manual mapping" instruction) and `tests.md` (with async patterns and your test base classes). Agent output improves noticeably in these areas.

Week 3: You notice agents generate EF Core migrations with missing indexes and no Down() methods. You add `migrations.md`. A teammate notices agents put validation logic in entities instead of using the Result pattern, so they add `domain.md`.

Week 4: The team reviews all rules. One rule about "always use StringBuilder for string concatenation" turns out to be unnecessary (the agent already does this for performance-sensitive code). It gets deleted. Another rule about Blazor components gets refined based on two weeks of actual usage.

By week 4, you have 4-5 well-tested rules that meaningfully improve agent output. Each one exists because of a real, observed problem. None of them are speculative.

Key Takeaways

1. **Path-scoped rules are surgical context.** They load the right instructions at the right time, keeping the agent focused on what matters for the specific file it's editing.

2. **Add rules reactively.** Wait for the agent to make a mistake twice, then create a rule to prevent it. Don't speculate about what rules you might need.
3. **Keep them focused and non-overlapping.** Each rule file should cover a distinct area of your codebase. Avoid duplication with AGENTS.md.
4. **Treat rules as code.** Version control them. Review them in PRs. Delete them when they're stale.
5. **Less is more.** Five well-targeted rules beat thirty broad ones. Every rule you add costs context window space. Make each one earn its place.

Path-scoped rules are the second layer of your context stack. They turn a good context file into a great one by adding precision where it matters most. But they're still passive: the agent reads them when it encounters matching files. The next chapter covers something more dynamic: skills, where you teach the agent entirely new workflows it can invoke on demand.

CHAPTER 8

Skills: Teaching Your Agent New Tricks

Context files tell agents how to write code. Path-scoped rules tell them how to write code in specific areas. Skills are different. Skills teach agents entirely new workflows, complete processes they can invoke when the situation calls for it.

Think of it this way: your AGENTS.md is the employee handbook. Path-scoped rules are the department-specific procedures. Skills are the specialized training courses, the ones an employee takes when they need to handle something they don't already know how to do.

What Skills Are (and Aren't)

A skill is a bundled set of instructions, documentation, and optional scripts that an agent can invoke on demand. Unlike context files that load automatically, skills load when the agent decides they're relevant based on the skill's description.

This distinction matters. Your AGENTS.md loads every session, consuming context window whether it's relevant or not. A skill for database migrations only loads when the agent is actually doing a migration. A deployment skill only loads when deploying. This keeps your base context lean while giving the agent access to deep, specialized knowledge when it needs it.

Skills are NOT: - Slash commands (those are deprecated in Claude Code, and for good reason) - Macros or scripts (though skills can include scripts) - Prompt templates (they're richer than that)

Skills ARE: - Reusable instruction sets that agents invoke autonomously - Self-contained packages of knowledge for specific workflows - The way you encode complex, multi-step processes that agents can follow

Skills vs. Slash Commands

If you've used Claude Code, you might have encountered slash commands: user-triggered shortcuts that expand into prompts. Slash commands are being deprecated, and skills replace them. The reason is fundamental to how agentic development works.

Slash commands are human-invoked. You type `/deploy` and the agent executes a predefined sequence. This requires you to know when to invoke the command and remember its exact name.

Skills are LLM-invokable. You say "deploy the API to staging" and the agent recognizes that its deployment skill is relevant, loads it, and follows the instructions. You don't need to remember a command name. You describe what you want, and the agent finds the right skill.

This is a meaningful upgrade. It means:

1. **Discovery is automatic.** The agent matches your intent to available skills based on their descriptions. No memorization required.
2. **Composition is natural.** An agent can invoke multiple skills in a single task. "Run the migrations, then deploy to staging" can chain the migration skill and the deployment skill without you managing the sequence.
3. **New team members benefit immediately.** They don't need to learn your project's slash commands. They describe what they want, and the agent finds the right skill.

How Skills Work in Claude Code

Claude Code looks for skills in the `.claude/skills/` directory (project-level) or `~/.claude/skills/` (user-level). Each skill is a directory containing at minimum a `SKILL.md` file that describes the skill and provides instructions.

```
.claude/
  skills/
    deploy/
      SKILL.md
      scripts/
        deploy.sh
    db-migrate/
      SKILL.md
      scripts/
        migrate.sh
    pr-review/
      SKILL.md
```

The `SKILL.md` file has a specific structure:

```
---
description: Deploy the API to a target environment (staging or production)
---

# Deployment Skill

## Prerequisites
- Authenticated to Azure CLI (`az account show` to verify)
- Docker running locally
- Access to the container registry

## Steps

1. Run tests first:
   ```bash
   dotnet test --configuration Release
   ```
   If any test fails, STOP. Do not deploy with failing tests.

2. Build the Docker image:
   ```bash
   docker build -t myapi:$(git rev-parse --short HEAD) -f src/Api/Dockerfile .
   ```

3. Tag and push to registry:
   ```bash
   docker tag myapi:$(git rev-parse --short HEAD) myregistry.azurecr.io/myapi:$(git rev-parse --
   docker push myregistry.azurecr.io/myapi:$(git rev-parse --short HEAD)
   ```

4. Deploy to target environment:
   - Staging: `az containerapp update --name myapi-staging --image myregistry.azurecr.io/myapi:
   - Production: `az containerapp update --name myapi-prod --image myregistry.azurecr.io/myapi:

5. Verify deployment:
   ```bash
```
```

```
    curl https://myapi-{environment}.azurecontainerapps.io/health
    ```


Safety
- NEVER deploy to production without explicit user confirmation
- Always run tests before deploying
- If health check fails after deployment, alert the user immediately
```

When a user says “deploy to staging,” the agent reads the skill descriptions, identifies this skill as relevant, loads the full SKILL.md, and follows the instructions step by step.

# Anatomy of a Good Skill

Every effective skill has four components, whether or not you formalize them into separate sections.

### 1. Description (The Hook)

The description in the frontmatter is how the agent finds the skill. It should be concise, specific, and use the language developers naturally use.

Good: `Deploy the API to a target environment (staging or production)` Bad: `Deployment procedures` Bad: `This skill handles the deployment workflow for the application API including building Docker containers and pushing to Azure Container Registry`

The first is searchable and clear. The second is too vague (deploy what? where?). The third is too long for a description, putting instructions where a summary belongs.

### 2. Instructions (The Steps)

The core of the skill: what to do, in what order, with what commands. Be explicit. Don’t assume the agent knows your deployment pipeline, your migration conventions, or your review process.
```

Write instructions the way you'd write them for a capable new hire on their first week. They know how to code. They know the tools. They don't know your specific process.

3. Scripts (The Automation)

Skills can include shell scripts, PowerShell scripts, or any executable that the agent can invoke. This is optional but powerful.

```
#!/bin/bash
# scripts/deploy.sh
set -euo pipefail

ENVIRONMENT=${1:?Usage: deploy.sh <staging|production>}
IMAGE_TAG=$(git rev-parse --short HEAD)

echo "Running tests..."
dotnet test --configuration Release || { echo "Tests failed. Aborting."; exit 1; }

echo "Building Docker image..."
docker build -t myapi:$IMAGE_TAG -f src/Api/Dockerfile .

echo "Pushing to registry..."
docker tag myapi:$IMAGE_TAG myregistry.azurecr.io/myapi:$IMAGE_TAG
docker push myregistry.azurecr.io/myapi:$IMAGE_TAG

echo "Deploying to $ENVIRONMENT..."
az containerapp update \
  --name myapi-$ENVIRONMENT \
  --image myregistry.azurecr.io/myapi:$IMAGE_TAG

echo "Verifying..."
curl --fail --silent https://myapi-$ENVIRONMENT.azurecontainerapps.io/health \
  || { echo "Health check failed!"; exit 1; }

echo "Deployed to $ENVIRONMENT successfully."
```

The skill's SKILL.md can reference this script: "Run `scripts/deploy.sh <environment>` to execute the deployment." The agent gets the benefit of your tested automation while the skill provides the decision-making context (when to deploy, what to verify, when to stop).

4. Safety Guardrails

Every skill that touches production systems, external services, or irreversible operations needs explicit safety instructions. The agent follows instructions. If your deployment skill doesn't say "ask before deploying to production," the agent might not ask.

Be explicit about: - What requires user confirmation - What to do when something fails - What to never do automatically - Rollback procedures

Real Examples

Database Migration Skill

```
---
description: Create and apply EF Core database migrations
---

# Database Migration Skill

## When to Use
When schema changes are needed: new tables, new columns, index changes,
relationship modifications.

## Creating a Migration

1. Verify the current migration state:
   ```bash
   dotnet ef migrations list --project src/Infrastructure --startup-project src/Api
   ```

2. Make the entity/configuration changes in the Domain and Infrastructure projects.

3. Generate the migration:
   ```bash
   dotnet ef migrations add <DescriptiveName> \
     --project src/Infrastructure \
     --startup-project src/Api
   ```

4. Review the generated migration file:
   - Verify the Up() method does what you expect
   - Verify the Down() method correctly reverses it
   - Check for missing indexes on foreign keys
   - Check string column lengths (never use unbounded nvarchar(max))

5. Apply locally and run tests:
   ```bash
   dotnet ef database update --project src/Infrastructure --startup-project src/Api
   dotnet test
```
```

```
  ```

Naming Conventions
- AddOrderStatusColumn (adding a column)
- CreatePaymentsTable (new table)
- AddIndexOnOrderCustomerId (new index)
- Breaking_RemoveDeprecatedFields (breaking change, prefixed)

Rules
- NEVER edit an existing migration that's been committed
- NEVER combine data migrations with schema migrations
- Always include Down() that cleanly reverses the migration
- Test Down() locally before committing
- If migration generation fails, check that the DbContext is correctly configured

Common Problems
- "No migrations configuration": Missing IDesignTimeDbContextFactory
- "Pending model changes": Run `dotnet ef migrations add` first
- "Column already exists": Someone else added a migration; pull and rebase
```
```

PR Review Skill

```
---
description: Review a pull request for code quality, architecture, and conventions
---

# PR Review Skill

## Process

1. Read the PR description and linked issue (if any)
2. Get the diff:
   ```bash
   git diff main...HEAD --stat
   git diff main...HEAD
   ```

3. Check the following, in order:

Architecture
- [ ] Changes follow the project's architecture patterns (see AGENTS.md)
- [ ] No layer violations (domain doesn't reference infrastructure)
- [ ] New features use vertical slice structure
- [ ] No circular dependencies introduced

Code Quality
- [ ] Methods under 30 lines
- [ ] Clear naming (no abbreviations, no generic names)
- [ ] No code duplication (check for similar existing code)
- [ ] Error handling uses Result pattern, not exceptions
- [ ] Async all the way down (no .Result or .Wait())

Testing
- [ ] New code has corresponding tests
- [ ] Test names follow MethodName_Scenario_ExpectedResult
- [ ] Integration tests for new endpoints
- [ ] Edge cases covered (null, empty, boundary values)
```

```
### Security
- [ ] No secrets or credentials in code
- [ ] New endpoints have [Authorize] (unless explicitly public)
- [ ] Input validation present for user-supplied data
- [ ] No SQL injection vectors (parameterized queries)

### Database
- [ ] Migrations are clean and reversible
- [ ] Indexes added for new foreign keys
- [ ] No N+1 query patterns introduced

4. Summarize findings as:
   - **Must fix:** Issues that block merge
   - **Should fix:** Issues that should be addressed but aren't blockers
   - **Nit:** Style preferences and minor suggestions

## Tone
Be constructive. Explain WHY something is an issue, not just that it is.
Suggest fixes, don't just flag problems.
```

Release Skill

```
---
description: Create a release with changelog, version bump, and Git tag
---

# Release Skill

## Steps

1. Determine the version bump:
   - Check commits since last tag: `git log $(git describe --tags --abbrev=0)..HEAD --oneline`
   - Breaking changes: major bump
   - New features: minor bump
   - Bug fixes only: patch bump

2. Update the version:
   ```bash
   # In the .csproj file
   # <Version>X.Y.Z</Version>
   ```

3. Generate changelog from commits:
   - Group by: Features, Bug Fixes, Breaking Changes
   - Include PR numbers and authors
   - Write to CHANGELOG.md, prepending to existing content

4. Create the release commit:
   ```bash
   git add -A
   git commit -m "release: vX.Y.Z"
   git tag vX.Y.Z
   ```

5. Ask user before pushing:
   ```bash
   git push origin main --tags
   ```
```

```
## Rules
- NEVER push without user confirmation
- Follow semantic versioning strictly
- Always update CHANGELOG.md
```

Creating Skills for Your Project

Start with the workflows you repeat most often. Think about the multi-step processes that you currently do manually or explain to teammates verbally.

Common candidates:

- **Deployment:** Building, testing, pushing, deploying, verifying
- **Database migrations:** Creating, reviewing, applying, rolling back
- **New feature scaffolding:** Creating the handler, command/query, controller, tests, and mapping
- **PR review:** Systematic code review against your project's standards
- **Release management:** Version bumping, changelog, tagging, publishing
- **Incident response:** Checking logs, identifying errors, creating hotfix branches
- **Dependency updates:** Checking for updates, upgrading, running tests, resolving breaking changes

The Skill Creation Process

1. **Do the task manually** and write down every step
2. **Identify decision points** (where you make choices based on context)
3. **Identify automation candidates** (steps that are always the same)
4. **Write the SKILL.md** with steps, decisions documented, and scripts for automation
5. **Test it** by asking the agent to perform the task
6. **Refine** based on where the agent struggled or deviated

The best skills emerge from real work, not from planning sessions. Do the task, then encode what you did.

Sharing Skills Across Projects and Teams

Skills are portable. A deployment skill written for one .NET project often works for another with minor modifications. A PR review skill based on general engineering principles works across the entire organization.

Project-Level Skills (.claude/skills/)

Skills specific to one project live in the project's repository. Database migration skills reference project-specific paths and configurations. Deployment skills reference project-specific infrastructure.

These are committed to the repo and evolve with the project.

User-Level Skills (~/.claude/skills/)

Skills that apply across all your projects live in your home directory. A general PR review skill, a "create a new .NET project" skill, or a "write a blog post" skill aren't project-specific.

These are personal productivity tools. They travel with you across projects.

Organization-Level Skills

For teams, shared skills can live in a dedicated repository or internal package. The pattern:

1. Create a shared `agent-skills` repository
2. Each skill is a directory with SKILL.md and optional scripts
3. Projects reference shared skills by copying or symlinking

This is still an emerging pattern. The tooling for skill distribution is immature (as of early 2026), but the concept is sound. Claude Code's skills system, for instance, supports installing skills from external sources, and community skill marketplaces are beginning to emerge.

What Makes Skills Shareable

A skill is easy to share when it: - Uses relative paths or environment variables instead of hardcoded paths - Documents its prerequisites clearly - Doesn't assume a specific project structure beyond what's standard - Separates project-specific configuration from reusable process

A skill is hard to share when it: - Hardcodes paths, URLs, or credentials - Assumes specific file structures unique to one project - Mixes process instructions with project-specific details

Skills vs. Other Approaches

Skills vs. Documentation

"Why not just write good documentation?" You should. But documentation is passive. An agent might or might not read your deployment guide when asked to deploy. A skill is explicitly designed to be discovered and invoked. Its description is optimized for agent matching. Its instructions are optimized for agent execution.

Skills are documentation that's designed to be consumed by agents, not just humans.

Skills vs. CI/CD Pipelines

"Why not just encode workflows in CI/CD?" You should do that too. CI/CD handles automated, triggered workflows. Skills handle interactive, agent-driven workflows. Your CI pipeline runs tests on push. Your deployment skill helps an agent deploy interactively, with decision points and confirmations.

They complement each other. The deployment skill might say "verify the CI pipeline passed before deploying."

Skills vs. Scripts

“Why not just write shell scripts?” Scripts are part of skills, not a replacement for them. A shell script handles the mechanical execution. A skill provides the context: when to run the script, what to check before running it, how to interpret the results, what to do when something goes wrong.

The script is the hands. The skill is the brain.

Anti-Patterns

The Mega-Skill

A skill that tries to handle every possible scenario becomes an AGENTS.md in disguise. If your deployment skill covers staging, production, canary releases, rollbacks, hotfixes, and database migrations, it’s too big. Split it into focused skills: deploy, rollback, hotfix.

The Stale Skill

Like stale context files, stale skills are worse than no skills. If your deployment process changed and the skill still references the old process, the agent will follow the old process confidently. Audit skills quarterly.

The Over-Documented Skill

A skill with 500 lines of instructions overwhelms the agent’s context. Skills should be concise, focused on the process, and light on background explanation. If the agent needs background knowledge, link to documentation rather than inlining it.

The Permission-Free Skill

Any skill that touches production, sends emails, modifies databases, or performs irreversible actions needs explicit confirmation steps. “Ask the user before proceeding” should appear at every dangerous decision point.

Key Takeaways

1. **Skills are agent-invokable workflows.** They load on demand, keeping your base context lean while giving the agent deep knowledge when needed.
2. **Skills replace slash commands.** They're discoverable by description, composable, and don't require memorization.
3. **Start with your most-repeated workflows.** Deployment, migrations, reviews, and scaffolding are common candidates.
4. **Include scripts for automation, instructions for decisions.** The skill provides the brain (when, why, what-if). The scripts provide the hands (how).
5. **Share skills across projects.** Project-level for specific workflows, user-level for personal productivity, organization-level for team standards.
6. **Keep skills focused and current.** One workflow per skill. Update them when processes change. Delete them when they're obsolete.

Skills complete the "passive context" portion of the context stack. Your AGENTS.md, path-scoped rules, and skills together give the agent everything it needs to know about your project's conventions and workflows. The next chapter introduces the "active context" layer: MCP servers, which give agents access to external tools and data in real time.

CHAPTER 9

MCP Servers and External Tool Integration

Context files are static. They tell the agent what you know at the time you write them. But agents also need access to things that change: your database schema, your API documentation, your deployment status, your issue tracker. This is where MCP comes in.

The Model Context Protocol (MCP) is a standardized way for AI agents to access external tools and data sources. If AGENTS.md is the employee handbook and skills are the training courses, MCP servers are the access badges, giving the agent permission and ability to interact with systems outside the codebase.

What MCP Is

MCP is an open protocol created by Anthropic that standardizes how AI models connect to external data sources and tools. The analogy that stuck (for good reason) is “USB for AI agents.” Before USB, every peripheral needed its own proprietary connector. Before MCP, every AI tool needed custom integration code for every external system.

MCP provides a common interface. A database MCP server exposes your schema and query capabilities through the same protocol as a GitHub MCP server that exposes issues and PRs. The agent doesn’t need to know the implementation details. It speaks MCP, and the server translates.

The protocol has three core concepts:

Tools: Functions the agent can call. A database MCP server might expose `query`, `list_tables`, and `describe_table` tools. A GitHub server might expose `list_issues`, `create_pr`, and `get_file`.

Resources: Data the agent can read. Database schemas, configuration files, API documentation. Resources are read-only context that helps the agent make better decisions.

Prompts: Predefined interaction patterns. A "review this PR" prompt that combines the right resources and tools for a code review workflow.

Why MCP Matters

Without MCP, connecting an agent to your database requires custom code: a script that queries the schema, formats it as context, and injects it into the agent's prompt. Do the same for your API docs, your CI system, your monitoring dashboard, and you've built a bespoke integration layer that's fragile and tool-specific.

With MCP, you configure a server once and every MCP-compatible agent can use it. Claude Code, Cursor, Windsurf, and other tools all support MCP. Write the integration once, use it everywhere.

For .NET developers specifically, MCP is significant because:

1. **Your database schema is living context.** Agents writing EF Core migrations need to know the current schema. An MCP server that exposes your database schema means the agent always sees the current state, not a stale document.

2. **Your APIs are discoverable.** An MCP server wrapping your Swagger/OpenAPI spec lets agents understand your API surface without you copying documentation into context files.

3. **Your infrastructure is queryable.** Agents can check deployment status, read logs, query monitoring systems, all through standardized tool calls.

Available MCP Servers

The MCP ecosystem has grown rapidly. Here are the categories most relevant to .NET development.

Database Servers

The most immediately useful MCP servers for application developers. They expose: - Table and column listings - Schema details (types, constraints, relationships) - Read-only query execution - Index and performance information

PostgreSQL, SQL Server, MySQL, and SQLite all have MCP servers available. For .NET projects using EF Core, a database MCP server means the agent can inspect the actual database before generating migrations, not just guess from entity classes.

Source Control Servers

GitHub and GitLab MCP servers expose: - Repository structure and file contents - Issues and pull requests - CI/CD pipeline status - Branch and tag information

This lets agents check whether CI passed before deploying, read linked issues when reviewing PRs, and understand the full context of a change.

API and Documentation Servers

Servers that expose your API documentation (OpenAPI/Swagger specs) or internal documentation (Confluence, Notion, or plain markdown). This gives agents access to your API contracts, architecture decisions, and team knowledge without embedding everything in context files.

Custom Servers

This is where it gets interesting. You can build MCP servers that expose anything: your internal tooling, your deployment pipeline, your feature flag system, your monitoring dashboards. If it has an API, you can wrap it in MCP.

Building an MCP Server in C

The MCP C# SDK (in preview since late 2025) makes it straightforward to build custom MCP servers. Let's build one that exposes a database schema, the most common use case for .NET developers.

Setup

```
dotnet new console -n DatabaseMcpServer
cd DatabaseMcpServer
dotnet add package ModelContextProtocol --prerelease
dotnet add package Npgsql
```

A Simple Schema Explorer

```
using ModelContextProtocol;
using ModelContextProtocol.Server;
using Npgsql;

var builder = McpServerBuilder.CreateStdio(args);

builder.AddTool("list_tables", "List all tables in the database",
    async (McpToolContext context) =>
    {
        await using var conn = new NpgsqlConnection(
            Environment.GetEnvironmentVariable("DATABASE_URL"));
        await conn.OpenAsync();

        await using var cmd = new NpgsqlCommand("""
            SELECT table_name
            FROM information_schema.tables
            WHERE table_schema = 'public'
            ORDER BY table_name
            """, conn);

        var tables = new List<string>();
        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            tables.Add(reader.GetString(0));
        }

        return McpToolResult.Text(string.Join("\n", tables));
    });

builder.AddTool("describe_table",
    "Describe a table's columns, types, and constraints",
    async (McpToolContext context) =>
    {
        var tableName = context.GetRequiredArgument<string>("table_name");
```

```
        await using var conn = new NpgsqlConnection(
            Environment.GetEnvironmentVariable("DATABASE_URL"));
        await conn.OpenAsync();

        await using var cmd = new NpgsqlCommand("""
            SELECT
                c.column_name,
                c.data_type,
                c.is_nullable,
                c.column_default,
                tc.constraint_type
            FROM information_schema.columns c
            LEFT JOIN information_schema.key_column_usage kcu
                ON c.column_name = kcu.column_name
                AND c.table_name = kcu.table_name
            LEFT JOIN information_schema.table_constraints tc
                ON kcu.constraint_name = tc.constraint_name
            WHERE c.table_name = @tableName
            AND c.table_schema = 'public'
            ORDER BY c.ordinal_position
            """, conn);

        cmd.Parameters.AddWithValue("tableName", tableName);

        var columns = new List<string>();
        await using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            var name = reader.GetString(0);
            var type = reader.GetString(1);
            var nullable = reader.GetString(2) == "YES" ? "NULL" : "NOT NULL";
            var defaultVal = reader.IsDBNull(3) ? "" : $" DEFAULT {reader.GetString(3)}";
            var constraint = reader.IsDBNull(4) ? "" : $" [{reader.GetString(4)}]";

            columns.Add($"  {name} {type} {nullable}{defaultVal}{constraint}");
        }
```

```
        return McpToolResult.Text(
            $"Table: {tableName}\n{string.Join("\n", columns)}");
    });

builder.AddTool("query",
    "Execute a read-only SQL query against the database",
    async (McpToolContext context) =>
    {
        var sql = context.GetRequiredArgument<string>("sql");

        // Safety: only allow SELECT statements
        if (!sql.TrimStart().StartsWith("SELECT", StringComparison.OrdinalIgnoreCase))
        {
            return McpToolResult.Error(
                "Only SELECT queries are allowed for safety.");
        }

        await using var conn = new NpgsqlConnection(
            Environment.GetEnvironmentVariable("DATABASE_URL"));
        await conn.OpenAsync();

        // Wrap in a read-only transaction for additional safety
        await using var tx = await conn.BeginTransactionAsync(
            System.Data.IsolationLevel.ReadCommitted);

        await using var cmd = new NpgsqlCommand(sql, conn, tx);
        await using var reader = await cmd.ExecuteReaderAsync();

        var results = new List<string>();
        var columnNames = Enumerable.Range(0, reader.FieldCount)
            .Select(i => reader.GetName(i))
            .ToList();

        results.Add(string.Join(" | ", columnNames));
        results.Add(new string('-', results[0].Length));
```

```
        while (await reader.ReadAsync())
        {
            var values = Enumerable.Range(0, reader.FieldCount)
                .Select(i => reader.IsDBNull(i) ? "NULL" : reader.GetValue(i).ToString()!)
                .ToList();
            results.Add(string.Join(" | ", values));
        }

        return McpToolResult.Text(string.Join("\n", results));
    });

var server = builder.Build();
await server.RunAsync();
```

This is a real, working MCP server. It's about 100 lines of C# and gives any MCP-compatible agent the ability to explore and query your database. The patterns are universal: the same approach works for SQL Server, MySQL, or any data source.

Configuring the Agent to Use It

In Claude Code, MCP servers are configured in `.claude/settings.json`:

```
{
  "mcpServers": {
    "database": {
      "command": "dotnet",
      "args": ["run", "--project", "tools/DatabaseMcpServer"],
      "env": {
        "DATABASE_URL": "Host=localhost;Database=myapp;Username=dev;Password=dev"
      }
    }
  }
}
```

In Cursor, the configuration goes in `.cursor/mcp.json` with a similar structure. The agent now has database access. When you say “look at the orders table schema before writing this migration,” the agent can actually do it.

Security Considerations

MCP servers are powerful. They give agents direct access to your systems. This demands careful thought about security.

The Principle of Least Privilege

Expose the minimum functionality the agent needs. For a database MCP server:

Do expose: - Schema inspection (tables, columns, types, relationships) - Read-only queries against development databases - Index and constraint information

Don’t expose: - Write operations (INSERT, UPDATE, DELETE) - DDL operations (CREATE TABLE, DROP TABLE) - Production database connections - Administrative functions (user management, permissions)

The example above enforces read-only access by rejecting non-SELECT queries and wrapping everything in a read-only transaction. This is the right default.

Environment Separation

Never point an MCP server at a production database during development. Use a development or staging copy. If you need production data for debugging, create a read-only replica connection.

```
{
  "mcpServers": {
    "database-dev": {
      "command": "dotnet",
      "args": ["run", "--project", "tools/DatabaseMcpServer"],
      "env": {
        "DATABASE_URL": "Host=localhost;Database=myapp_dev;Username=dev;Password=dev"
      }
    }
  }
}
```

The connection string makes the boundary explicit. The agent physically cannot access production because the MCP server isn't configured with production credentials.

Credential Management

Don't hardcode credentials in MCP configuration files that get committed to source control. Use environment variables, secret managers, or local configuration files that are gitignored.

```
{
  "mcpServers": {
    "database": {
      "command": "dotnet",
      "args": ["run", "--project", "tools/DatabaseMcpServer"],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

Audit Logging

For MCP servers that access sensitive systems, log every tool invocation. This gives you visibility into what the agent is doing and helps you catch unexpected behavior.

```
builder.AddTool("query", "Execute a read-only SQL query",
    async (McpToolContext context) =>
    {
        var sql = context.GetRequiredArgument<string>("sql");

        // Log every query
        Console.Error.WriteLine(
            $"[MCP] Query executed at {DateTime.UtcNow}: {sql}");

        // ... execute query
    });
```

Log to stderr (which goes to the agent's log, not the agent's response) so you have a record without polluting the agent's context.

Real Workflow: Schema-Aware Migration Generation

Here's where MCP transforms a common workflow. Without MCP, generating a database migration looks like this:

1. Developer asks agent to add a new feature requiring schema changes
2. Agent guesses at the current schema based on entity classes
3. Agent generates a migration that might conflict with existing columns, miss indexes, or duplicate constraints
4. Developer reviews, fixes, and iterates

With a database MCP server, the workflow becomes:

1. Developer asks agent to add a new feature requiring schema changes
2. Agent calls `list_tables` to see the current schema
3. Agent calls `describe_table` for relevant tables to understand existing columns and constraints
4. Agent calls `query` to check for existing indexes and relationships
5. Agent generates a migration that accounts for the actual current state
6. Developer reviews and (usually) approves on the first pass

The difference in quality is significant. The agent isn't guessing. It's working from the same source of truth you'd use: the actual database.

Here's what the agent's internal process looks like:

```
User: "Add a priority field to the Orders table. High, Medium, Low. Default to Medium."

Agent thinks: I should check the current Orders table schema first.

Agent calls: describe_table(table_name: "orders")
Result:
  Table: orders
    id uuid NOT NULL [PRIMARY KEY]
    customer_id uuid NOT NULL [FOREIGN KEY]
    status integer NOT NULL DEFAULT 0
    total_amount numeric(18,2) NOT NULL
    created_at timestamp NOT NULL DEFAULT now()
    updated_at timestamp NULL

Agent thinks: Good, no existing priority column. Status uses integer (enum mapping).
I should use the same pattern for priority. Let me check if there are any
indexes I should be aware of.

Agent calls: query(sql: "SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders'"
Result:
  orders_pkey | CREATE UNIQUE INDEX orders_pkey ON public.orders USING btree (id)
  ix_orders_customer_id | CREATE INDEX ix_orders_customer_id ON public.orders USING btree (cust

Agent: Now I can generate an accurate migration.
```

The resulting migration is precise because it's based on facts, not assumptions.

Building Beyond Databases

The database example is the most common, but MCP servers can wrap anything.

API Documentation Server

Expose your OpenAPI/Swagger spec so agents can understand your API surface:

```
builder.AddTool("list_endpoints", "List all API endpoints",
    async (McpToolContext context) =>
    {
        var spec = await File.ReadAllTextAsync("openapi.json");
        var doc = JsonDocument.Parse(spec);

        var endpoints = new List<string>();
        foreach (var path in doc.RootElement.GetProperty("paths").EnumerateObject())
        {
            foreach (var method in path.Value.EnumerateObject())
            {
                var summary = method.Value.TryGetProperty("summary", out var s)
                    ? s.GetString() : "No description";
                endpoints.Add($"{method.Name.ToUpper()} {path.Name} - {summary}");
            }
        }

        return McpToolResult.Text(string.Join("\n", endpoints));
    });
```

Feature Flag Server

Let agents check which features are enabled:

```
builder.AddTool("check_feature_flag",
    "Check if a feature flag is enabled in the target environment",
    async (McpToolContext context) =>
    {
        var flag = context.GetRequiredArgument<string>("flag_name");
        var env = context.GetRequiredArgument<string>("environment");

        // Query your feature flag service
        var client = new HttpClient();
        var response = await client.GetAsync(
            $"https://flags.internal/api/flags/{flag}?env={env}");

        var result = await response.Content.ReadAsStringAsync();
        return McpToolResult.Text(result);
    });
```

CI/CD Status Server

Let agents check build status before deploying:

```
builder.AddTool("check_ci_status",
    "Check the CI/CD pipeline status for a branch",
    async (McpToolContext context) =>
    {
        var branch = context.GetRequiredArgument<string>("branch");

        var client = new HttpClient();
        client.DefaultRequestHeaders.Add("Authorization",
            $"Bearer {Environment.GetEnvironmentVariable("GITHUB_TOKEN")}");

        var response = await client.GetAsync(
            $"https://api.github.com/repos/myorg/myrepo/actions/runs?branch={branch}&per_page=1

        var json = await response.Content.ReadAsStringAsync();
        return McpToolResult.Text(json);
    });
```

The pattern is always the same: wrap an existing system's API in MCP tools with clear names and descriptions. The agent handles the orchestration.

Connecting Multiple MCP Servers

Real projects often need multiple MCP servers working together. A typical .NET development setup might include:

```
{
  "mcpServers": {
    "database": {
      "command": "dotnet",
      "args": ["run", "--project", "tools/DatabaseMcpServer"],
      "env": { "DATABASE_URL": "${DATABASE_URL}" }
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./docs"]
    }
  }
}
```

The agent can now: 1. Read the database schema to understand current state 2. Check GitHub issues for requirements 3. Read architecture documentation from the docs folder 4. Generate code that accounts for all of this context

This is context engineering at its most powerful: the agent has access to the same information sources you do.

When NOT to Use MCP

MCP isn't always the answer. Some situations are better served by simpler approaches.

Static information: If your database schema changes rarely and you already document it in your AGENTS.md, an MCP server adds complexity without proportional benefit.

Use MCP when the data changes frequently enough that static documentation goes stale.

Sensitive production systems: MCP makes it easy for agents to query systems. Maybe too easy. If the risk of an agent accidentally querying your production payment database outweighs the benefit, stick to documentation.

Simple projects: A solo developer working on a small API doesn't need an MCP server for their local SQLite database. The overhead of building and maintaining the server exceeds the benefit. MCP shines in larger projects with complex infrastructure.

When security review hasn't happened: Don't deploy MCP servers that access sensitive systems without your team reviewing the security implications. The convenience isn't worth the risk if the server has vulnerabilities.

Key Takeaways

1. **MCP standardizes tool access for agents.** Build an integration once, use it with any MCP-compatible agent. No more custom glue code per tool.
2. **Database MCP servers are the highest-leverage starting point.** Agents that can inspect the actual schema produce dramatically better migrations, queries, and data access code.
3. **Security is non-negotiable.** Read-only access, development databases only, no hardcoded credentials, audit logging. These aren't suggestions.
4. **Building MCP servers in C# is straightforward.** The MCP C# SDK handles the protocol. You just implement the tools.
5. **Start simple.** One MCP server for your database. Add more as you identify real needs. Don't build an MCP server for every system on day one.
6. **MCP completes the active context layer.** Static context (AGENTS.md, rules, skills) plus dynamic context (MCP servers) gives agents a comprehensive understanding of your project and its environment.

With context files, path-scoped rules, skills, and MCP servers, you have the full context engineering toolkit. But there’s one more piece that affects everything: the codebase itself. How you structure your code, name your types, organize your files, all of that is context too. The next chapter covers how to design a codebase that makes agents dramatically more effective.

CHAPTER 10

Designing an AI-Friendly Codebase

Here's the thing nobody tells you about agentic development: the biggest factor in agent output quality isn't your prompt, your context file, or your model choice. It's your codebase.

Your existing code is the single largest source of context an agent has. When an agent needs to add a feature, it reads your existing features. When it needs to write a test, it reads your existing tests. When it needs to follow a pattern, it looks at how you've implemented that pattern before.

A well-structured codebase makes agents dramatically more effective. A poorly structured one makes them confidently produce garbage that matches your existing garbage.

The practices in this chapter aren't new. They're the same engineering principles that senior developers have advocated for decades: clear naming, small focused files, good documentation, comprehensive tests. What's new is the payoff. These practices always helped human developers. Now they help AI agents too, and the compounding effect is enormous.

Naming Conventions That Help Agents (and Humans)

Agents take names at face value. A human teammate might know that `OrderProcessor` actually handles payment processing because they sat in the meeting where someone

named it badly. An agent doesn't have that institutional knowledge. It reads `OrderProcessor` and assumes it processes orders.

Be Descriptive, Not Clever

```
// Bad: What does this process?
public class OrderProcessor { }

// Good: Exactly what it does
public class OrderPaymentProcessor { }

// Bad: Manager of what? How?
public class DataManager { }

// Good: Specific responsibility
public class CustomerRepository { }
public class OrderCacheInvalidator { }
```

Name Methods for Behavior, Not Implementation

```
// Bad: Describes the mechanism
public async Task<List<Order>> RunQuery(string customerId) { }

// Good: Describes the behavior
public async Task<List<Order>> GetOrdersByCustomer(string customerId) { }

// Bad: Too generic
public async Task<Result> Process(Command command) { }

// Good: Clear intent
public async Task<Result<OrderConfirmation>> PlaceOrder(
    PlaceOrderCommand command) { }
```

Use Consistent Naming Patterns

When agents see consistent patterns, they replicate them. If every handler in your project follows `{Action}{Entity}Handler`, the agent will follow the same pattern for new handlers. If your naming is inconsistent (some handlers are `{Entity}{Action}Handler`, others are `Handle{Entity}{Action}`), the agent picks whichever pattern it encounters first, which might not be your preferred one.

Document your naming conventions in AGENTS.md, but also follow them consistently. Agents learn more from your code than from your documentation.

```
Commands:  Create{Entity}Command, Update{Entity}Command, Delete{Entity}Command
Queries:   Get{Entity}Query, List{Entities}Query, Search{Entities}Query
Handlers:  {CommandOrQuery}Handler
Responses: {Entity}Response, {Entity}ListResponse
```

File Size and Cohesion

This is the single highest-impact structural change you can make for agent effectiveness: keep files small and focused.

Why Small Files Win

Context windows are finite. When an agent needs to modify a 2,000-line file, it has to load that entire file into context. That's expensive in tokens and leaves less room for other relevant context. A 50-line file leaves room for the agent to also read related files, tests, and documentation.

Agents make fewer mistakes in small files. A study by GitClear found a 4x increase in code clones when developers used AI assistants. The root cause: agents working with large, complex files produce more duplicate and inconsistent code. Small, focused files have clear boundaries that agents respect.

Modifications are surgical. When an agent needs to change one behavior in a small, focused file, it modifies the right thing. When it needs to change one behavior in a God class, it might accidentally affect other behaviors.

The Practical Threshold

There's no magic number, but here's a useful guideline:

- **Under 100 lines:** Agent can fully understand and modify confidently
- **100-200 lines:** Agent handles well with good structure
- **200-500 lines:** Agent starts making more mistakes, missing context from distant parts of the file
- **Over 500 lines:** Expect quality issues, extract classes and methods

These aren't hard rules. A 300-line file with clear sections and good naming works fine. A 150-line file with tangled responsibilities is worse. Cohesion matters more than raw line count.

Extracting for AI-Readability

If you have large files, refactoring them for agent-friendliness is the same as refactoring for human-friendliness: extract classes, extract methods, separate concerns.

```
// Before: 800-line OrderService doing everything
public class OrderService
{
    public async Task<Result<Order>> PlaceOrder(...) { /* 100 lines */ }
    public async Task<Result> CancelOrder(...) { /* 80 lines */ }
    public async Task<Result> RefundOrder(...) { /* 120 lines */ }
    public async Task<List<Order>> GetOrderHistory(...) { /* 60 lines */ }
    public async Task<OrderReport> GenerateReport(...) { /* 150 lines */ }
    // ... more methods, private helpers, validation logic
}

// After: Focused files the agent can work with independently
public class PlaceOrderHandler { /* 60 lines */ }
public class CancelOrderHandler { /* 50 lines */ }
public class RefundOrderHandler { /* 70 lines */ }
public class GetOrderHistoryHandler { /* 40 lines */ }
public class GenerateOrderReportHandler { /* 80 lines */ }
```

Each handler can be modified without the agent needing to understand the others. Each can have its own test file. Each is small enough that the agent can hold the entire implementation in context.

Project Structure Patterns

How you organize files determines how easily agents navigate your codebase.

Feature Folders (Vertical Slices)

The most agent-friendly project structure groups code by feature, not by technical layer.

```
src/Api/
  Features/
    Orders/
      PlaceOrder/
        PlaceOrderCommand.cs
        PlaceOrderHandler.cs
        PlaceOrderValidator.cs
        PlaceOrderEndpoint.cs
        PlaceOrderTests.cs
      CancelOrder/
        CancelOrderCommand.cs
        CancelOrderHandler.cs
        CancelOrderEndpoint.cs
      GetOrderHistory/
        GetOrderHistoryQuery.cs
        GetOrderHistoryHandler.cs
        GetOrderHistoryEndpoint.cs
    Payments/
      ProcessPayment/
        ...
      RefundPayment/
        ...
```

Why this is agent-friendly:

Locality of reference. When an agent needs to work on order placement, everything it needs is in one folder. It doesn't need to navigate between `Controllers/`, `Services/`, `Models/`, and `Validators/` to understand one feature.

Pattern replication. When you say "add a feature to update order status," the agent looks at existing features, sees the consistent structure, and replicates it. The more consistent your feature folders, the better the agent's output.

Isolation. Changes to one feature don't require the agent to understand other features. This reduces the context needed per task.

Layer-Based Structure (Traditional)

The traditional structure groups by technical layer:

```
src/Api/
  Controllers/
    OrdersController.cs
    PaymentsController.cs
  Services/
    OrderService.cs
    PaymentService.cs
  Models/
    Order.cs
    Payment.cs
  Validators/
    CreateOrderValidator.cs
    ProcessPaymentValidator.cs
```

This works but is less agent-friendly because a single feature is spread across multiple directories. The agent needs to navigate more files to understand one feature. It's not a dealbreaker, but feature folders give agents a meaningful advantage.

The Hybrid Approach

Most real projects use a hybrid: feature folders for application logic, layer separation for cross-cutting concerns.

```
src/
  Api/
    Features/           # Vertical slices for business features
      Orders/
      Payments/
    Middleware/          # Cross-cutting (auth, error handling, logging)
    Infrastructure/     # Database, caching, external services
    Common/             # Shared types (Result<T>, PagedList<T>)
```

This gives agents the best of both worlds: feature locality for business logic, clear homes for shared concerns.

Documentation-as-Context

Every piece of documentation in your repository is context that agents can read. The question is whether your documentation is useful to them.

Architecture Decision Records (ADRs)

ADRs are the highest-value documentation for agents. They explain WHY your codebase is structured the way it is, which helps agents make decisions that align with your architectural intent.

```
# ADR-003: Use Result Pattern Instead of Exceptions for Business Logic

## Status
Accepted

## Context
We need a consistent error handling approach for business logic.
Exceptions are expensive in .NET, difficult to compose, and make
control flow implicit.

## Decision
Use a Result<T> type for all business logic outcomes. Only use
exceptions for truly exceptional cases (infrastructure failures,
programming errors).

## Consequences
- Handlers return Result<T> instead of throwing
- Controllers map Result failures to HTTP status codes
- No try/catch in business logic layer
- All business error cases are explicit in the type signature
```

An agent reading this ADR understands not just the pattern but the reasoning. When it encounters a situation where throwing an exception might seem natural, it knows to use the Result pattern instead, and it knows why.

Store ADRs in `docs/adr/` or `docs/decisions/` and reference the directory in your AGENTS.md.

Inline Comments: When and Why

Agents read comments. Strategic comments can prevent entire categories of mistakes.

Comment the WHY, not the WHAT:

```
// BAD: Describes what the code does (the agent can read the code)
// Loop through orders and calculate total
foreach (var order in orders)
{
    total += order.Amount;
}

// GOOD: Explains why a non-obvious approach is used
// DECISION: Raw SQL here because EF Core generates a suboptimal
// execution plan for this specific join pattern. See ADR-007.
var orders = await db.Database.SqlQueryRaw<OrderDto>("""
    SELECT o.id, o.total, c.name
    FROM orders o
    INNER JOIN customers c ON o.customer_id = c.id
    WHERE o.created_at > @cutoff
    """, new NpgsqlParameter("cutoff", cutoffDate));

// GOOD: Warns about a non-obvious constraint
// WARNING: This method must be called BEFORE SaveChangesAsync
// because the domain events are cleared during save.
public IReadOnlyList<IDomainEvent> GetDomainEvents()
{
    return _domainEvents.AsReadOnly();
}
```

Comment the boundaries:

```
// IMPORTANT: This endpoint is intentionally [AllowAnonymous]
// because it serves the public product catalog. Do not add [Authorize].
[AllowAnonymous]
[HttpGet]
public async Task<ActionResult<List<ProductResponse>>> ListProducts() { }
```

Without that comment, an agent following your “all endpoints require [Authorize]” convention would “helpfully” add authentication to your public catalog endpoint.

README Patterns

Your project's README is one of the first things an agent reads. Make it useful:

```
# MyApp API

Order management API for the MyApp platform.

## Quick Start
```bash
docker compose up -d    # Start dependencies (PostgreSQL, Redis, RabbitMQ)
dotnet run --project src/Api
```
```

Architecture

Vertical slice architecture with CQRS. See docs/adr/ for decisions.

Key Patterns

- **Commands/Queries:** MediatR handlers in Features/ folders
- **Error handling:** Result pattern (no exceptions for business logic)
- **Testing:** xUnit + FluentAssertions + NSubstitute

Project Map

- `src/Api/Features/` , Business features (vertical slices)
- `src/Api/Infrastructure/` , Database, caching, external services
- `tests/` , Unit and integration tests

- `docs/adr/` , Architecture decision records

```
Short, actionable, structured. An agent reading this knows how to build, where to find things, a

## The Test Suite as Your Most Powerful Context

Your test suite is arguably more valuable than your context file for agent quality. Here's why:

### Tests Are Executable Specifications

When an agent reads your tests, it learns:

1. **What the code should do.** Each test is a behavior specification. `PlaceOrder_WithValidIte

2. **What edge cases matter.** `PlaceOrder_WithEmptyCart_ReturnsValidationError` tells the agen

3. **What patterns to follow.** Test structure, naming, mocking patterns, assertion styles, all

4. **Whether changes work.** An agent that can run tests after making changes can self-correct.

### Tests Enable the Agent Self-Correction Loop

The most effective agentic workflow is test-first development:

1. You write (or describe) the test for the feature you want
2. The agent implements the feature
3. The agent runs the tests
4. If tests fail, the agent reads the error and fixes the code
5. Repeat until all tests pass

Without tests, the agent produces code and hopes it works. With tests, the agent iterates until

### Test Coverage and Agent Confidence

Higher test coverage means agents can make changes more confidently. If your project has 80%+ c

- The agent can refactor safely (tests catch regressions)
```

```
- The agent can add features incrementally (tests verify each step)
- The agent can resolve conflicts (tests confirm correctness after merge)

If your project has 10% coverage:

- Every agent change is a gamble
- You manually verify everything
- Refactoring is risky

Investing in tests has always paid off. With agents, the ROI is 10x what it was.

## Anti-Patterns: What Makes Codebases AI-Hostile

### God Classes

A single class handling validation, business logic, data access, notification, and logging. Age

**Fix:** Extract each responsibility into its own class. Use dependency injection to compose th

### Hidden Dependencies

Classes that reach into static state, service locators, or ambient contexts instead of declarin

```csharp
// AI-hostile: Hidden dependency on static context
public class OrderHandler
{
    public async Task Handle(CreateOrderCommand command)
    {
        var user = HttpContext.Current.User; // Where did this come from?
        var config = ConfigurationManager.AppSettings["MaxOrderSize"]; // And this?
        var db = DatabaseFactory.Create(); // And this?
    }
}

// AI-friendly: All dependencies visible
```
```

```
public class OrderHandler(
    ICurrentUser currentUser,
    IOptions<OrderSettings> settings,
    IOrderRepository repository)
{
    public async Task<Result<Order>> Handle(CreateOrderCommand command)
    {
        var user = currentUser.Get();
        var maxSize = settings.Value.MaxOrderSize;
        // Dependencies are clear, mockable, and discoverable
    }
}
```

Agents can only work with what they can see. Hidden dependencies are invisible to them.

Convention-Free Codebases

When every file follows a different pattern, every class uses a different naming convention, and there's no consistency anywhere, agents have no patterns to replicate. They fall back on generic training data patterns, which probably don't match your project.

Fix: Pick conventions. Document them. Follow them. Consistency is free and compounds over time.

Magic Strings and Numbers

```
// AI-hostile: What do these mean?
if (order.Status == 3 && order.Total > 500)
{
    SendNotification("order-alert", order.CustomerId);
}

// AI-friendly: Self-documenting
if (order.Status == OrderStatus.Shipped && order.Total > OrderThresholds.HighValue)
{
    await notificationService.SendAsync(
        NotificationType.HighValueOrderShipped,
        order.CustomerId);
}
```

Agents can reason about `OrderStatus.Shipped`. They can't reason about `3`.

Deep Inheritance Hierarchies

```
// AI-hostile: Agent needs to trace 5 levels to understand behavior
public class PremiumOrderHandler
    : SpecialOrderHandler
    : OrderHandler
    : BaseHandler<Order>
    : AbstractHandler
    : IHandler { }
```

Each layer of inheritance is another file the agent needs to read and hold in context. Composition over inheritance isn't just cleaner for humans. It's dramatically easier for agents to work with.

Refactoring for AI-Readability

If your codebase has some of these anti-patterns, here's the good news: refactoring for AI-readability is the same as refactoring for human-readability. Every change that helps agents also helps your team.

Priority Order

1. **Extract God classes.** Highest impact. Split large classes into focused, single-responsibility classes.
2. **Make dependencies explicit.** Replace static access and service locators with constructor injection.
3. **Establish naming conventions.** Pick patterns and apply them consistently across new code. Gradually rename existing code.
4. **Add tests for untested areas.** Focus on the areas agents will modify most. Tests enable the self-correction loop.
5. **Write ADRs for non-obvious decisions.** Start with the decisions agents get wrong most often.
6. **Move to feature folders.** Reorganize from layer-based to feature-based structure, one feature at a time.

The Incremental Approach

You don't need to refactor your entire codebase before using agents. That's backwards. Start using agents now. Pay attention to where they struggle. Refactor those areas first.

If agents keep producing bad code in your payment module because it's a 3,000-line God class, that's your signal to refactor the payment module. If agents handle your user module perfectly because it's clean and well-structured, leave it alone.

Let agent struggles guide your refactoring priorities. You're optimizing for the areas where it matters most.

The Compound Effect

Every practice in this chapter reinforces the others:

- Clear naming makes small files easy to discover
- Small files make feature folders practical
- Feature folders make tests easy to colocate
- Colocated tests make agent self-correction possible
- Agent self-correction produces better code
- Better code is easier to name, keep small, and organize

This is why AI-friendly codebase design isn't a separate concern from good engineering. It IS good engineering. The practices that have always been best practices now have a measurable, immediate payoff through agent productivity.

Addy Osmani put it well: "AI-assisted development actually rewards good engineering practices MORE than traditional coding does." Every shortcut you took, every God class you tolerated, every naming convention you skipped, now costs you twice: once in human comprehension, and again in agent output quality.

The flip side is encouraging. Every investment in code quality now pays off twice too.

Key Takeaways

1. **Your codebase is your largest context source.** How you structure it matters more than any context file.
2. **Small, focused files are the highest-impact change.** Under 100 lines per file is the sweet spot for agent effectiveness.
3. **Feature folders beat layer folders for agents.** Locality of reference reduces the context needed per task.

4. **Tests are your most powerful agent tool.** They specify behavior, enable self-correction, and give agents confidence to refactor.
5. **Name things for discoverability.** Agents read names literally. Clear, consistent naming compounds across every agent interaction.
6. **Document the WHY.** ADRs and strategic comments prevent entire categories of agent mistakes.
7. **Refactor incrementally, guided by agent struggles.** Fix the areas where agents produce the worst output first.
8. **AI-friendly and human-friendly are the same thing.** Every improvement for agents also improves your team's experience.

This concludes Part II: Context Engineering. You now have the complete toolkit: context files for project-wide guidance, path-scoped rules for targeted instructions, skills for complex workflows, MCP servers for external data access, and codebase design principles that make everything work better.

Part III shifts from preparation to execution: the workflows and practices that make day-to-day agentic development productive, reliable, and safe.

PART III

The Tools

CHAPTER 11

IDE Agents

Your IDE is where you spend most of your day. It's where code gets written, reviewed, debugged, and shipped. So it makes sense that the most impactful AI tools are the ones embedded directly in your editor, the ones that understand your project, your open files, and your current intent without you having to explain anything.

But here's the problem: there are now at least four serious IDE-integrated AI tools, each with overlapping features, different strengths, and separate subscription costs. Picking the wrong one (or worse, using the right one badly) costs you time and money every single day.

This chapter breaks down the major players: GitHub Copilot, Cursor, Windsurf, and JetBrains AI Assistant. Not with marketing bullet points, but with real workflows, honest opinions, and a decision framework you can actually use.

GitHub Copilot: The Default Choice

Copilot is the tool most developers try first, and for good reason. It's built into VS Code (and Visual Studio, and JetBrains, and Neovim), it's backed by GitHub's deep integration with your repositories, and it's the most polished of the bunch.

Inline Completions

The original feature, and still the most used. You type, Copilot suggests. Accept with Tab. This sounds trivial until you realize how much of coding is boilerplate: property declarations, null checks, LINQ queries, constructor injection patterns. Copilot handles these with near-perfect accuracy in C# because it's seen millions of .NET projects.

The key to getting good completions: write clear, descriptive names. A method called `GetActiveUsersByDepartment` gets better completions than one called `GetData`. Your naming conventions are context. Treat them that way.

Copilot Chat

The chat panel in VS Code gives you a conversation interface with full awareness of your workspace. You can ask questions about your codebase, request explanations of unfamiliar code, or ask for refactoring suggestions. It's essentially a senior developer sitting next to you who has read every file in your project.

Where Chat shines is codebase archaeology. Joining a new project? Ask Chat: "How does the authentication middleware work in this project?" It will trace through the code and give you a coherent explanation. That alone saves hours of onboarding time.

Agent Mode

This is where Copilot became a real agent, not just an assistant. Agent Mode (launched in VS Code in early 2025 and refined throughout the year) can:

- Edit multiple files simultaneously
- Run terminal commands
- Execute your tests and iterate on failures
- Create and delete files
- Install packages

The workflow is simple. You describe what you want in the chat panel, select Agent Mode, and Copilot proposes a plan. It shows you the files it will touch, the changes it will make, and the commands it will run. You approve, and it executes. If tests fail, it reads the output and tries again.

Here's a real example. You're adding a new endpoint to an ASP.NET Core API:

```
Add a GET /api/orders/{id}/history endpoint that returns the status
change history for an order. Use the existing OrderRepository pattern.
Include a DTO, a service method, controller action, and a unit test.
```

Agent Mode will create the DTO class, add the service method, wire up the controller, and write a test. Four files, one prompt. You review the diff, tweak anything you don't like, and move on.

The limitation: Copilot Agent Mode is conservative by default. It asks for approval before running terminal commands, which is the right default but slows down iterative workflows. You can adjust this in settings, but be deliberate about what you auto-approve.

The Edits Panel

Introduced alongside Agent Mode, the Edits panel gives you a middle ground between Chat and full Agent Mode. You select files, describe a change, and Copilot generates a multi-file diff that you review and apply. No terminal access, no command execution, just code changes across files.

This is perfect for refactoring. "Rename the UserService to AccountService and update all references" is exactly the kind of task the Edits panel handles cleanly. It's less powerful than Agent Mode but faster for targeted changes.

Configuring Copilot

Copilot reads project-level instructions from `.github/copilot-instructions.md`. This file is your chance to tell Copilot how your project works:

```
# Copilot Instructions

## Architecture
This is an ASP.NET Core 10 Web API using Clean Architecture.
- Controllers in src/Api/Controllers/
- Business logic in src/Core/Services/
- Data access in src/Infrastructure/Repositories/
- DTOs in src/Api/Models/

## Conventions
- Use record types for DTOs
- Repository pattern with interfaces in Core, implementations in Infrastructure
- All endpoints return ActionResult<T>
- Use FluentValidation for input validation
- xUnit + NSubstitute for testing

## Do NOT
- Use static methods for business logic
- Put EF Core DbContext in controllers
- Skip null checks on repository returns
```

This file is loaded automatically when Copilot processes your workspace. Every interaction gets better because Copilot understands your conventions without you repeating them.

You can also add VS Code settings to fine-tune behavior:

```
{
  "github.copilot.chat.codeGeneration.useReferencedFiles": true,
  "github.copilot.chat.agent.runCommand.enabled": true
}
```

Model Selection

As of early 2026, Copilot lets you choose between multiple models: GPT-4.1, Claude Sonnet 4.5, and others. For C# work, Claude Sonnet tends to produce more idiomatic code. For quick completions and chat, GPT-4.1 is faster. You can switch models per conversation, so use the right one for the task.

Cursor: The Power User's Editor

Cursor is a fork of VS Code that went all-in on AI. If Copilot is AI added to an editor, Cursor is an editor built around AI. The difference shows in every interaction.

Tab Completion

Cursor's Tab completion is similar to Copilot's inline suggestions but more aggressive. It predicts multi-line completions, suggests entire code blocks, and adapts quickly to your patterns. The "Tab" model is specifically trained for code completion and feels noticeably faster than Copilot's inline suggestions for complex completions.

Where Cursor Tab really differentiates is in its ability to predict your *next edit*. After you change a method signature, Tab will suggest the corresponding changes in calling code. After you rename a variable, it suggests updates in related files. It's not just completing the current line; it's completing the current *thought*.

Chat and Inline Editing

Cursor's chat works similarly to Copilot's, but with one critical addition: you can `@mention` files, folders, documentation, and even web URLs directly in your prompts. `@src/Core/Services/OrderService.cs` pulls that file into context explicitly. `@docs` references your project documentation. This explicit context management makes a huge difference in output quality.

Inline editing (Cmd+K / Ctrl+K) lets you select code and describe a change in natural language. Cursor rewrites the selection in place. This is faster than chat for targeted

edits: select a method, type “add caching with a 5-minute expiration,” and the code transforms.

Agent Mode

Cursor’s Agent Mode (called Composer in earlier versions) is the most capable IDE agent as of early 2026. It handles multi-file editing, terminal execution, test running, and iterative development in a single flow.

The key differentiator: Cursor’s agent is more autonomous than Copilot’s. It will run commands, observe output, and iterate without asking for permission at every step (configurable, of course). For developers who trust the agent and have a good test suite, this means faster iteration cycles.

A typical Cursor Agent Mode workflow:

1. Open a new Composer tab
2. Describe the feature or fix
3. Cursor proposes a plan with file changes
4. You approve or modify the plan
5. Cursor executes, runs tests, fixes issues
6. You review the final diff

Background Agents

This is Cursor’s standout feature. Background Agents run in the cloud, working on tasks asynchronously while you continue coding. You describe a task, spin up a background agent, and it works independently. When it finishes, it creates a branch with its changes for your review.

The workflow is similar to cloud agents like Codex Cloud (covered in Chapter 12), but integrated directly into your editor. You can launch a background agent, switch to another task, and come back to review its work later.

Use cases that work well for Background Agents:

- Writing tests for existing code

- Refactoring large files into smaller ones
- Adding documentation comments
- Updating deprecated API calls
- Migrating configuration formats

Use cases that don't work well: anything requiring deep architectural understanding of runtime behavior, anything involving secrets or environment-specific configuration.

Configuring Cursor

Cursor reads project rules from `.cursor/rules/` (the newer format) or `.cursorrules` (legacy, still supported). The newer format supports path-scoped rules:

```
.cursor/
  rules/
    general.mdc       # Always applied
    csharp.mdc         # Applied when editing .cs files
    testing.mdc        # Applied when editing test files
```

A `.mdc` rule file looks like this:

```
---
description: Rules for C# source files
globs: ["**/*.cs"]
---

# C# Conventions

- Use file-scoped namespaces
- Prefer primary constructors where appropriate
- Use `required` keyword for mandatory properties on DTOs
- Collections should use IReadOnlyList<T> in return types
- Always use async/await for I/O operations, never .Result or .Wait()
```

The glob-based scoping is powerful. You can have different rules for your API layer, your domain layer, and your test project. The agent only sees the rules relevant to the files it's currently working with.

Cursor's Cost

Cursor Pro runs $20/month and includes generous usage of fast models. The Business plan at $40/month adds admin controls and team features. Compared to Copilot ($10/month for Individual, $19/month for Business), Cursor is more expensive but includes more capable agent features.

Windsurf (Codeium): The Underdog Worth Watching

Windsurf, built by Codeium, takes a different approach. While Copilot and Cursor focus on individual agent interactions, Windsurf centers its experience around what it calls Cascade and Flows.

Cascade

Cascade is Windsurf's agent mode, and its defining characteristic is deep codebase awareness. When you make a request, Cascade automatically identifies relevant files across your entire project, pulling them into context without you having to `@mention` anything. It reads your project structure, understands import chains, and follows references.

For large codebases, this automatic context gathering is genuinely useful. You don't have to know which files are relevant; Cascade figures it out. The trade-off is that it can sometimes pull in too much context, slowing down responses on very large projects.

Flows

Flows are Windsurf's take on multi-step workflows. Instead of a single prompt-response cycle, a Flow maintains state across multiple interactions, remembering what you've done and building on it. This is useful for complex tasks that unfold over multiple prompts: "First set up the database model, then create the repository, then add the API endpoint, then write the tests."

Each step in a Flow has access to the results of previous steps, which reduces the context you need to provide manually.

Where Windsurf Shines

Windsurf's free tier is the most generous of any IDE agent. You get meaningful access to AI features without paying anything, which makes it a solid choice for developers who want to try agentic development without committing to a subscription.

Windsurf also handles large monorepo codebases better than Cursor or Copilot in my experience. The automatic context gathering, while sometimes over-eager, scales better than manual `@mentions` when you're working across dozens of projects in a single solution.

Configuration

Windsurf reads project-level instructions from `.windsurfrules` in your project root. The format is similar to `.cursorrules`: plain markdown describing your conventions and preferences.

JetBrains AI Assistant

If you're a Rider user (and many .NET developers are), JetBrains AI Assistant deserves a mention. It integrates directly into Rider's existing refactoring, inspection, and navigation features, which gives it an advantage in code-aware suggestions.

The AI Assistant can generate code, explain unfamiliar code, suggest refactorings, and write tests. It's competent but not best-in-class for agentic workflows. As of early

2026, JetBrains doesn't offer the same level of autonomous agent mode that Copilot, Cursor, and Windsurf provide. It's more of an enhanced assistant than a true agent.

Where it does excel: if you're deeply embedded in the JetBrains ecosystem and use Rider's refactoring tools heavily, the AI Assistant enhances those existing workflows rather than replacing them. It suggests ReSharper-style fixes powered by AI, which feels natural if you're already a Rider power user.

JetBrains also announced Junie, their dedicated coding agent, which operates more autonomously. Keep an eye on it, but as of this writing, it's still catching up to the competition in capability.

The Decision Matrix

Here's when to reach for each tool:

Choose Copilot when: - You want the most stable, well-integrated experience in VS Code or Visual Studio - You're already paying for GitHub and want seamless repository integration - You need inline completions that just work with minimal configuration - Your team needs a single, standardized tool with admin controls - You want access to multiple models (GPT-4.1, Claude) in one subscription

Choose Cursor when: - You want the most powerful agent mode available in an IDE - You use Background Agents for parallel async work - You want fine-grained, path-scoped rules for your project - You're comfortable with a VS Code fork (same extensions, slightly different update cycle) - You're a power user who wants maximum control over AI interactions

Choose Windsurf when: - You're working with large monorepos or complex multi-project solutions - You want automatic context gathering without manual `@mentions` - Budget matters and you want a strong free tier - You prefer the Flows model for multi-step development tasks

Choose JetBrains AI when: - Rider is your primary editor and you won't switch - You want AI that enhances existing JetBrains refactoring workflows - You value tight integration with ReSharper inspections

Subscription Stacking: The Cost Reality

Let's talk money, because nobody else does.

Most serious agentic developers are paying for at least two subscriptions. A typical stack:

Tool	Cost/ Month	What You Get
GitHub Copilot Individual	$10	Inline completions, Chat, Agent Mode
Cursor Pro	$20	Tab, Chat, Agent, Background Agents
Claude Pro (for Claude Code)	$20	CLI agent (covered in Chapter 12)
Total	**$50**	

Is $50/month worth it? If you're billing $100+/hour and these tools save you even 30 minutes per day, that's over $1,500/month in recovered time. The math isn't even close.

But don't pay for redundancy. If you're using Cursor as your primary editor, you probably don't need Copilot's inline completions (Cursor's Tab is comparable). If you're using Copilot Agent Mode heavily, you might not need Cursor's agent features.

My recommendation: start with one tool, learn it deeply, then add a second only if you hit clear limitations. For most .NET developers, the highest-impact stack is Cursor (primary editor) plus Claude Code (CLI agent for terminal work). More on that combination in Chapter 12.

Real Workflow: Building a Feature with an IDE Agent

Let's walk through a realistic workflow. You need to add a feature to an ASP.NET Core API: an endpoint that lets users export their order history as a CSV file.

Step 1: Write the spec (you, not the agent)

Before touching the IDE, write what you want in plain language. This becomes your prompt:

```
Add a GET /api/orders/export endpoint that:
- Returns a CSV file of the authenticated user's orders
- Includes columns: OrderId, Date, Status, Total, ItemCount
- Filters by optional date range (startDate, endDate query params)
- Returns 404 if the user has no orders
- Uses the existing IOrderRepository and ICurrentUserService
- Follows the existing controller patterns in OrdersController
- Add a unit test for the service method and an integration test for the endpoint
```

Step 2: Let the agent work

Paste this into Agent Mode (Copilot or Cursor). The agent will: 1. Create a `CsvExportService` with the export logic 2. Add the endpoint to `OrdersController` 3. Create a DTO for the query parameters 4. Write unit tests for the service 5. Write an integration test for the endpoint 6. Run the tests

Step 3: Review

This is where your expertise matters. Check: - Does the CSV generation handle special characters (commas in product names)? - Is the date filtering applied at the database level, not in memory? - Are the tests testing behavior, not implementation? - Does the endpoint properly handle the case where the date range is invalid?

Step 4: Iterate

Tell the agent what to fix. “The date filtering should be applied in the repository query, not after fetching all orders.” The agent makes the change, reruns the tests, and you review again.

Total time: 15-20 minutes for a feature that would have taken an hour or more by hand. And the code is consistent with your existing patterns because the agent read your context files.

Context Files: The Setup That Pays for Itself

Every IDE agent reads some form of project-level configuration. Here’s a quick reference:

Tool	File/Location	Format
Copilot	`.github/copilot-instructions.md`	Markdown
Cursor	`.cursor/rules/*.mdc` or `.cursorrules`	Markdown with frontmatter
Windsurf	`.windsurfrules`	Markdown
Claude Code	`CLAUDE.md`	Markdown
Cross-tool	`AGENTS.md`	Markdown (emerging standard)

The emerging `AGENTS.md` standard (from agents.md) aims to unify these into a single file that all tools can read. It’s worth adopting now: put your main instructions in `AGENTS.md` at the project root, then create tool-specific files that reference or extend it.

What to put in these files was covered in depth in Chapter 4 (Context Engineering). The short version: build commands, architecture overview, coding conventions, testing patterns, and explicit “do not” rules. Keep it under 500 lines. Review and update it monthly.

The Bottom Line

IDE agents are the most accessible entry point to agentic development. They meet you where you already work, in your editor, and they layer intelligence on top of your existing workflow.

But they’re only one piece of the puzzle. For tasks that require deep codebase reasoning, long-running operations, or parallel execution, you need the tools covered in the next chapter: CLI agents and cloud agents. The real power comes from combining both.

CHAPTER 12

CLI and Cloud Agents

IDE agents are where most developers start. CLI and cloud agents are where the serious productivity gains live.

The difference is fundamental. An IDE agent operates inside your editor, constrained by the UI, limited to the files you have open, and dependent on your active attention. A CLI agent operates in your terminal with full access to your development environment: your file system, your build tools, your test runner, your git history, everything. A cloud agent doesn't even need your machine running. It works in the background, creates a PR, and pings you when it's done.

If IDE agents are your pair programmer, CLI agents are your senior developer who works independently and comes back with a pull request. Cloud agents are the contractor you hire for well-defined tasks and check in on later.

This chapter covers the major players, when to use each, and how to combine them into a workflow that lets you do the work of a small team.

Claude Code: The Terminal-Native Agent

Claude Code is, in my opinion, the most capable coding agent available as of early 2026. It runs entirely in your terminal, which sounds like a limitation until you realize it's actually a superpower.

Because Claude Code operates in the terminal, it has access to everything your terminal has access to: your full file system, your shell, your environment variables, your build tools, your test runner, your git configuration. It doesn't need a plugin or extension. It doesn't need a specific editor. It works everywhere.

What Claude Code Actually Does

When you launch Claude Code in a project directory, it reads your codebase and understands your project structure. Then you give it a task in natural language:

```
$ claude
> Add rate limiting to the /api/orders endpoint. Use the built-in
  ASP.NET Core rate limiter with a fixed window of 100 requests
  per minute per user. Add integration tests.
```

Claude Code will:

1. Read your existing code to understand the patterns
2. Identify which files need changes
3. Make the edits
4. Run your tests to verify nothing broke
5. If tests fail, read the output and iterate
6. Present you with a summary of changes

The key differentiator: Claude Code *runs your tools*. It doesn't just generate code and hope for the best. It executes `dotnet build`, reads the compiler errors, fixes them, runs `dotnet test`, reads the failures, and fixes those too. This iterative loop is what separates it from chat-based tools.

Sub-Agents

Claude Code can spawn sub-agents for parallel work. When you give it a complex task, it can break it into pieces and assign each piece to a separate agent. One sub-agent researches the existing codebase structure. Another implements the feature. A third writes the tests. The main agent orchestrates.

This isn't theoretical. On a task like "refactor the notification system from synchronous to event-driven," Claude Code might:

1. Spawn a sub-agent to analyze the current notification call sites

2. Spawn another to design the event schema
3. Implement the changes in the main agent
4. Spawn a sub-agent to update all the tests

The result is faster completion of complex, multi-file tasks.

Skills

Skills are reusable instruction sets that Claude Code loads on demand. Think of them as specialized playbooks. You might have a skill for “creating a new ASP.NET Core endpoint” that includes your specific patterns: where to put the controller, how to structure the DTO, which validation library to use, which test patterns to follow.

Skills are stored as markdown files and loaded by description matching. When Claude Code encounters a task that matches a skill’s description, it reads the skill file and follows the instructions. This is more powerful than context files because skills can include step-by-step procedures, not just conventions.

Setting Up Claude Code

The most important setup step is creating a `CLAUDE.md` file at the root of your project. This is Claude Code’s equivalent of `.cursorrules` or `copilot-instructions.md`, but it tends to be more detailed because Claude Code is more autonomous and needs more guidance.

A solid `CLAUDE.md` for a .NET project:

```
# CLAUDE.md

## Project
ASP.NET Core 10 Web API with Clean Architecture.
Solution file: GameStore.sln

## Build & Test
- Build: `dotnet build`
- Test: `dotnet test --no-build`
- Single test: `dotnet test --filter "FullyQualifiedName~TestName"`
- Run: `cd src/Api && dotnet run`

## Architecture
- src/Api/ - Controllers, DTOs, middleware
- src/Core/ - Domain entities, interfaces, services
- src/Infrastructure/ - EF Core, external services
- tests/Api.Tests/ - Integration tests (WebApplicationFactory)
- tests/Core.Tests/ - Unit tests

## Conventions
- File-scoped namespaces
- Record types for DTOs
- Primary constructors where appropriate
- IReadOnlyList<T> for collection return types
- Async all the way down, never .Result or .Wait()
- FluentValidation for request validation
- xUnit + NSubstitute for testing

## Patterns
- Repository pattern: interface in Core, implementation in Infrastructure
- CQRS-lite: separate query and command services, no full MediatR
- Result pattern for service returns (not exceptions for business logic)

## Do NOT
- Add packages without asking
- Modify the DbContext directly (use migrations)
```

```
- Use Console.WriteLine (use ILogger)
- Skip writing tests
```

Permissions and Allowed Tools

Claude Code has a permission system that controls what it can do without asking. Configure this based on your comfort level:

```
# Allow running build and test commands without prompting
claude config set allowedTools "dotnet build" "dotnet test" "dotnet run" "git status" "git diff
```

For trusted projects where you have a good test suite, allowing build and test commands to run without approval dramatically speeds up the iterative loop. Claude Code builds, tests, observes failures, and fixes them without interrupting you for permission at each step.

For less trusted contexts (unfamiliar codebases, production-adjacent work), keep the defaults restrictive and approve each command manually.

Codex CLI: OpenAI's Terminal Agent

Codex CLI is OpenAI's answer to Claude Code. It runs in your terminal, understands your project, and can make multi-file edits. It's powered by GPT-5-Codex (or whatever model OpenAI is shipping when you read this).

How It Compares to Claude Code

Codex CLI is competent and improving rapidly. Its strengths:

- Tight integration with OpenAI's model ecosystem
- Good at following existing code patterns
- Solid at explaining code and reasoning through problems
- Fast response times

Its weaknesses relative to Claude Code (as of early 2026):

- Less autonomous in iterative workflows (doesn't loop on test failures as aggressively)
- Smaller effective context window for very large codebases
- Fewer extensibility features (no equivalent to Claude Code's skills system)

That said, having both installed gives you optionality. Some tasks respond better to one model's reasoning style than the other. When Claude Code struggles with a particularly gnarly debugging session, switching to Codex CLI sometimes produces a fresh perspective that cracks the problem.

Basic Usage

```
$ codex
> Fix the flaky test in OrderServiceTests.cs. It intermittently
  fails on the date comparison assertion. Find the root cause
  and fix it properly, don't just add retry logic.
```

Codex CLI will read the test, trace the code under test, identify the timezone or timing issue, and propose a fix. Like Claude Code, it can run your build and test commands to verify its changes.

Cloud Agents: Fire and Forget

Cloud agents represent a fundamentally different workflow. You don't sit with them. You don't watch them work. You assign a task, walk away, and come back to a pull request.

Codex Cloud (GitHub)

Codex Cloud (previously called GitHub Copilot Coding Agent, now accessible through what GitHub calls "Agent HQ") runs in GitHub's cloud infrastructure. You can launch it from:

- The GitHub web interface
- GitHub Mobile (your phone)
- VS Code's agent panel

The workflow:

1. Open a GitHub issue or describe a task
2. Assign it to Codex Cloud
3. The agent spins up an environment, clones your repo, makes changes
4. It runs your CI pipeline (if configured)
5. It opens a pull request with its changes
6. You review the PR like any other

This is the "fire-from-phone" workflow that changes how you think about productivity. Sitting in a waiting room? Open GitHub Mobile, create an issue: "Add pagination to the /api/products endpoint, following the existing pagination pattern in /api/orders." Assign it to Codex. Put your phone away. By the time you're back at your desk, there's a PR waiting.

What works well with Codex Cloud:

- Well-defined, scoped tasks with clear acceptance criteria
- Tasks that follow existing patterns in your codebase
- Bug fixes with clear reproduction steps
- Adding tests for existing untested code
- Documentation updates

What doesn't work well:

- Architectural changes that require deep understanding of system behavior
- Tasks that need access to running services or databases
- Anything requiring interactive debugging
- Tasks where the spec is ambiguous

Google Jules

Jules is Google's background coding agent, integrated with GitHub repositories. It occupies similar territory to Codex Cloud: you assign a task, it creates a branch, makes changes, and opens a PR.

Jules differentiates itself with its plan-first approach. Before writing any code, Jules generates a detailed plan and asks for your approval. This adds a step but catches misunderstandings early. If the plan looks wrong, you redirect before any code is written, saving time overall.

Jules works well for:

- Dependency updates and migration tasks
- Code cleanup and refactoring
- Implementing features from well-written specs
- Bug fixes with clear context

Amazon Q Developer

Amazon Q Developer is AWS's entry in the coding agent space. If you're working heavily with AWS services (and many .NET developers are, especially those using Lambda, ECS, or App Runner), Q Developer has the deepest understanding of AWS APIs, IAM policies, CloudFormation templates, and AWS-specific patterns.

For .NET on AWS specifically, Q Developer handles:

- Lambda function scaffolding and deployment configuration
- CDK (Cloud Development Kit) template generation

- IAM policy generation with least-privilege principles
- AWS SDK integration patterns

If your stack is Azure-centric, Q Developer is less compelling. But if you're deploying .NET to AWS, it's worth having in your toolkit alongside your primary agent.

When CLI Beats IDE (and Vice Versa)

This isn't an either/or choice. CLI and IDE agents excel at different things.

Reach for CLI agents when:

- The task involves multiple files across the codebase
- You need the agent to run builds, tests, or other tools
- The task requires reading and understanding a large portion of the codebase
- You want the agent to iterate autonomously until tests pass
- You're working on a task that doesn't require visual context (no UI work)
- You want to work in parallel, running the agent in one terminal while you code in another

Reach for IDE agents when:

- You're doing inline edits to a single file or small set of files
- You need visual diff review in the editor
- The task benefits from seeing the code structure in the file tree
- You're doing exploratory work where you want to see completions as you type
- You need tight integration with editor features (debugging, go-to-definition)

The practical split: Use your IDE agent for the code you're actively writing and thinking about. Use CLI agents for tasks you want to delegate and review later. Use cloud agents for tasks you want to fire and forget entirely.

The Parallel Coding Agent Lifestyle

Simon Willison coined this term, and it describes the workflow that delivers the biggest productivity multiplier: running multiple agents simultaneously on different tasks.

The concept: instead of doing one thing at a time, you break your work into independent tasks and assign each to a separate agent. While they work, you either work on something yourself or orchestrate more agents. Your role shifts from implementer to project manager.

Git Worktrees: The Enabler

The practical foundation for parallel agents is git worktrees. A worktree lets you have multiple working copies of the same repository, each on a different branch, without cloning the repo multiple times.

```
# Create worktrees for parallel agent sessions
git worktree add ../myproject-feature-auth feature/auth-refactor
git worktree add ../myproject-feature-export feature/csv-export
git worktree add ../myproject-fix-flaky fix/flaky-tests
```

Now you have three separate directories, each on its own branch. Launch a Claude Code session in each:

```
# Terminal 1
cd ../myproject-feature-auth
claude "Refactor the auth middleware to support both JWT and API key authentication"

# Terminal 2
cd ../myproject-feature-export
claude "Add CSV export to the orders endpoint, following the existing export pattern in reports

# Terminal 3
cd ../myproject-fix-flaky
claude "Find and fix all flaky tests. Run the test suite 3 times to verify stability"
```

Three agents working simultaneously on three different tasks. You check in periodically, review their progress, provide guidance if they're stuck, and approve or reject their changes.

When they finish, merge the branches:

```
git worktree remove ../myproject-feature-auth
git worktree remove ../myproject-feature-export
git worktree remove ../myproject-fix-flaky
```

What Works for Parallel Agents

Simon Willison identified four categories of tasks that work well in parallel:

1. **Research and proof-of-concept**: "Investigate whether we can replace Newtonsoft.Json with System.Text.Json. Identify all breaking changes."
2. **Codebase questions**: "Document how the payment processing pipeline works. Trace from API endpoint to Stripe webhook handler."
3. **Small maintenance tasks**: "Update all deprecated API calls to their modern equivalents." "Fix all compiler warnings."
4. **Carefully specified implementation**: Tasks where you've written a clear spec. This is the most productive category because the agent has clear success criteria.

What Doesn’t Work

Don’t run parallel agents on tasks that touch the same files. Merge conflicts are painful enough between human developers. Between agents that don’t know about each other, they’re worse.

Don’t run parallel agents on architecturally coupled tasks. If task A changes the database schema and task B adds a new query, they need to be sequential, not parallel.

And don’t run more agents than you can review. The bottleneck in parallel agent work is always review. Five agents producing code simultaneously sounds productive until you realize you have five PRs to review, each with 200+ lines of changes. Start with two or three parallel agents and scale up as your review speed improves.

The Fire-From-Phone Workflow

This deserves its own section because it’s genuinely transformative. The ability to launch coding tasks from your phone changes the economics of dead time.

The setup:

1. **GitHub Mobile** installed on your phone
2. **Codex Cloud** (or equivalent cloud agent) enabled on your repository
3. **CI/CD pipeline** that runs tests automatically on new PRs
4. **Clear task templates** so you can describe tasks quickly from a small screen

The workflow:

- Commuting, waiting, walking, whatever: you think of something that needs doing
- Open GitHub Mobile, create an issue with a clear description
- Assign it to the cloud agent
- Put your phone away
- Later, at your desk, the PR is waiting with CI status

Tasks that work from phone:

- “Add input validation to the CreateOrder endpoint”
- “Write unit tests for the DiscountCalculator class”
- “Update the README with the new deployment steps”
- “Fix the N+1 query in the GetOrdersWithItems method”

Tasks that don’t work from phone (not enough screen real estate to spec properly):

- Complex architectural changes
- Features requiring detailed acceptance criteria
- Anything where the spec needs diagrams or extensive context

A Day in the Life: Combining Everything

Here’s what a real day looks like when you combine IDE, CLI, and cloud agents:

8:00 AM, coffee in hand. Open Cursor. Review the two PRs that Codex Cloud created overnight (you launched them from your phone yesterday evening). One looks good, approve and merge. The other needs tweaks, leave comments.

8:30 AM. Start your main task for the day: redesigning the notification system. Write a spec document first (30 minutes of focused thinking, no agents). Break it into three subtasks.

9:00 AM. Launch Claude Code in a worktree for subtask 1 (new event types and handlers). Open Cursor for subtask 2 (updating the API endpoints), which you’ll do yourself with Agent Mode because it requires design decisions. Assign subtask 3 (updating tests) to a Cursor Background Agent.

9:30 AM. Claude Code has a draft of the event handlers. Review in terminal, provide feedback (“use the MediatR notification pattern, not raw events”). It iterates. Meanwhile, you’re working on the API changes in Cursor.

10:00 AM. Your Cursor Background Agent finishes updating the tests. Review the diff. Mostly good, but it missed an edge case. Add a comment and let it iterate.

10:30 AM. All three subtasks are done. Merge the branches. Run the full test suite. Two failures from integration issues between the subtasks. Fix them manually (5 minutes, this is where your expertise matters).

11:00 AM. During a break, open your phone. Create two GitHub issues for tomorrow's work: a performance optimization and a documentation update. Assign both to Codex Cloud. They'll be PRs by morning.

That's five developer-tasks completed by lunch, with one person. Not by working faster, but by working in parallel and focusing your attention on the parts that require human judgment: specs, architecture, reviews, and integration.

Setting Up Your Multi-Agent Environment

Here's the practical setup to enable this workflow:

1. Install the tools

```
# Claude Code
npm install -g @anthropic-ai/claude-code

# Codex CLI
npm install -g @openai/codex-cli

# Cursor (download from cursor.com)
# Copilot (VS Code extension)
```

2. Create project context files

At minimum, create `CLAUDE.md` and `.cursor/rules/general.mdc` for your project. If you want cross-tool compatibility, create an `AGENTS.md` that both tools can reference.

3. Set up git worktree aliases

Add to your shell config:

```
# Quick worktree creation for agent sessions
agent-worktree() {
  local branch="agent/$1"
  local dir="../$(basename $PWD)-$1"
  git checkout -b "$branch" 2>/dev/null || true
  git worktree add "$dir" "$branch"
  echo "Worktree ready at $dir"
}

# Quick cleanup
agent-cleanup() {
  for dir in ../$(basename $PWD)-*; do
    [ -d "$dir" ] && git worktree remove "$dir" --force
  done
  git branch --list 'agent/*' | xargs -r git branch -D
}
```

4. Configure Claude Code permissions

```
# For trusted projects with good test coverage
claude config set allowedTools \
  "dotnet build" \
  "dotnet test" \
  "dotnet run" \
  "git status" \
  "git diff" \
  "git add" \
  "git commit"
```

5. Set up CI for cloud agents

Ensure your GitHub Actions (or equivalent) run tests on every PR. Cloud agents create PRs, and your CI pipeline is their verification step. Without CI, you're reviewing agent code without any automated safety net.

The Cost of Multi-Agent Work

Let's be honest about costs. Running multiple agents simultaneously burns tokens fast.

A typical day of heavy agent use:

Activity	Estimated Cost
Claude Code (3-4 sessions)	$5-15
Cursor Pro (subscription)	~$1/day
Copilot (subscription)	~$0.50/day
Codex Cloud (2-3 tasks)	$2-5
Daily total	**$8-22**

That's $200-500/month for heavy users. Sounds steep until you compare it to developer salaries. If these tools save you 2 hours per day at a $75/hour effective rate, that's $3,000/month in recovered time. The ROI is 6-15x.

But monitor your usage. Token costs can spike unexpectedly on complex tasks where agents iterate many times. Set budget alerts on your Claude and OpenAI accounts. Review your usage weekly.

The Bottom Line

CLI and cloud agents are where the "parallel coding agent lifestyle" becomes real. IDE agents help you code faster. CLI agents let you delegate entire tasks. Cloud agents let you delegate from anywhere, even your phone.

The developers getting the most out of this era aren't the ones using a single tool really well. They're the ones combining multiple tools, each in its sweet spot: IDE agents for active coding, CLI agents for complex delegated tasks, cloud agents for async fire-and-forget work.

The setup takes an afternoon. The payoff lasts your entire career. Or at least until the next paradigm shift, which, at the current pace, might be next Tuesday.

PART IV

The Workflows

CHAPTER 13

The TDD Agent Loop

This is the workflow that changed everything for me. Not gradually, not theoretically, but immediately and measurably. The TDD Agent Loop is the single highest-confidence way to work with AI coding agents, and once you've used it, going back feels reckless.

The concept is simple: write the tests first, then let the agent write the code to make them pass.

That's it. That's the workflow. But the devil (and the value) is in the details of how you structure the tests, how you prompt the agent, how you handle failures, and how you review the result.

Why TDD + Agents Is a Perfect Match

Test-driven development has been around since the early 2000s. It's always been a good practice. But it had a cost: writing tests first is slower than writing code first, at least in the short term. You're essentially writing the code twice, once as a specification (the tests) and once as an implementation.

Agents eliminate the second half of that cost. You write the tests. The agent writes the implementation. The total time is less than writing just the implementation by hand, because:

1. **The tests serve as an unambiguous spec.** The agent doesn't have to guess what you want. The tests define it.
2. **The agent can self-correct.** When a test fails, the agent sees the failure message, understands what went wrong, and fixes it. No human in the loop needed for mechanical corrections.

3. **Your review is simplified.** Instead of reviewing the code to see if it works, you verify that the tests are comprehensive and the code is clean. The “does it work?” question is already answered.
4. **Confidence is built-in.** When all tests pass, you know the code does what you specified. Not what the agent assumed. What you specified.

Addy Osmani put it well: “The more comprehensive your tests, the more confidently you can delegate.” The TDD Agent Loop takes this to its logical conclusion: your tests ARE your delegation.

The Loop in Detail

Here’s the workflow, step by step.

Step 1: Understand the Requirement

Before writing anything, be clear on what you’re building. This sounds obvious, but agents amplify confusion. If you’re unclear about the requirement, you’ll write vague tests, the agent will write code that passes vague tests, and you’ll end up with code that technically works but doesn’t actually solve the problem.

Spend 5 minutes with the requirement. Write it in plain language if it helps:

```
I need a service that calculates shipping costs.
- Free shipping for orders over $100
- Flat rate $5.99 for standard shipping
- Flat rate $12.99 for express shipping
- No shipping to PO boxes for express
- Tax is NOT applied to shipping costs
```

This isn’t a prompt for the agent yet. This is your own understanding.

Step 2: Write the Tests

Now write the tests. All of them. Before any implementation exists.

```
public class ShippingCalculatorTests
{
    private readonly ShippingCalculator _calculator = new();

    [Fact]
    public async Task Calculate_OrderOver100_StandardShipping_ReturnsFree()
    {
        // Arrange
        var order = new Order { Total = 150.00m, ShippingMethod = ShippingMethod.Standard };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(0m);
    }

    [Fact]
    public async Task Calculate_OrderUnder100_StandardShipping_ReturnsFlatRate()
    {
        // Arrange
        var order = new Order { Total = 50.00m, ShippingMethod = ShippingMethod.Standard };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(5.99m);
    }

    [Fact]
    public async Task Calculate_OrderOver100_ExpressShipping_ReturnsFree()
    {
        // Arrange
        var order = new Order { Total = 200.00m, ShippingMethod = ShippingMethod.Express };
```

```
        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(0m);
    }

    [Fact]
    public async Task Calculate_OrderUnder100_ExpressShipping_ReturnsFlatRate()
    {
        // Arrange
        var order = new Order { Total = 75.00m, ShippingMethod = ShippingMethod.Express };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(12.99m);
    }

    [Fact]
    public async Task Calculate_ExpressToPoBox_ThrowsInvalidOperationException()
    {
        // Arrange
        var order = new Order
        {
            Total = 50.00m,
            ShippingMethod = ShippingMethod.Express,
            Address = new Address { Line1 = "PO Box 123" }
        };

        // Act
        var act = () => _calculator.CalculateAsync(order);

        // Assert
        await act.Should().ThrowAsync<InvalidOperationException>()
```

```
            .WithMessage("*PO Box*");
    }

    [Fact]
    public async Task Calculate_ShippingCost_DoesNotIncludeTax()
    {
        // Arrange
        var order = new Order
        {
            Total = 50.00m,
            ShippingMethod = ShippingMethod.Standard,
            TaxRate = 0.08m
        };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
        result.ShippingCost.Should().Be(5.99m);
        result.TaxOnShipping.Should().Be(0m);
    }

    [Theory]
    [InlineData(99.99, 5.99)]
    [InlineData(100.00, 5.99)]
    [InlineData(100.01, 0)]
    public async Task Calculate_BoundaryValues_ReturnsCorrectCost(
        decimal orderTotal, decimal expectedShipping)
    {
        // Arrange
        var order = new Order { Total = orderTotal, ShippingMethod = ShippingMethod.Standard };

        // Act
        var result = await _calculator.CalculateAsync(order);

        // Assert
```

```
        result.ShippingCost.Should().Be(expectedShipping);
    }
}
```

A few things to notice about these tests:

They're thorough. Happy path, edge cases, boundary values, error conditions. This isn't minimal test coverage. This is a complete specification.

They define the API. The tests force you to decide on the interface: `ShippingCalculator` with a `CalculateAsync` method that takes an `Order` and returns a result with `ShippingCost` and `TaxOnShipping`. You're designing the API, not the agent.

They won't compile yet. `ShippingCalculator`, `Order`, `ShippingMethod`, `Address`, none of these exist. That's fine. That's the point. The tests define what needs to exist.

The boundary test is critical. Is $100.00 exactly free or not? You decided: it's NOT free (the tests show $100.00 maps to $5.99, and $100.01 maps to free). That's a business decision you made explicitly. Without the test, the agent would guess, and guessing business rules is how bugs are born.

> ***Note:*** *These examples use C# with xUnit and FluentAssertions, but the TDD Agent Loop works identically with any language and test framework. Python with pytest, TypeScript with Jest, Go with the testing package, Java with JUnit. The pattern is universal.*

Step 3: Prompt the Agent

Now you hand it to the agent. The prompt is simple because the tests do the heavy lifting:

```
Make all the tests in ShippingCalculatorTests pass.

The tests are in tests/Domain.Tests/ShippingCalculatorTests.cs.
Create the necessary types and implementation in src/Domain/.

Run the tests after implementation to verify they pass.
```

That's it. No 50-line prompt. No detailed specification. The tests ARE the specification.

If you've set up your AGENTS.md (Chapter 4), the agent already knows your project structure, coding conventions, and architecture patterns. If you have path-scoped rules, it knows the conventions for the Domain project specifically.

Step 4: Let the Agent Iterate

Here's where the magic happens. The agent will:

1. Read the test file
2. Understand what types need to exist (`Order` , `Address` , `ShippingMethod` , `ShippingResult` , `ShippingCalculator`)
3. Create those types
4. Implement `CalculateAsync`
5. Run `dotnet test`
6. See which tests fail (if any)
7. Fix the failures
8. Run the tests again
9. Repeat until all tests pass

This iteration loop is the key to the whole workflow. The agent isn't generating code and hoping it works. It's generating code, testing it, and fixing what's broken. The feedback loop is tight, automatic, and objective.

In my experience, most tasks complete in 1-3 iterations. The agent gets it mostly right on the first try (thanks to the tests being clear and the context file providing conventions), catches a bug or two on the test run, and fixes them.

Step 5: Review the Result

When all tests pass, the agent presents you with the implementation. Now you review.

But here's what's different about this review compared to reviewing code without the TDD loop: **you already know it works.** The tests pass. The behavior is correct. Your review focuses on:

Code quality: Is it clean? Is it readable? Does it follow your conventions?

Design decisions: Did the agent make reasonable choices about how to structure the code? Is the approach maintainable?

Edge cases you missed: Did the agent handle any cases your tests didn't cover? (Sometimes agents add defensive code that's valuable. Sometimes they add unnecessary complexity.)

Security and performance: Does anything look risky? For a shipping calculator, probably not. For an authentication handler, this review is critical.

Here's what the agent might produce:

```
namespace Domain;

public enum ShippingMethod
{
    Standard,
    Express
}

public record Address
{
    public required string Line1 { get; init; }
    public string? Line2 { get; init; }
    public string? City { get; init; }
    public string? State { get; init; }
    public string? ZipCode { get; init; }
}

public record Order
{
    public decimal Total { get; init; }
    public ShippingMethod ShippingMethod { get; init; }
    public Address? Address { get; init; }
    public decimal TaxRate { get; init; }
}

public record ShippingResult
{
    public decimal ShippingCost { get; init; }
    public decimal TaxOnShipping { get; init; }
}

public class ShippingCalculator
{
    private const decimal FreeShippingThreshold = 100.01m;
    private const decimal StandardRate = 5.99m;
    private const decimal ExpressRate = 12.99m;
```

```
    public Task<ShippingResult> CalculateAsync(Order order)
    {
        if (order.ShippingMethod == ShippingMethod.Express &&
            order.Address?.Line1?.Contains("PO Box", StringComparison.OrdinalIgnoreCase) == tru
        {
            throw new InvalidOperationException(
                "Express shipping is not available for PO Box addresses.");
        }

        var isFreeShipping = order.Total >= FreeShippingThreshold;

        var shippingCost = isFreeShipping
            ? 0m
            : order.ShippingMethod == ShippingMethod.Express
                ? ExpressRate
                : StandardRate;

        return Task.FromResult(new ShippingResult
        {
            ShippingCost = shippingCost,
            TaxOnShipping = 0m
        });
    }
}
```

Review checklist: - ✅ All tests pass - ✅ Uses records for DTOs (matches our convention) - ✅ Constants extracted (clean) - ✅ Async pattern correct (even though this is synchronous internally, the interface is async for future flexibility) - ✅ PO Box check is case-insensitive (good defensive choice) - ⚠️ The “PO Box” detection is naive (only checks Line1, only checks exact string). Might want to improve this later.

That last point is a judgment call. The agent implemented what the tests specified. If you want better PO Box detection, write a test for it. That’s the loop.

Step 6: Expand (Optional)

Sometimes after the initial implementation, you think of additional cases:

```
[Fact]
public async Task Calculate_ExpressToPoBox_CaseInsensitive_ThrowsException()
{
    var order = new Order
    {
        Total = 50.00m,
        ShippingMethod = ShippingMethod.Express,
        Address = new Address { Line1 = "po box 456" }
    };

    var act = () => _calculator.CalculateAsync(order);

    await act.Should().ThrowAsync<InvalidOperationException>();
}

[Fact]
public async Task Calculate_ExpressToPoBoxInLine2_ThrowsException()
{
    var order = new Order
    {
        Total = 50.00m,
        ShippingMethod = ShippingMethod.Express,
        Address = new Address { Line1 = "123 Main St", Line2 = "PO Box 789" }
    };

    var act = () => _calculator.CalculateAsync(order);

    await act.Should().ThrowAsync<InvalidOperationException>();
}
```

Run the agent again: “Make the new tests pass without breaking existing tests.” The agent updates the implementation. You review. Cycle complete.

When the Loop Shines

The TDD Agent Loop is particularly effective for:

Business Logic

Services, calculators, validators, anything with clear inputs and outputs. These are the tasks where tests are easiest to write and agent implementations are most reliable.

API Endpoints (Integration Tests)

Write integration tests that hit your endpoints through `WebApplicationFactory`, specifying the expected request/response contracts. Let the agent build the full vertical slice: controller, handler, repository calls.

```
[Fact]
public async Task CreateOrder_ValidRequest_Returns201WithLocation()
{
    // Arrange
    var client = _factory.CreateClient();
    var request = new CreateOrderRequest("SKU-001", 2);

    // Act
    var response = await client.PostAsJsonAsync("/api/v1/orders", request);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.Created);
    response.Headers.Location.Should().NotBeNull();

    var order = await response.Content.ReadFromJsonAsync<OrderResponse>();
    order.Should().NotBeNull();
    order!.Sku.Should().Be("SKU-001");
    order.Quantity.Should().Be(2);
}
```

This single test tells the agent to create the endpoint, the request/response types, the handler, and wire everything up. The agent builds the full feature to make the test pass.

Refactoring

Write tests that capture the current behavior, then ask the agent to refactor the implementation. The tests ensure the refactoring doesn't break anything. This is one of the safest ways to let agents work on existing code.

Bug Fixes

Write a test that reproduces the bug (it should fail). Ask the agent to make it pass without breaking existing tests. The bug fix is verified by definition.

When the Loop Struggles

The TDD Agent Loop isn't a silver bullet. It works poorly for:

UI work. Testing visual output is hard to express in unit/integration tests. You can test behavior (click this, expect this state) but visual correctness requires human eyes.

Exploratory work. When you don't know what you want yet, you can't write tests for it. Use agents in chat/explore mode first, figure out the approach, then switch to TDD for the implementation.

Infrastructure and configuration. Testing Terraform modules, Docker configurations, or CI/CD pipelines requires different tools than `dotnet test`. The loop concept still applies (define expected behavior, let the agent implement, verify) but the test framework is different.

Performance optimization. Tests verify correctness, not performance. An agent can make all tests pass with an $O(n^2)$ algorithm when you needed $O(n \log n)$. Performance review remains a human judgment.

The Full Cycle in Practice

Let me walk through a realistic workday scenario using the TDD loop.

9:00 AM: You pick up a ticket to add a discount code system. You spend 15 minutes understanding the requirements and sketching the domain model on paper.

9:15 AM: You write 8 tests covering: valid discount application, expired code rejection, usage limit enforcement, minimum order threshold, percentage vs. fixed discounts, stacking rules, and case-insensitive code lookup.

9:45 AM: You prompt the agent: "Make all tests in DiscountServiceTests pass. Create types in src/Domain/Discounts/. Follow the existing patterns in src/Domain/Shipping/ for reference."

9:50 AM: Agent completes. All 8 tests pass on the second iteration (it missed the case-insensitive lookup on the first try, caught it when the test failed, fixed it).

9:55 AM: You review the implementation. Clean code, follows conventions, handles all cases. You notice it created a separate `DiscountCode` entity and `IDiscountRepository` interface, which fits your architecture. You approve.

10:00 AM: You write 3 integration tests for the API endpoint: apply discount to order (happy path), apply invalid code (400 response), and apply to order below minimum (422 response).

10:10 AM: Agent prompt: "Make the new integration tests pass. Create the endpoint, handler, and wire up the discount service."

10:15 AM: Agent completes. All tests pass. You review the controller, handler, and DI registration. Everything follows your conventions (thanks to the AGENTS.md file). You approve and commit.

Total time: 1 hour 15 minutes for a complete discount code feature with full test coverage, clean architecture, and verified behavior.

Without the TDD loop, this would be 3-4 hours of implementation, manual testing, debugging, and review. With naive agent use (no tests, no context), it might be 2-3

hours plus accumulated technical debt from code that “works” but doesn’t fit your architecture.

Making It Stick

The hardest part of the TDD Agent Loop isn’t the technique. It’s the habit change. You have to resist the urge to prompt the agent immediately and instead invest the 15-30 minutes in writing tests first.

Here’s what helps:

Start small. Pick one task tomorrow and try the loop. A bug fix is ideal, because writing a reproducing test is natural. See how it feels. See the results.

Timebox the tests. Give yourself 30 minutes maximum to write tests. If the tests are taking longer than that, the task might be too big. Break it down.

Keep a “tests I wish I’d written” list. When you skip the TDD loop and regret it (because the agent produced garbage you had to rewrite), note it. The list builds the habit by making the cost of skipping visible.

Pair the loop with context files. The TDD Agent Loop is good on its own. Combined with a well-maintained AGENTS.md, it’s transformative. The context file handles the “how” (conventions, patterns, structure), and the tests handle the “what” (behavior, correctness, edge cases). Together, they constrain the agent into a small space where almost any output is acceptable.

The Confidence Equation

Here’s the mental model that ties this chapter to the rest of the book:

Agent confidence = Context quality × Test coverage × Specification clarity

If any of those three is zero, your confidence in the output is zero, regardless of how good the other two are. But when all three are high:

- **Context** tells the agent how to write (conventions, patterns, architecture)
- **Tests** tell the agent what to write (behavior, edge cases, contracts)
- **Specification** tells the agent why (business rules, constraints, intent)

The TDD Agent Loop maximizes the first two. The specification (your test names, your requirement notes, your prompt) handles the third. When all three are dialed in, you can trust agent output at a level that makes the productivity gains real, not illusory.

This is the answer to the METR paradox. The developers in that study had low context, no tests-first workflow, and vague specifications. Of course they were slower. The agents had nothing to work with except the prompt and the code.

Give agents context, tests, and clarity, and the equation inverts. You're not 19% slower. You're 2-3x faster. And you can prove it, because the tests prove it.

That's the TDD Agent Loop. Write the tests. Let the agent handle the rest. Review with confidence.

CHAPTER 14

Spec-First Development

The previous chapter showed you the TDD Agent Loop: write tests, let the agent implement, review with confidence. That workflow is powerful for well-defined features where you can express the behavior in code. But what about the 30 minutes before you write those tests? What about the architectural decisions, the trade-offs, the “how should this thing even work?” questions?

That’s where spec-first development comes in. You write the design document before you write any code or tests. You think through the problem, make the hard decisions, then hand the spec to an agent and say “build this.”

Simon Willison nailed the insight: “Code that started from your own specification is a lot less effort to review.” He’s right, and the reason is simple. When you wrote the spec, you already made the decisions. The agent is executing your decisions, not making its own. Reviewing execution is fast. Reviewing decisions is slow.

Why Specs Matter More Now

Before agents, writing a detailed spec felt like overhead. You’d write the spec, then write the code yourself, and the spec would rot in a Confluence page that nobody ever looked at again. The spec was documentation of intent, disconnected from the implementation.

With agents, the spec IS the implementation instruction. The spec doesn’t rot because it gets consumed immediately. You write it, the agent reads it, code comes out. The spec is the prompt, just a structured, thorough one.

This changes the economics of specification. Writing a detailed spec used to double your work (spec + implementation). Now it replaces your work (spec instead of

implementation). The time you invest in the spec is the time you save on implementation, review, and debugging.

Addy Osmani put it well: “The better your specs, the better the AI’s output.” This isn’t motivational fluff. It’s a mechanical truth. Agents are completion machines. Give them a vague spec, they’ll complete it with assumptions. Give them a precise spec, they’ll complete it with precision.

The Spec Format

A good spec for agentic development isn’t the same as a traditional design document. It’s shorter, more structured, and optimized for consumption by both humans and agents. Here’s the format I use:

```
# Feature: Discount Code System

## Overview
Allow customers to apply discount codes at checkout. Codes can be
percentage-based or fixed-amount, with optional constraints on minimum
order value, usage limits, and expiration dates.

## Requirements
1. Customers can apply a single discount code per order
2. Codes are case-insensitive ("SAVE20" == "save20")
3. Percentage discounts apply to the subtotal before tax
4. Fixed-amount discounts reduce the subtotal (minimum $0, no negative totals)
5. Expired codes return a clear error message with the expiration date
6. Codes at their usage limit return a clear error message
7. Orders below the minimum threshold return a clear error with the minimum

## Constraints
- Discount codes are stored in the existing PostgreSQL database
- Use EF Core for data access (follow existing repository patterns)
- API follows our REST conventions: POST /api/v1/orders/{id}/discount
- Response codes: 200 (applied), 400 (invalid code), 422 (business rule violation)
- No discount stacking in v1 (explicitly reject if discount already applied)

## Architecture Notes
- New DiscountCode entity in the Domain layer
- DiscountService in Domain (pure business logic, no I/O)
- IDiscountCodeRepository interface in Domain, implementation in Infrastructure
- ApplyDiscountHandler in Application layer (orchestrates service + repo)
- Endpoint in API layer using minimal APIs

## Test Scenarios
- Apply valid percentage code: 20% off $100 = $80
- Apply valid fixed code: $15 off $100 = $85
- Apply expired code: returns 422 with expiration date
- Apply code at usage limit: returns 422 with limit info
- Apply code below minimum order: returns 422 with minimum amount
```

```
- Apply code to order with existing discount: returns 422
- Case-insensitive lookup: "save20" finds "SAVE20"
- Fixed discount exceeding subtotal: caps at $0, not negative
- Apply invalid/nonexistent code: returns 400

## Out of Scope
- Discount stacking (v2)
- Automatic discount application (v2)
- Discount analytics/reporting (v2)
```

Notice what this spec includes and what it doesn't.

It includes decisions. The spec says "single discount code per order." That's a product decision you made. The spec says "POST /api/v1/orders/{id}/discount" with specific response codes. That's an API design decision you made. The agent doesn't have to guess any of this.

It includes the architecture. The spec tells the agent exactly where things go: Domain layer, Application layer, Infrastructure layer. This eliminates the most common source of agent-generated code that "works but doesn't fit."

It includes test scenarios. These aren't code yet, but they're specific enough that you (or the agent) can turn them into tests quickly. Each scenario has concrete numbers, not "apply a valid code" but "20% off $100 = $80."

It excludes implementation details. The spec doesn't say how to validate expiration dates or how to check usage limits. Those are implementation decisions the agent can make. You're constraining the what and where, not the how.

It explicitly calls out scope boundaries. The "Out of Scope" section prevents the agent from gold-plating. Without it, agents love to add features you didn't ask for, especially stacking logic that makes the code 3x more complex.

Breaking Specs into Agent-Sized Chunks

A spec like the one above is too big for a single agent pass. Not because the agent can't handle it technically, but because the output would be too large to review effectively. Remember: you're trading implementation time for review time. If the review takes an hour because the agent generated 15 files, you've lost the advantage.

Break the spec into chunks that each produce a reviewable unit of work:

Chunk 1: Domain model and service

```
Implement the DiscountCode entity and DiscountService based on the spec.
Create the types in src/Domain/Discounts/.
Write unit tests in tests/Domain.Tests/Discounts/.
Follow the test scenarios from the spec.
Run tests to verify.
```

Chunk 2: Repository and persistence

```
Implement IDiscountCodeRepository and its EF Core implementation.
Add the DiscountCode entity configuration.
Add a migration for the new table.
Create types in src/Infrastructure/Discounts/.
Follow existing repository patterns (see src/Infrastructure/Orders/).
```

Chunk 3: Application handler

```
Implement ApplyDiscountHandler in src/Application/Discounts/.
Wire up the DiscountService and IDiscountCodeRepository.
Write integration tests in tests/Application.Tests/Discounts/.
Follow existing handler patterns (see src/Application/Orders/).
```

Chunk 4: API endpoint

```
Add the POST /api/v1/orders/{id}/discount endpoint.
Follow minimal API conventions from the spec.
Write integration tests using WebApplicationFactory.
Test all response codes: 200, 400, 422.
Register the handler and service in DI.
```

Each chunk produces 2-5 files. Each chunk is reviewable in 5-10 minutes. Each chunk builds on the previous one. And each chunk can use the TDD Agent Loop from the previous chapter: write the tests (or let the agent write them from the spec's test scenarios), then implement.

The Spec, Agent, Review, Iterate Cycle

Here's the full workflow in practice:

Phase 1: Spec (you, 20-40 minutes)

Write the spec document. This is your thinking time. You're making architectural decisions, defining constraints, listing test scenarios. This is the highest-value work you do as a developer.

Save the spec as a markdown file in your repo. I keep them in `docs/specs/` alongside the code. Some teams use `docs/adr/` for architectural decisions. The location doesn't matter. What matters is that it's in the repo, versioned, and accessible to agents.

Phase 2: Chunk and delegate (you, 5 minutes)

Break the spec into agent-sized chunks as described above. You can do this in your head or write it out. For complex features, I write the chunks as a checklist in the spec itself.

Phase 3: Agent implementation (agent, 5-15 minutes per chunk)

Hand each chunk to the agent with the spec as context:

```
Read the spec in docs/specs/discount-codes.md.

Implement Chunk 1: Domain model and service.
Create DiscountCode entity and DiscountService in src/Domain/Discounts/.
Write tests based on the test scenarios in the spec.
Run tests to verify.
```

The agent reads the spec, understands the full picture, and implements the chunk within that context. This is critical: the agent sees the whole spec even when implementing a piece. It knows what's coming next, so it makes design choices that support the full feature.

Phase 4: Review (you, 5-10 minutes per chunk)

Review the output. With a good spec, your review focuses on: - Does the code match the spec's decisions? - Is the code clean and idiomatic? - Did the agent make reasonable implementation choices? - Are the tests comprehensive?

If something's wrong, either fix it directly (if it's small) or prompt the agent with corrections. This is iteration, not failure. Two passes is normal. Three is fine. Ten means your spec was unclear.

Phase 5: Commit and continue

Commit the chunk. Move to the next one. Each chunk builds on committed, reviewed, working code.

ADR Generation

Architecture Decision Records (ADRs) are a natural byproduct of spec-first development. You're already making architectural decisions in the spec. Capturing them as ADRs is a small extra step with outsized long-term value.

An ADR records why you made a decision, not just what you decided. Six months from now, when someone asks “why did we use a separate DiscountService instead of putting the logic in the handler?”, the ADR answers that question.

Here’s the thing: agents are excellent at generating ADRs from specs. After you’ve implemented a feature, prompt the agent:

```
Based on the spec in docs/specs/discount-codes.md and the
implementation in src/Domain/Discounts/ and src/Application/Discounts/,
generate an ADR for the key architectural decisions.

Use the format in docs/adr/template.md.
Focus on decisions that future developers would question.
```

The agent produces something like:

```
# ADR-017: Discount Code Architecture

## Status
Accepted

## Context
We need to support discount codes at checkout. The system must handle
percentage and fixed-amount discounts with expiration, usage limits,
and minimum order constraints.

## Decision
- Pure domain service (DiscountService) for business logic, no I/O
- Separate repository for discount code persistence
- Application handler orchestrates service and repository
- Single discount per order (no stacking in v1)
- Case-insensitive code lookup at the repository level

## Consequences
- Business logic is testable without infrastructure dependencies
- Adding discount stacking in v2 requires changes to DiscountService
  and the handler, but not the repository or API contract
- Case-insensitive lookup in the repository means the database
  handles collation, not the application code
```

You review and commit this alongside the code. Future you (and future team members) will thank present you.

The combination of spec + ADR creates a documentation trail that's actually useful. The spec says what you planned. The ADR says what you decided and why. The code says what you built. The tests say what it does. That's complete traceability from intent to implementation.

A Real Example: Building a Complete API Feature

Let me walk through a full spec-first workflow for a realistic feature. We're adding a notification preferences system to an existing .NET API.

The Spec (25 minutes)

```
# Feature: Notification Preferences

## Overview
Users can configure how they receive notifications: email, SMS, push,
or in-app. Each notification type (order updates, promotions,
security alerts) can be independently toggled per channel.

## Requirements
1. Users have a preferences record created on account registration
2. Default: all channels enabled for security alerts,
   email-only for everything else
3. Users can update preferences via PATCH endpoint
4. Security alerts cannot be fully disabled (at least one channel required)
5. SMS requires a verified phone number (return 422 if unverified)
6. Preference changes are eventually consistent (async event published)

## Constraints
- PATCH /api/v1/users/{id}/notification-preferences
- Request body: partial update (only changed fields)
- EF Core with existing User entity relationship
- Publish NotificationPreferencesChanged event via MediatR
- No new external dependencies

## Architecture Notes
- NotificationPreferences entity in Domain, owned by User
- NotificationPreferencesService in Domain (validation logic)
- UpdateNotificationPreferencesHandler in Application
- Minimal API endpoint in API layer

## Test Scenarios
- Get default preferences for new user: security=all, others=email-only
- Update email preference for promotions: toggles correctly
- Disable all channels for security alerts: returns 422
- Enable SMS without verified phone: returns 422
- Partial update: only specified fields change, others unchanged
```

```
- Publish event on successful update: event contains user ID and changes
- Update nonexistent user: returns 404
```

Chunk 1: Domain Model (10 minutes with agent)

I write the unit tests for the domain service based on the test scenarios, then hand them to the agent:

```
Make all tests in NotificationPreferencesServiceTests pass.
Create types in src/Domain/Notifications/.
The NotificationPreferences entity should be an owned entity of User.
Run tests to verify.
```

Agent produces: `NotificationChannel` enum (flags), `NotificationType` enum, `NotificationPreferences` entity, `NotificationPreferencesService` with validation logic. All tests pass. Review takes 5 minutes. Clean code, good use of flags enum for channel combinations.

Chunk 2: Persistence (8 minutes with agent)

```
Add EF Core configuration for NotificationPreferences as an owned
entity of User. Add a migration. Follow the existing owned entity
pattern in src/Infrastructure/EntityConfigurations/UserConfiguration.cs.
```

Agent adds the configuration, generates the migration. Review takes 3 minutes (mostly checking the migration SQL).

Chunk 3: Handler and Event (12 minutes with agent)

I write integration tests for the handler, including verification that the MediatR event is published. Agent implements the handler. First pass misses the partial update logic (overwrites all fields instead of merging). Test catches it, agent fixes it on second iteration.

Chunk 4: API Endpoint (8 minutes with agent)

Agent builds the PATCH endpoint, maps the request to the handler, returns appropriate status codes. Integration tests via `WebApplicationFactory` cover all response codes.

Total time: 25 minutes speccing + 38 minutes implementation + ~20 minutes review = ~83 minutes for a complete feature with full test coverage, clean architecture, ADR, and documentation.

The spec drove the whole thing. Every decision in the code traces back to a line in the spec. Every test scenario maps to a requirement. The agent didn't guess, didn't improvise, didn't gold-plate. It built exactly what the spec described.

When Spec-First Doesn't Fit

Not every task deserves a full spec. Bug fixes, small refactors, and quick scripts don't need a design document. The TDD Agent Loop from Chapter 13 handles those beautifully.

Spec-first shines for: - New features with multiple components - Anything touching the API contract - Features that span multiple layers of the architecture - Work that multiple developers (or agents) will touch - Anything you'll need to explain to someone else later

Use your judgment. A five-line spec is still a spec. A mental model you hold in your head while writing tests is, loosely, a spec. The format matters less than the act of thinking through the problem before writing code.

The real insight is this: in an agentic workflow, the spec is your highest-leverage artifact. It's the one thing that scales your judgment across every agent interaction. Write it once, use it across every chunk, reference it in every review. The 25 minutes you spend writing a spec saves hours of debugging agent output that went in the wrong direction.

Write the spec. Break it into chunks. Let agents build it. Review with the spec as your guide. That's spec-first development.

CHAPTER 15

Error-Message-Driven Debugging

Paste the error. Get the fix. This is the simplest agentic workflow, and it's probably the one you'll use most often.

Every developer has done this manually for years: copy the error message, search Stack Overflow, read three answers, try the most upvoted one, realize it's for a different version, try the second answer, realize you need more context, go back to the code, eventually figure it out. The whole process takes 10-45 minutes for a typical error.

With an agent, it takes 30 seconds to 5 minutes. You paste the error with context, the agent diagnoses the root cause, suggests a fix, and (if you're using an agentic IDE or CLI) applies it and runs the tests. Done.

But there's a difference between doing this well and doing this poorly. The difference is context. A bare error message gets you a generic fix that might work. A well-structured error context gets you a precise fix that almost certainly works.

How to Structure Error Context

The quality of your debugging prompt directly determines the quality of the fix. Here's what to include, in order of importance:

1. The Full Error Message and Stack Trace

Not a summary. Not "I got a null reference exception." The full thing:

```
System.InvalidOperationException: Unable to resolve service for type
'IDiscountCodeRepository' while attempting to activate
'Application.Discounts.ApplyDiscountHandler'.

   at Microsoft.Extensions.DependencyInjection.ActivatorUtilities
      .ThrowHelperUnableToResolveService(Type type, Type requiredBy)
   at Microsoft.Extensions.DependencyInjection.ActivatorUtilities
      .ConstructorMatcher.CreateInstance(IServiceProvider provider)
   at Microsoft.Extensions.DependencyInjection.ServiceLookup
      .CallSiteRuntimeResolver.VisitConstructor(...)
   at Microsoft.Extensions.DependencyInjection.ServiceLookup
      .CallSiteRuntimeResolver.VisitScopeCache(...)
```

The stack trace tells the agent exactly where the failure occurs and what the runtime was trying to do. Without it, the agent guesses. With it, the agent knows.

2. The Relevant Code

Don't paste your entire codebase. Paste the file(s) that the error points to. If the error is in `ApplyDiscountHandler`, paste that class. If the error is about DI registration, paste `Program.cs` or wherever you register services.

```
// Program.cs (relevant section)
builder.Services.AddScoped<ApplyDiscountHandler>();
builder.Services.AddScoped<DiscountService>();
// Note: IDiscountCodeRepository registration is missing
```

If you're using an agentic IDE (Copilot, Cursor, Windsurf) or a CLI agent (Claude Code, Codex CLI), the agent can read the files itself. Just point it to the right location: "The error is in the DI setup. Check Program.cs and the ApplyDiscountHandler constructor."

3. What You Already Tried

This prevents the agent from suggesting the first thing you already ruled out:

```
I already checked:
- The repository interface is defined in Domain/Discounts/
- The implementation exists in Infrastructure/Discounts/
- Other repositories (IOrderRepository) work fine
- The Infrastructure project is referenced by the API project
```

This context is gold. It narrows the search space dramatically. Without it, the agent's first suggestion is often "did you forget to create the implementation?" which wastes a round trip.

4. What Changed Recently

Errors don't come from nowhere. Something changed:

```
This started after I added the discount code feature.
The handler and service are new (committed in the last chunk).
The repository implementation was generated by the agent
in the previous step.
```

Knowing what changed helps the agent focus on the delta, not the entire system.

The Ideal Debugging Prompt

Combining all of that, here's what a well-structured debugging prompt looks like:

```
I'm getting this error when hitting POST /api/v1/orders/{id}/discount:

[paste full error and stack trace]

The ApplyDiscountHandler is in src/Application/Discounts/.
The IDiscountCodeRepository is defined in src/Domain/Discounts/.
The implementation is in src/Infrastructure/Discounts/.

I already verified the implementation exists and other repositories
work fine. This broke after adding the discount feature.

Diagnose and fix.
```

An agent with codebase access reads the relevant files, identifies that the DI registration for `IDiscountCodeRepository` is missing from `Program.cs`, adds it, and runs the tests to verify. Total time: under a minute.

Log Analysis and Root Cause Identification

Error messages from your IDE are the easy case. The harder case is production logs, where the error might be buried in noise, or the root cause is three services upstream from the symptom.

Agents are surprisingly good at log analysis. They can process large volumes of structured logs and identify patterns that humans miss, especially when the logs are in a standard format (Serilog's JSON output, Application Insights traces, structured logging with correlation IDs).

Here's an effective approach:

```
Here are the last 50 log entries from our order service around
the time of the failure (14:23:05 UTC):

[paste structured logs]

The user reported that their discount code wasn't applied,
but they got a 200 response. Help me find the root cause.
```

The agent scans the logs, notices that the `DiscountService` returned a valid result but the `OrderService` received a stale order total (a race condition between reading the order and applying the discount), and identifies the missing transaction scope.

This works because log analysis is pattern matching over structured text, which is exactly what language models are good at. The agent isn't guessing; it's reading the evidence and following the logical chain.

When to Give More Context

For complex production issues, structure your prompt like a bug report:

```
## Symptom
Intermittent 500 errors on POST /api/v1/orders/{id}/discount,
approximately 2% of requests. Started after deploy on Feb 15.

## Evidence
- Error logs show DbUpdateConcurrencyException
- Only happens under load (>50 concurrent requests)
- The discount_codes table has a row version column
- Same code works fine in integration tests (single-threaded)

## Hypothesis
Possible optimistic concurrency conflict when multiple users
apply the same discount code simultaneously, decrementing
the usage count.

## Relevant Code
[paste DiscountCodeRepository.ApplyAsync method]
[paste EF Core entity configuration for DiscountCode]
```

The agent confirms your hypothesis (or offers an alternative), suggests a fix (retry logic with exponential backoff, or a pessimistic lock for the usage count update), and explains the trade-offs. This is debugging at machine speed with human judgment.

Debugging Unfamiliar Code

This is where error-driven debugging really shines. You’ve joined a new team. You’re staring at a codebase you’ve never seen before. Something breaks.

In the old world, you’d spend hours reading code, understanding the architecture, building a mental model, then finally diagnosing the issue. With an agent that has codebase access, you skip straight to diagnosis:

```
I'm new to this codebase. The checkout flow is returning a 500 error.
Here's the stack trace:

[paste stack trace]

Read the relevant files, explain what's happening, and suggest a fix.
Explain the surrounding architecture as you go so I can learn the
codebase while we fix this.
```

The agent reads the files the stack trace points to, explains the checkout flow (controller calls handler, handler calls payment service, payment service calls Stripe), identifies the null reference (the Stripe customer ID isn't being passed because the user profile query is missing an Include), fixes it, and gives you an architecture overview in the process.

You fixed the bug AND onboarded to the codebase simultaneously. This is one of those workflow changes that feels like cheating the first time you do it.

When Agents Diagnose Well vs. When They Hallucinate Fixes

Agents are excellent debuggers for certain categories of errors and unreliable for others. Knowing the difference saves you from applying hallucinated fixes that create new bugs.

Agents diagnose well when:

The error message is specific. DI resolution failures, type mismatches, missing configuration keys, null reference exceptions with clear stack traces. These have unambiguous root causes that the agent can trace.

The fix is mechanical. Missing a using statement, wrong parameter order, incorrect configuration value, missing DI registration. These fixes are deterministic. There's one right answer.

The pattern is common. EF Core migration conflicts, ASP.NET Core middleware ordering, serialization issues, CORS configuration. Agents have seen millions of examples of these errors and their fixes.

The context is sufficient. When the agent can see the relevant code, the error, and the project structure, its diagnostic accuracy is high.

Agents hallucinate fixes when:

The error is a symptom, not the cause. A `TimeoutException` could be caused by a slow database query, a deadlock, a network issue, or a misconfigured connection pool. The agent might confidently "fix" the timeout by increasing the timeout value, which masks the real problem.

The fix requires understanding business logic. An agent can tell you that a calculation returns the wrong number. It can't tell you what the right number should be without understanding the business rules. If the discount is applying 15% instead of 20%, the agent might fix the percentage without knowing which is correct.

The error involves concurrency or distributed systems. Race conditions, eventual consistency issues, and distributed transaction failures are hard for agents because the root cause isn't visible in any single file or stack trace. The agent needs to reason about timing and state across multiple components, which it does poorly.

The codebase has unconventional patterns. If your team invented a custom middleware pipeline, a bespoke ORM wrapper, or a novel caching strategy, the agent's training data won't help. It'll suggest fixes based on the standard patterns, which might be wrong.

The Practical Rule

Trust agent fixes for compilation errors, configuration errors, and well-defined runtime errors. Verify agent fixes for logic errors, performance issues, and anything involving

state across multiple components. Reject agent fixes for concurrency bugs unless you can reproduce them with a test.

The “Reproduce First” Principle

The most reliable error-driven debugging workflow combines this chapter with the TDD Agent Loop from Chapter 13: reproduce the bug as a failing test first, then fix it.

```
This test should fail (it reproduces a reported bug where
discount codes are case-sensitive despite our spec requiring
case-insensitive matching):

[Fact]
public async Task ApplyDiscount_LowercaseCode_MatchesUppercaseEntry()
{
    // Arrange
    await SeedDiscountCode("SAVE20", percentage: 20);
    var orderId = await CreateTestOrder(100.00m);

    // Act
    var response = await Client.PostAsJsonAsync(
        $"/api/v1/orders/{orderId}/discount",
        new { code = "save20" });

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.OK);
    var order = await GetOrder(orderId);
    order.DiscountApplied.Should().Be(20.00m);
}

Run this test to confirm it fails, then fix the code to make it pass
without breaking other tests.
```

This approach has three advantages:

1. **The bug is confirmed.** The test fails, proving the bug exists. No ambiguity.
2. **The fix is verified.** When the test passes, the bug is fixed. No manual testing needed.
3. **Regression is prevented.** The test stays in the suite forever. The bug can never come back without the test catching it.

The agent runs the test, sees the failure (the repository query uses a case-sensitive comparison), fixes the query to use `StringComparison.OrdinalIgnoreCase` (or, better, pushes the case-insensitivity to the database with a `COLLATE` clause in the EF Core configuration), runs the tests again, and confirms all pass.

A Debugging Workflow in Practice

Here's what a realistic debugging session looks like:

2:15 PM: Slack notification: "Discount codes aren't working in production."

2:16 PM: You check the logs. `InvalidOperationException: Sequence contains no elements` in `DiscountCodeRepository.GetByCodeAsync`.

2:17 PM: You paste the error, stack trace, and the repository method to the agent:

```
Production error. The SingleAsync call throws when no discount code
matches. This should return a 400 (invalid code), not a 500.

[stack trace]
[repository method]

This needs two fixes:
1. Immediate: handle the "not found" case gracefully
2. Test: add a failing test that reproduces this, then fix
```

2:19 PM: Agent writes a test that calls the endpoint with a nonexistent code, confirms it returns 500 (reproducing the bug), changes `SingleAsync` to `SingleOrDefaultAsync` with a null check that returns a proper `NotFound` result, runs the tests, and confirms the new test passes along with all existing tests.

2:22 PM: You review the fix. Clean, correct, handles the edge case. You also notice the agent added a test for empty string codes, which you hadn't thought of. Good catch. Commit and deploy.

Total time: 7 minutes from notification to fix, with a regression test included.

That's error-driven debugging with agents. Paste the error, structure the context, let the agent diagnose, reproduce with a test, fix with confidence. The simplest workflow in the book, and the one you'll reach for daily.

CHAPTER 16

Codebase Archaeology

Every developer has inherited a codebase they didn't write. You stare at the folder structure, open a few files, try to trace a request from the controller to the database, get lost somewhere in a maze of abstractions, and wonder if anyone on the original team is still at the company.

This is codebase archaeology: understanding existing code well enough to change it safely. Before agents, it was slow, manual, and error-prone. You'd read code for hours, scribble diagrams on whiteboards, and hope your mental model was accurate.

Agents change this completely. Instead of reading code, you interview your codebase.

"How Does Our Auth System Work?"

This single prompt, given to an agent with codebase access, produces more useful output than an hour of code reading:

```
Explain how authentication and authorization work in this codebase.
Start from an incoming HTTP request and trace through every layer.
Include middleware, token validation, claims extraction, and how
authorization policies are enforced on endpoints.
Reference specific files and line numbers.
```

The agent reads your `Program.cs`, finds the authentication middleware configuration, traces through the JWT bearer setup, identifies the custom claims transformer, finds the authorization policies, and produces a clear explanation with file references. In 2-3 minutes, you have a mental model that would have taken an hour to build manually.

This works because agents are good at two things that make code reading hard for humans: holding a lot of context simultaneously, and following reference chains across files without losing track.

When you read code manually, you open a file, see a method call, navigate to the definition, see another method call, navigate again, and by the time you're four files deep, you've forgotten why you started. Agents don't have this problem. They can hold the entire call chain in context and produce a coherent summary.

Onboarding to Unfamiliar Codebases

The most valuable application of codebase archaeology is onboarding. Whether you're joining a new team, picking up a legacy project, or reviewing an open-source library, agents compress the onboarding timeline dramatically.

Here's an onboarding workflow I use when joining a new codebase:

Step 1: The Overview

```
Give me a high-level overview of this codebase.
What does it do? What's the architecture?
What are the main projects/modules and their responsibilities?
What frameworks and libraries does it use?
```

This gives you the 30,000-foot view. The agent reads the solution file, project references, key configuration files, and top-level folder structure. You get a summary like:

"This is an ASP.NET Core 9 API with a clean architecture structure: Domain (business logic, no dependencies), Application (use cases, MediatR handlers), Infrastructure (EF Core, external services), and API (minimal API endpoints). It uses PostgreSQL via EF Core, MassTransit for messaging, and Redis for caching. There are 47 endpoints organized by feature."

In two minutes, you know more about this codebase than most README files would tell you.

Step 2: Key Flows

Pick the 3-4 most important user flows and trace them:

```
Trace the complete flow for creating an order:
1. HTTP request hits the endpoint
2. Request validation
3. Business logic execution
4. Database persistence
5. Any events/messages published
6. Response returned

Show me the specific files and methods involved at each step.
```

This builds your mental model of how the codebase actually works, not how it's supposed to work (those are often different). The agent follows the real code path, including any middleware, filters, or interceptors that affect the flow.

Step 3: The Gotchas

```
What are the non-obvious aspects of this codebase?
Look for:
- Custom conventions that differ from standard ASP.NET Core
- Hidden side effects (event handlers, background jobs triggered by actions)
- Configuration that changes behavior based on environment
- Any global filters, middleware, or DI decorators that affect all requests
- Known workarounds or technical debt (check for TODO/HACK/FIXME comments)
```

This is where agents really earn their keep. They'll find the custom exception filter that swallows certain errors, the background job that runs on every order creation (but only in production), and the environment variable that switches between two completely different caching strategies. These are the things that bite you on day three.

Step 4: The Test Suite

```
Analyze the test suite:
- What's the testing strategy? Unit, integration, end-to-end?
- Which areas have good coverage? Which are thin?
- What test frameworks and libraries are used?
- Are there any test helpers or base classes I should know about?
- How do I run the tests locally?
```

Understanding the test suite tells you how the previous team thought about quality and where the safety nets are (and aren't).

Generating Internal Documentation

Most codebases are under-documented. Not because developers are lazy, but because documentation rots. You write a README when you start the project, and by month six, it's out of date. You write API docs once, and after twenty endpoint changes, they describe a system that no longer exists.

Agents can generate documentation from the code itself, which means the documentation reflects what the code actually does, not what someone remembered to write down.

README Generation

```
Generate a comprehensive README.md for this project. Include:
- What the project does (read the code, don't guess)
- Prerequisites and setup instructions (check Docker files, scripts, config)
- How to run locally (check launchSettings.json, docker-compose, scripts)
- How to run tests
- Project structure with brief description of each project/folder
- Key configuration settings and environment variables
- API overview (list main endpoint groups)
```

The agent produces a README based on reality. It checks `docker-compose.yml` for service dependencies, reads `launchSettings.json` for ports and URLs, examines the test project for the test runner, and lists the actual endpoints from the code.

API Documentation

For internal APIs that don't justify a full OpenAPI setup:

```
Generate API documentation for all endpoints in the API project.
For each endpoint, include:
- HTTP method and path
- Request body schema (if any)
- Response codes and their meanings
- Response body schema
- Authentication requirements
- Any notable behavior (caching, rate limiting, side effects)

Format as markdown. Group by feature area.
```

The agent reads every endpoint definition, traces the request/response types, checks for `[Authorize]` attributes or authorization policies, and produces complete API docs. For a 50-endpoint API, this takes the agent a few minutes and saves you days of manual documentation.

Architecture Documentation

```
Generate an architecture document that describes:
- The overall architecture pattern and layer responsibilities
- How dependency injection is configured
- The data access strategy (EF Core configuration, migrations, conventions)
- The messaging/event system (publishers, consumers, message types)
- Cross-cutting concerns (logging, error handling, caching, auth)
- External service integrations

Include a component diagram in Mermaid format.
```

The Mermaid diagram is particularly useful. Agents generate reasonable architecture diagrams from code that you can include in your docs:

```
graph TD
    A[API Layer] ⟶ B[Application Layer]
    B ⟶ C[Domain Layer]
    B ⟶ D[Infrastructure Layer]
    D ⟶ C
    D ⟶ E[(PostgreSQL)]
    D ⟶ F[(Redis Cache)]
    D ⟶ G[Stripe API]
    B ⟶ H[MassTransit]
    H ⟶ I[(RabbitMQ)]
```

Is it perfect? No. Is it better than no diagram at all? Absolutely. And you can refine it in 5 minutes.

Architecture Decision Records from Existing Code

This is one of my favorite uses of codebase archaeology. Most teams don't write ADRs. But the architectural decisions are embedded in the code. Agents can extract them.

```
Analyze this codebase and identify the major architectural decisions
that were made. For each one, generate an ADR that includes:
- What the decision was
- What the likely alternatives were
- Why this choice was probably made (infer from the code and patterns)
- What the consequences are

Focus on decisions that a new developer would question, like:
- Why clean architecture instead of vertical slices?
- Why MassTransit instead of raw RabbitMQ?
- Why repository pattern when EF Core already abstracts the database?
- Why MediatR for in-process messaging?
```

The agent produces retrospective ADRs that capture tribal knowledge. These aren't as good as ADRs written at decision time (you're inferring intent, not documenting it), but they're vastly better than nothing. They give new team members a starting point for understanding why the codebase looks the way it does.

Mark these as "retrospective" ADRs so readers know they're inferred:

```
# ADR-001 (Retrospective): Clean Architecture Pattern

## Status
Accepted (inferred from codebase analysis, February 2026)

## Context
The codebase uses a clean architecture pattern with four layers...
```

From “Reading Code” to “Interviewing Your Codebase”

The mental shift here is significant. You’re not reading code line by line. You’re asking questions and getting answers.

Some questions that work well as codebase interviews:

Understanding behavior: - “What happens when a user’s subscription expires?” - “How does the retry logic work for failed payment charges?” - “What validations run before an order is submitted?”

Understanding design: - “Why does the OrderService depend on IEventPublisher?” - “What’s the relationship between User, Account, and Profile?” - “How are database transactions managed across multiple repository calls?”

Understanding risk: - “Which endpoints have no authorization checks?” - “Where does the codebase handle sensitive data (PII, payment info)?” - “What happens if the Redis cache is unavailable?”

Understanding dependencies: - “Which services would break if the payment provider’s API changes?” - “What external services does the order creation flow depend on?” - “Which NuGet packages are outdated by more than one major version?”

Each of these questions would take 15-60 minutes to answer manually. An agent with codebase access answers them in 1-3 minutes. Over the course of onboarding to a new codebase, this compresses weeks of exploration into days.

Keeping Generated Docs Current

Generated documentation has the same rot problem as hand-written documentation: the code changes, and the docs don't.

The solution is to regenerate, not maintain. Treat generated docs as artifacts, not source documents. Here's how:

Option 1: Regeneration Prompts in Your Repo

Keep the prompts that generated your docs alongside the docs:

```
docs/
  architecture.md
  api-reference.md
  prompts/
    generate-architecture.md
    generate-api-reference.md
```

When the code changes significantly, rerun the prompts. The agent generates fresh docs from the current code. You review and commit.

Option 2: CI-Triggered Doc Generation

For teams, add a documentation step to your CI pipeline that runs on significant changes:

```yaml
# .github/workflows/docs.yml
on:
  push:
    branches: [main]
    paths:
      - 'src/**'
      - '!src/**/*.Tests/**'

jobs:
  update-docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Generate docs
        run: |
          # Use your agent CLI of choice to regenerate docs
          # from the prompts in docs/prompts/
```

This isn't fully automated (you still review the PR the agent creates), but it ensures documentation stays within one merge of the current code.

Option 3: Review-Time Generation

The lightest approach: generate docs only when you need them. Before a code review, before onboarding a new team member, before a planning session. The docs are ephemeral, generated on demand, always current because they're generated from the current code.

This sounds wasteful (regenerating docs every time) but agent-generated docs are cheap. A full API reference for a 50-endpoint API takes the agent 2-3 minutes. That's cheaper than maintaining a stale document that misleads people.

The Compounding Value

Codebase archaeology with agents has a compounding effect. Each answer builds your mental model. After a few sessions of interviewing your codebase, you understand it well enough to work effectively. The agent didn't just answer your questions; it transferred knowledge from the code to you.

And the documentation you generate along the way helps the next person. They read the architecture doc, the API reference, the retrospective ADRs, and they start from a much higher baseline. Then they use agents to fill in the gaps specific to their needs.

This is how institutional knowledge stops being locked in people's heads and starts being accessible in the codebase itself. Not as stale wiki pages, but as living documents that can be regenerated from the code at any time.

The codebase is the source of truth. Agents are the interpreters. Documentation is the output. Keep the interpreters handy, and the output is always fresh.

CHAPTER 17

Migration Generation

Nobody likes migrations. Framework upgrades, database schema changes, API versioning, language version bumps. They're tedious, error-prone, and the kind of work where a small mistake cascades into hours of debugging. They're also exactly the kind of work that agents handle well.

Migrations are pattern-heavy. Upgrade .NET 8 to .NET 10, and 90% of the changes are mechanical: update the target framework moniker, update NuGet packages, fix the handful of breaking API changes. The other 10% requires judgment. Agents crush the 90% and flag the 10% for your review.

This chapter covers the major categories of migration work and how to structure each for agentic development.

Framework Version Upgrades

Let's start with the most common migration: upgrading your .NET version. We'll use .NET 8 to .NET 10 as the example, but the patterns apply to any framework upgrade.

What Agents Handle Well

Target framework updates. Changing `<TargetFramework>net8.0</TargetFramework>` to `net10.0` across every project file. Mechanical, repetitive, zero judgment required.

NuGet package updates. Bumping `Microsoft.AspNetCore.*`, `Microsoft.EntityFrameworkCore.*`, and other first-party packages to their .NET 10 versions. The agent reads the current versions, checks compatibility, and updates them.

Breaking API changes. Each .NET release has a documented list of breaking changes. Agents are good at identifying which ones affect your code and applying the fixes. For example, if a method signature changed or an API was obsoleted, the agent finds every usage and updates it.

Build error resolution. After the initial upgrade, `dotnet build` produces errors. The agent reads each error, applies the fix, rebuilds, and iterates until the build succeeds. This is the TDD Agent Loop applied to compilation instead of tests.

What Agents Handle Poorly

Behavioral changes. Some upgrades change how things work without changing the API. A serialization library might handle null values differently. A middleware might process requests in a different order. These don't produce build errors, and agents can't detect them without tests.

Performance regressions. The upgrade might introduce performance changes (positive or negative) that only show up under load. Agents can't benchmark for you.

Configuration changes. New versions sometimes change default configuration values or add required configuration. The agent might miss these if they're not documented in the breaking changes list.

The Upgrade Workflow

Here's how I structure a framework upgrade with agents:

Step 1: Preparation (you, 15 minutes)

Read the official migration guide. Yes, actually read it. The agent will handle the mechanical work, but you need to know about the behavioral changes and new features that might affect your architecture.

Create a migration spec (Chapter 14's format works well):

```
# Migration: .NET 8 → .NET 10

## Scope
- All 12 projects in the solution
- Target framework, NuGet packages, breaking API changes
- Do NOT adopt new features yet (separate PR)

## Known Breaking Changes Affecting Us
- JsonSerializerOptions.PropertyNameCaseInsensitive default changed
- IHostEnvironment.ContentRootPath behavior in test host
- Obsolete middleware removed (we use UseExceptionHandler override)

## Out of Scope
- Adopting new .NET 10 features (HybridCache, new minimal API features)
- Performance optimization
- Dependency injection changes
```

Step 2: Mechanical upgrade (agent, 10-20 minutes)

```
Upgrade this solution from .NET 8 to .NET 10.

1. Update TargetFramework in all .csproj files
2. Update all Microsoft.* NuGet packages to .NET 10 compatible versions
3. Fix any build errors from breaking API changes
4. Run dotnet build and iterate until it compiles cleanly
5. Run dotnet test and fix any test failures

Read the migration spec in docs/specs/net10-migration.md for known
breaking changes and scope boundaries.
```

The agent works through the solution project by project. It updates the framework, bumps packages, resolves build errors, and runs the tests. Most of this is mechanical. The agent handles it in a fraction of the time you would.

Step 3: Review (you, 20-30 minutes)

Review the changes. Focus on: - Did the agent handle the known breaking changes correctly? - Are there any changes that look suspicious (logic changes, not just API updates)? - Do all tests pass? Are there any tests that pass but might be testing the wrong thing now?

Step 4: Manual verification (you)

Test the behavioral changes from your spec manually or with targeted integration tests. The agent can help you write those tests:

```
The .NET 10 migration changed JsonSerializerOptions defaults.
Write integration tests that verify our API responses still
serialize property names the same way as before the upgrade.
```

Database Schema Evolution

EF Core migrations with agents are one of the smoothest agentic workflows. The combination of a declarative model (your entity classes), a diff engine (EF Core's migration scaffolder), and a test framework (integration tests against a real database) creates a tight feedback loop.

Adding a New Entity

```
Add a DiscountCode entity to the domain model with these properties:
- Id (Guid, primary key)
- Code (string, max 50, unique index)
- Type (enum: Percentage, FixedAmount)
- Value (decimal)
- MinimumOrderAmount (decimal, nullable)
- MaxUsageCount (int, nullable)
- CurrentUsageCount (int, default 0)
- ExpiresAt (DateTimeOffset, nullable)
- CreatedAt (DateTimeOffset)

Create the entity in src/Domain/Discounts/.
Add EF Core configuration in src/Infrastructure/EntityConfigurations/.
Generate a migration.
Follow existing patterns (see OrderConfiguration.cs for reference).
```

The agent creates the entity, writes the configuration (column types, indexes, constraints), and runs `dotnet ef migrations add AddDiscountCode`. The generated migration is deterministic based on the model, so reviewing it is straightforward: check that the SQL matches your intent.

Modifying Existing Entities

Schema changes to existing entities are trickier because they involve data migration. Agents handle the schema part well but need explicit guidance for data:

```
Add a "Tier" property to the Customer entity:
- Enum: Standard, Silver, Gold, Platinum
- Default: Standard
- All existing customers should be Standard

Generate the migration. The migration should:
1. Add the column with a default value of 0 (Standard)
2. NOT use a data migration script (the default handles existing rows)
```

Without that explicit guidance, agents sometimes generate data migration scripts that are unnecessary or, worse, incorrect. Be specific about data handling.

Complex Migrations

For migrations that involve data transformation (splitting a table, merging columns, changing a relationship), write the migration spec carefully:

```
# Migration: Split Address from Customer

## Current State
Customer table has address columns inline:
Street, City, State, ZipCode, Country

## Target State
Separate Address table with one-to-many relationship to Customer
(customers can have multiple addresses, one marked as default)

## Data Migration
1. Create Address table
2. For each Customer, create an Address record from their inline columns
3. Mark the created Address as the default
4. Remove inline address columns from Customer table

## Safety
- Migration must be reversible (down migration restores inline columns)
- Run in a transaction
- Test with production-like data volume (500K customers)
```

Hand this to the agent:

```
Generate an EF Core migration based on the spec in
docs/specs/split-address-migration.md.

Create the new Address entity and configuration first,
then generate the migration. Include the data migration
in the Up method using raw SQL for performance.
```

The agent generates the migration with the data transformation. Review the SQL carefully. Run it against a copy of production data before deploying.

API Versioning and Contract Migration

API versioning is one of those tasks that's conceptually simple but mechanically tedious. You're duplicating endpoints, adjusting request/response models, maintaining backward compatibility, and updating documentation. Agents handle the mechanical parts well.

Adding a New API Version

```
Create v2 of the Orders API endpoints.

Changes from v1:
- CreateOrder request adds an optional "discountCode" field
- OrderResponse includes "discountAmount" and "finalTotal" fields
- GET /orders supports filtering by status (query parameter)

Requirements:
- v1 endpoints continue to work unchanged
- v2 endpoints live in a new OrdersV2 folder
- Shared logic stays in the Application layer (handlers are version-aware)
- Both versions share the same domain model

Follow the existing versioning pattern in the Users API (v1 and v2 exist).
```

The agent creates the v2 endpoint definitions, the new request/response models, adapts the handlers, and wires everything up. The key here is pointing the agent to an existing versioning pattern. Without that reference, agents make inconsistent choices about how to structure versioned endpoints.

Contract Migration

When you need to migrate consumers from v1 to v2, agents help generate the migration guide:

```
Generate a migration guide for consumers of our Orders API,
covering the upgrade from v1 to v2.

For each changed endpoint, show:
- The v1 request/response
- The v2 request/response
- What changed and why
- Code examples for updating client code (C# HttpClient)

Also generate a deprecation timeline recommendation.
```

This produces documentation that your API consumers actually need, generated directly from the code differences between v1 and v2.

Large-Scale Refactoring

Large refactors are where agents provide the most dramatic time savings, and where they're most dangerous if uncontrolled. The key is breaking the refactor into safe, reviewable steps.

The Safe Refactoring Pattern

Step 1: Write characterization tests.

Before changing anything, capture the current behavior:

```
Write integration tests that capture the current behavior of
the OrderService. Test every public method with representative
inputs and outputs. These tests should pass against the current
implementation. They're our safety net for the refactoring.
```

Step 2: Break the refactor into atomic steps.

Each step should be independently committable and deployable:

```
# Refactoring Plan: Extract Payment Processing

## Step 1: Extract interface
Create IPaymentProcessor interface from existing PaymentService methods.
No behavior change. All existing code continues to work.

## Step 2: Implement adapter
Create StripePaymentProcessor implementing IPaymentProcessor.
Move Stripe-specific logic from PaymentService. PaymentService
delegates to the adapter. No behavior change.

## Step 3: Wire up DI
Register IPaymentProcessor → StripePaymentProcessor.
Update consumers to depend on IPaymentProcessor instead of
PaymentService. No behavior change.

## Step 4: Remove PaymentService
Once all consumers use IPaymentProcessor, remove the old
PaymentService class. Characterization tests still pass.
```

Step 3: Agent implements each step.

Give the agent one step at a time. After each step, run the characterization tests. If they pass, commit. If they fail, the agent broke something; fix it before moving on.

This approach works because each step is small enough to review, safe enough to commit, and testable. The agent can work quickly because the scope is constrained. And you can stop after any step if something goes wrong, because each step is independently valid.

The Migration Safety Net

Every migration, regardless of type, needs a safety net. The safety net has three layers:

Layer 1: Tests

Tests are your first line of defense. Before any migration, ensure your test suite covers the areas you're changing. If coverage is thin, write characterization tests first (let the agent help, it's fast at generating tests from existing code).

After the migration, all existing tests must pass. New tests cover any new behavior or changed contracts.

Layer 2: Staged Rollout

Don't deploy migrations to all users at once. Use feature flags, canary deployments, or blue-green deployments to limit blast radius:

- **Feature flags** for API changes (new version behind a flag)
- **Canary deployments** for framework upgrades (10% of traffic first)
- **Blue-green** for database migrations (new schema active, old schema on standby)

Agents can help set up the staged rollout:

```
Add a feature flag for the v2 Orders API using our existing
feature flag system (Microsoft.FeatureManagement).
When the flag is disabled, v2 endpoints return 404.
When enabled, they work normally.
```

Layer 3: Rollback Plan

Every migration needs a rollback plan before it starts. For framework upgrades, that's usually "revert the commit." For database migrations, it's the `Down` method in the EF Core migration. For API versions, it's disabling the feature flag.

Document the rollback plan in the migration spec. Make the agent include it:

```
Add a "Rollback Plan" section to the migration spec.
Include specific commands to reverse each step.
```

Legacy Modernization Patterns

The biggest migration challenge most teams face isn't upgrading from .NET 8 to 10. It's modernizing legacy systems: .NET Framework 4.x to modern .NET, monoliths to services, old patterns to new ones.

Agents are surprisingly effective here because legacy modernization is largely pattern translation: take this old pattern, produce the equivalent in the new framework. The logic stays the same; the infrastructure changes.

The Strangler Fig Approach

Instead of rewriting everything at once, wrap the legacy system with a modern API and migrate feature by feature:

```
Analyze the legacy OrderController (ASP.NET MVC 5) and create
an equivalent minimal API endpoint in the new .NET 10 project.

The new endpoint should:
- Accept the same request format (for backward compatibility)
- Call the same business logic (through an anti-corruption layer)
- Return the same response format
- Add the new response fields our mobile app needs

Reference the legacy controller at legacy/Controllers/OrderController.cs
and the new project structure at src/Api/.
```

The agent reads the legacy code, understands the behavior, and produces a modern equivalent. Your job is reviewing the translation for correctness, especially around edge cases the legacy code handles implicitly (null checks that aren't obvious, error handling buried in base classes, configuration that affects behavior).

Batch Migration of Patterns

For large codebases, some migrations involve changing hundreds of files in the same way. Converting from `ILogger` property injection to constructor injection. Replacing `async void` event handlers with `async Task`. Updating from old EF Core query syntax to the new patterns.

Agents handle batch migrations well when you give them a clear before/after pattern:

```
Find all classes in src/ that inject ILogger via a property
(pattern: "public ILogger<T> Logger { get; set; }").

Convert each one to constructor injection:
- Add ILogger<T> parameter to the constructor
- Assign to a private readonly field
- Remove the property
- Update all usages of Logger to _logger

Do one project at a time. Run tests after each project.
```

The agent processes each file, applies the transformation, and verifies with tests. What would take you a day of tedious find-and-replace takes the agent 20 minutes.

The Migration Mindset

Migrations are where agents provide some of the clearest ROI in agentic development. The work is mechanical, pattern-heavy, and well-defined. The risks are manageable with tests and staged rollouts. And the time savings are substantial: a framework upgrade that takes a team two weeks by hand takes a few days with agents.

The key principles:

1. **Spec the migration before starting.** Know the scope, the breaking changes, and the rollback plan.

2. **Break it into atomic steps.** Each step is reviewable, testable, and committable.
3. **Tests are non-negotiable.** They're your proof that the migration preserved behavior.
4. **Stage the rollout.** Limit blast radius. Verify in production gradually.
5. **Let agents handle the mechanical work.** Save your judgment for the decisions that matter.

Migrations aren't exciting. But they're necessary, and with agents, they're fast. That's a win.

PART V

Scaling Up

CHAPTER 18

The Parallel Agent Lifestyle

You have one brain. You have one set of eyes. And starting today, you have five agents.

This is the parallel agent lifestyle, and it changes everything about how you structure your workday. Not because the tools are flashy, but because the economics are different. When code generation takes seconds and review takes minutes, the bottleneck shifts from “how fast can I write” to “how fast can I review.” And the only way to maximize review throughput is to have work ready to review at all times.

Simon Willison, creator of Datasette and one of the most public practitioners of agentic development, described his daily workflow in late 2025: three to five agents running simultaneously, each on a different task, each producing work he could review in bursts. He was skeptical at first. Then he wasn’t. “The natural bottleneck,” he said, “is how fast I can review.”

That’s the insight this chapter is built on. You’re not trying to replace yourself with agents. You’re trying to keep yourself, the reviewer, fully utilized.

The Four Categories

Not all agent tasks are created equal. Willison identified four categories of work that parallelize well, and this framework has held up under heavy use by thousands of developers since.

1. Research and Proof-of-Concept

“How does the Stripe subscription API handle prorated upgrades?” “What’s the recommended way to implement distributed caching with Redis in .NET Aspire?” “Show me three different approaches to implementing event sourcing in C#.”

These are research tasks. The agent reads documentation, explores APIs, builds small prototypes, and reports back. You don’t need to review these with the same rigor as production code. You’re gathering information, not shipping features.

Research agents are the lowest-risk parallel work. If the agent goes sideways, you’ve lost nothing. If it finds something useful, you’ve saved an hour of reading docs.

2. Codebase Questions

“How does our authentication middleware work?” “Trace the request flow from the API gateway to the order service.” “What’s the relationship between the User aggregate and the Permission entities?”

These are codebase archaeology tasks. The agent reads your code, follows the call chains, and produces documentation that helps you (or your teammates) understand what exists. This is one of the most underrated uses of agents: generating internal documentation from code.

The output is markdown, not code. The review is fast: does this accurately describe what I know about the system? If yes, you’ve just created documentation that would have taken a junior developer a full day to write.

3. Small Maintenance Tasks

Fix deprecation warnings. Update a NuGet package and fix the breaking changes. Rename a poorly named class across the codebase. Add XML documentation to public APIs. Convert a set of classes from the old pattern to the new pattern.

Maintenance tasks are the bread and butter of parallel agents. They’re well-defined, low-risk, and tedious for humans. An agent can churn through 50 deprecation warnings while you focus on actual feature work. The review is mechanical: did it fix the warnings without changing behavior? Run the tests and move on.

4. Carefully Specified Implementation Work

This is the high-value category, and the one that requires the most discipline. You write a specification, the agent implements it. The specification has to be precise enough that you'd feel comfortable handing it to a junior developer, because that's essentially what you're doing.

"Implement a `SubscriptionService` that handles plan upgrades with prorated billing. Use the Stripe API via `IStripeClient`. Return `Result<Subscription>` for all operations. Handle these error cases: invalid plan, payment failed, already subscribed. Write unit tests using xUnit and NSubstitute. Follow the existing service patterns in `src/Services/`."

That's a spec that produces reviewable output. Contrast it with "build the subscription feature," which produces code you'll spend more time understanding than you saved.

Git Worktrees: The Practical Foundation

You can't run three agents in the same directory. They'll step on each other's files, create merge conflicts in real time, and generally make a mess. Git worktrees solve this problem elegantly.

A worktree is a separate working directory that shares the same Git repository. Each worktree has its own branch, its own file state, and its own index. But they all share the same `.git` directory, which means they share history, remotes, and refs.

Here's the setup:

```
# You have your main repo
cd ~/repos/my-app

# Create worktrees for parallel agent sessions
git worktree add ../my-app-agent1 -b agent/subscription-service
git worktree add ../my-app-agent2 -b agent/fix-deprecations
git worktree add ../my-app-agent3 -b agent/add-caching
```

Now you have four directories:

```
~/repos/my-app/           # Your main working copy
~/repos/my-app-agent1/    # Agent 1: subscription service
~/repos/my-app-agent2/    # Agent 2: deprecation fixes
~/repos/my-app-agent3/    # Agent 3: caching layer
```

Each agent gets its own terminal, its own worktree, its own branch. They can all build and run tests independently. No conflicts. No coordination needed.

When an agent finishes, you review the diff, merge the branch, and either reuse the worktree for the next task or tear it down:

```
# Clean up a finished worktree
git worktree remove ../my-app-agent2
```

The Worktree Workflow in Practice

Here’s what a typical morning looks like with worktrees:

```
# Terminal 1: Main work (you're here)
cd ~/repos/my-app

# Terminal 2: Launch agent on subscription feature
cd ~/repos/my-app-agent1
claude "Implement the SubscriptionService according to the spec in docs/subscription-spec.md"

# Terminal 3: Launch agent on maintenance
cd ~/repos/my-app-agent2
claude "Fix all CS0618 deprecation warnings in the src/ directory. Run dotnet build after each

# Terminal 4: Launch agent on research
cd ~/repos/my-app-agent3
claude "Explore how to integrate Redis caching with our existing IRepository pattern. Build a p
```

You've just launched three agents. Total time: maybe five minutes to write the prompts. Now you go back to Terminal 1 and do your own work (the thing that actually requires your brain, like architectural design or a complex algorithm).

Thirty minutes later, Agent 2 finishes the deprecation fixes. You review the diff (2 minutes), run the tests (they pass), merge the branch. That worktree is now free for the next task.

An hour later, Agent 1 has the subscription service roughed out. You review, leave some feedback ("use the Result pattern, not exceptions, for business logic failures"), and let it iterate.

Agent 3 comes back with a caching proof-of-concept and a write-up. You read the summary, decide the approach is sound, and file it away for tomorrow's implementation.

Task Decomposition: The Parallel Mindset

The hardest part of parallel agents isn't the tooling. It's learning to think in parallel. Most developers are trained to work sequentially: finish one thing, start the next. Parallel agents require you to decompose work differently.

The key question isn't "what do I need to build?" It's "what can I break into independent pieces that don't touch the same files?"

Good Decomposition

Consider building a new feature: order notifications. A customer places an order, and the system sends email and SMS notifications. Here's how you might decompose it:

Agent 1: Notification Domain Model - `Notification` entity, `NotificationType` enum, `INotificationRepository` - Unit tests for the domain model - Files touched: `src/Domain/Notifications/` , `tests/Domain.Tests/Notifications/`

Agent 2: Email Integration - `EmailNotificationSender` implementing `INotificationSender` - Integration with SendGrid via `ISendGridClient` - Files touched: `src/Infrastructure/Email/` , `tests/Infrastructure.Tests/Email/`

Agent 3: SMS Integration - `SmsNotificationSender` implementing `INotificationSender` - Integration with Twilio via `ITwilioClient` - Files touched: `src/Infrastructure/Sms/` , `tests/Infrastructure.Tests/Sms/`

You (manually): Orchestration - `NotificationOrchestrator` that ties it all together - Integration tests that verify the full flow - This is the part that requires understanding all three pieces

Notice the pattern: each agent works in a different directory, on a different concern, with a shared interface (`INotificationSender`) that you defined upfront. The interface is the contract. The agents implement it independently. You wire it together.

Bad Decomposition

“Agent 1, build the backend. Agent 2, build the frontend. Agent 3, write the tests.”

This fails because the agents are working on the same conceptual feature from different angles, which means they’ll make conflicting assumptions. Agent 1 designs the API one way, Agent 2 expects it another way, and Agent 3 tests assumptions that match neither.

The rule of thumb: **decompose by bounded context or module, not by layer.** Each agent should own a vertical slice, not a horizontal layer.

The Interface-First Pattern

The best parallel agent workflows start with interfaces. Before launching any agents, you define the contracts:

```
public interface INotificationSender
{
    Task<Result<NotificationReceipt>> SendAsync(
        Notification notification,
        CancellationToken cancellationToken = default);
}

public interface INotificationRepository
{
    Task<Result<Notification>> GetByIdAsync(
        NotificationId id,
        CancellationToken cancellationToken = default);

    Task<Result> SaveAsync(
        Notification notification,
        CancellationToken cancellationToken = default);
}
```

These interfaces go into your main branch. Every worktree gets them. Every agent codes against them. Conflicts become nearly impossible because each agent is implementing a different side of the same contract.

This is spec-first development taken to its logical conclusion. The interfaces ARE the spec. The agents implement. You review.

Managing the Review Bottleneck

Here's the uncomfortable truth about parallel agents: the more agents you run, the more review work piles up. And if you let that pile grow, you lose the context that makes review efficient.

Three parallel agents producing 200-400 lines each gives you 600-1,200 lines to review. That's a big pull request by any standard. If you wait until all three are done and review them in a batch, you're going to miss things.

The Continuous Review Model

The better approach is continuous review. Instead of "launch everything, review later," you treat agent output like a stream.

Check in every 20-30 minutes. Glance at agent progress. If one is done, review it immediately while the context is fresh. If one is stuck or going in the wrong direction, intervene early.

Review in order of complexity. Start with the maintenance tasks (mechanical review: did the tests pass?). Move to the research (conceptual review: is this approach sound?). Finish with the implementation (deep review: is this correct, secure, maintainable?).

Timebox reviews. If a review is taking more than 15 minutes, the spec wasn't precise enough. Don't fix the code yourself. Fix the spec and re-run the agent. This feels slower in the moment but builds the habit of writing better specs.

You Are the Limiting Factor

This is the mindset shift that trips people up. In traditional development, the limiting factor is typing speed, comprehension speed, the raw mechanics of turning thoughts into code. With agents, you are no longer limited by production. You are limited by consumption.

Your job is to stay fed. Always have something to review. Always have agents working on the next batch. The moment you're sitting idle while agents work, you've under-scheduled. The moment you're drowning in unreviewed output, you've over-scheduled.

Three to five parallel agents is the sweet spot for most developers. Fewer than three and you'll have idle time. More than five and the review backlog becomes unmanageable. Find your personal number and stay there.

The Daily Rhythm

After a few weeks of parallel agent work, a natural rhythm emerges. Here's what a typical day looks like:

Morning (9:00 - 9:30): Launch and Spec

Review any overnight agent work (if you're using background agents, covered in Chapter 19). Write specs for the day's major tasks. Launch 2-3 agents on morning work.

Your morning specs are the most important writing you'll do all day. Spend the time. A 15-minute spec saves an hour of back-and-forth.

Late Morning (10:00 - 12:00): Deep Work + Review Cycles

Do your own deep work (architecture, complex problems, meetings). Check agent progress every 30 minutes. Review completed work. Re-launch agents on new tasks as old ones finish.

This is the productive core of the day. You're doing human-level thinking while agents handle implementation. The review breaks actually help your own work by providing natural Pomodoro-style intervals.

Afternoon (13:00 - 15:00): Integration and Second Wave

Merge morning agent work. Handle integration issues (the places where separately developed pieces don't quite fit). Launch afternoon agents on the next batch of tasks.

Integration is where your expertise matters most. The agents built the pieces. You're fitting them together, resolving the assumptions that don't align, and making architectural adjustments.

Late Afternoon (15:00 - 17:00): Review and Polish

Final review pass on the day's agent output. Write specs for tomorrow's morning tasks (or for overnight background agents). Clean up, commit, push.

Writing tomorrow's specs today means you can launch agents the moment you sit down in the morning. The parallel lifestyle rewards preparation.

When Parallel Helps vs. When It Creates Chaos

Parallel agents are not universally better. There are specific conditions where they shine and specific conditions where they make things worse.

Parallel Helps When:

Tasks are genuinely independent. Different modules, different files, different concerns. The agents don't need to know about each other's work.

You have a strong test suite. Tests are the fastest review mechanism. If the tests pass and they're comprehensive, you can merge with high confidence. If you don't have tests, every review becomes a manual audit.

Interfaces are defined upfront. When agents code against shared contracts, they produce compatible work. Without contracts, you're merging assumptions, not code.

Tasks are well-specified. The spec is the multiplier. Good specs produce good code from multiple agents simultaneously. Bad specs produce bad code from multiple agents simultaneously (which is worse, because you have more bad code to deal with).

Parallel Creates Chaos When:

Tasks have hidden dependencies. "Build the payment system" and "build the order system" sound independent until you realize they both need to update the same `Order` entity. Two agents editing the same file is a recipe for merge conflicts and logical inconsistencies.

The codebase lacks boundaries. If your code is a big ball of mud where everything touches everything, agents will create conflicts no matter how carefully you decompose. Parallel agents reward (and reveal) good architecture.

You skip the review. The temptation with parallel agents is to merge fast and move on. But every unreviewed merge is a time bomb. The debt compounds. Three agents producing unreviewed code for a week will create a codebase nobody understands, including you.

You're exploring, not building. When you don't know what you want yet, running three agents in three different directions doesn't help. It gives you three prototypes to evaluate instead of one, which sounds good until you realize evaluation takes longer than exploration.

Practical Example: Building a Feature with Three Parallel Agents

Let's walk through a real example. You're building an order export feature for an e-commerce application. Customers can export their order history as CSV or PDF.

Step 1: Define the Interfaces (15 minutes, you do this)

```
public interface IOrderExporter
{
    Task<Result<ExportResult>> ExportAsync(
        ExportRequest request,
        CancellationToken cancellationToken = default);
}

public record ExportRequest(
    Guid CustomerId,
    ExportFormat Format,
    DateRange DateRange);

public enum ExportFormat { Csv, Pdf }

public record ExportResult(
    byte[] Content,
    string ContentType,
    string FileName);
```

Step 2: Write Three Specs (20 minutes, you do this)

Spec A (Agent 1): CSV Exporter

> *Implement* `CsvOrderExporter : IOrderExporter`*. Use CsvHelper for CSV generation. Query orders from* `IOrderRepository.GetByCustomerIdAsync()`*. Include columns: OrderId, Date, Items (comma-separated), Total, Status. Write unit tests with xUnit. Mock the repository with NSubstitute. Test empty orders, single order, multiple orders, and date range filtering.*

Spec B (Agent 2): PDF Exporter

> *Implement* `PdfOrderExporter : IOrderExporter`*. Use QuestPDF for PDF generation. Same data source as CSV. Include a header with customer name and export date.*

> *Table layout for order rows. Write unit tests. Test the same scenarios as the CSV exporter.*

Spec C (Agent 3): API Endpoint

> *Add a `POST /api/customers/{customerId}/orders/export` endpoint to the existing `CustomersController`. Accept `ExportRequest` as the body. Resolve the correct `IOrderExporter` from DI based on the format. Return `FileResult` with appropriate content type. Add integration tests using `WebApplicationFactory`. Test both formats, invalid customer, empty date range.*

Step 3: Launch (5 minutes)

```
# Three terminals, three worktrees, three agents
cd ~/repos/shop-agent1 && claude "$(cat ~/specs/csv-exporter.md)"
cd ~/repos/shop-agent2 && claude "$(cat ~/specs/pdf-exporter.md)"
cd ~/repos/shop-agent3 && claude "$(cat ~/specs/api-endpoint.md)"
```

Step 4: Review and Merge (45 minutes over the next 2 hours)

Agent 2 finishes first (PDF generation is more complex, but QuestPDF has great docs and the agent nails it). You review: the code follows the existing patterns, the tests pass, the PDF looks reasonable. Merge.

Agent 1 finishes next. The CSV output is correct but the agent used manual string concatenation instead of CsvHelper. You point this out, the agent fixes it in 30 seconds. Merge.

Agent 3 finishes last (it needed to reference the other implementations for DI registration). You review the endpoint, run the integration tests, spot a missing authorization attribute (`[Authorize]`), the agent adds it. Merge.

Total time: about 40 minutes of your active attention, spread over 2 hours. What you got: a complete feature with two exporters, an API endpoint, unit tests, and integration tests. What it would have taken without agents: probably a full day.

That’s the parallel agent lifestyle. Not magic. Not autopilot. But a fundamentally different ratio of effort to output.

The Compound Effect

The real power of parallel agents isn’t any single session. It’s the compound effect over weeks and months. When you consistently produce 2-3x the output with roughly the same review quality, projects that used to take a quarter take a month. Features that used to take a sprint take a few days.

But only if you invest in the foundations: good architecture with clear boundaries, comprehensive tests, well-defined interfaces, and the discipline to write precise specs. Without those foundations, parallel agents just produce parallel messes faster.

The developers who thrive in the parallel agent lifestyle aren’t the fastest coders. They’re the clearest thinkers. They can decompose problems into independent pieces, specify them precisely, and review the results efficiently.

That’s the new skill. And it’s one worth developing.

CHAPTER 19

Background Agents

There's a particular kind of satisfaction in assigning a coding task from your phone while lying on the couch, then waking up the next morning to a pull request waiting for review. It feels like having a junior developer who works overnight, doesn't need coffee, and never complains about the assignment.

That's the promise of background agents. And in early 2026, the promise is largely delivered.

Background agents represent a fundamentally different interaction model from the IDE-based and CLI-based agents we've covered so far. With those, you're present. You're watching, guiding, iterating in real time. Background agents are fire-and-forget. You describe the task, the agent works autonomously in a cloud sandbox, and you come back later to review the results.

This chapter covers how to use them effectively, which tasks to trust them with, and how to set up your repository so background agents can actually succeed.

The Big Three

As of early 2026, three background agent platforms have meaningful traction.

GitHub Copilot Coding Agent

Microsoft's entry works directly with GitHub Issues. You assign an issue to "Copilot," and the agent creates a branch, writes the code, runs your CI pipeline, and opens a pull request. The entire workflow happens inside GitHub's infrastructure.

The integration is the selling point. If your team already lives in GitHub Issues and Pull Requests, the Copilot Coding Agent slots into your existing workflow with zero friction.

You don't need a new tool, a new dashboard, or a new process. You need an issue with a clear description.

In practice, the workflow looks like this:

1. Create a GitHub Issue: "Add rate limiting middleware to the API gateway. Use `AspNetCoreRateLimit` package. Apply a 100-requests-per-minute limit per API key. Return 429 with a `Retry-After` header. Add integration tests."
2. Assign the issue to Copilot.
3. Walk away.
4. Thirty minutes later (sometimes faster, sometimes slower), a PR appears with the implementation.

The quality of the PR is directly proportional to the quality of the issue description. Vague issues produce vague code. Detailed issues, with specific packages, patterns, test expectations, and error handling requirements, produce code that often needs only minor adjustments.

Between May and September 2025, GitHub reported over a million pull requests created by agents. That number has only grown. The Copilot Coding Agent is rapidly becoming the default way teams handle well-specified, moderate-complexity tasks.

Codex Cloud (OpenAI)

OpenAI's Codex Cloud takes a different approach. Instead of integrating with your issue tracker, it provides a dedicated interface (web and mobile) where you describe tasks and the agent works in a sandboxed cloud environment with a full copy of your repository.

The key differentiator is the mobile experience. You can launch a Codex task from your phone while commuting, while cooking dinner, while waiting for a meeting to start. The agent clones your repo, sets up the environment, writes the code, runs the tests, and presents you with a diff to review.

Codex Cloud is particularly good for the "I just thought of something" workflow. You're in the shower and realize you forgot to add input validation to the new endpoint. Pull

out your phone, describe the task, fire it off. By the time you're dressed, the PR is ready.

The tradeoff is that Codex Cloud operates in a sandbox that may not perfectly replicate your development environment. Complex build configurations, private NuGet feeds, or environment-specific dependencies can cause issues. The setup cost is higher, but once configured, the async workflow is seamless.

Google Jules

Google's Jules operates similarly to Codex Cloud but with tighter integration into the Google Cloud ecosystem. For teams already on Google Cloud, Jules provides a natural extension of their development workflow.

Jules has been particularly strong at multi-file refactoring tasks and framework migrations. Its context window and planning capabilities make it well-suited for tasks that require understanding broad codebase patterns before making changes.

Others Worth Watching

The background agent space is evolving rapidly. Amazon Q Developer handles AWS-centric tasks. Various startups (Sweep, Devin's async mode, and others) offer specialized background agent capabilities. The common thread is the same: describe a task, let the agent work, review the result.

The "Launch and Review" Model

Background agents enable a workflow pattern I call "launch and review." It's simple in concept and transformative in practice.

Evening (15-20 minutes): Launch

Before you close your laptop for the day, review your backlog. Pick 2-4 tasks that are well-specified, independent, and testable. Write clear descriptions (or use pre-written specs from your task board). Assign them to background agents.

```
Task 1: Add health check endpoints for all external dependencies
         (database, Redis, message broker). Use AspNetCore.HealthChecks
         packages. Register in Program.cs. Add integration tests.

Task 2: Migrate the UserService from the old repository pattern to
         the new CQRS pattern. Follow the pattern established in
         OrderService. Update all tests.

Task 3: Add structured logging with Serilog to the PaymentController.
         Log request/response for all endpoints. Mask sensitive fields
         (card numbers, CVV). Add unit tests for the masking logic.
```

Morning (30-45 minutes): Review

You sit down with coffee and three pull requests waiting. You review each one, leave comments where needed, approve the ones that pass muster, and request changes on the ones that don't. The agents iterate on your feedback while you move to your day's primary work.

This model effectively gives you a few extra hours of productive output per day, but those hours happen while you're sleeping, eating, or living your life. Over a week, that's 10-20 additional hours of implementation work. Over a month, it's the equivalent of having a part-time developer on your team.

The Morning Review Ritual

The review part of this workflow deserves its own process. Here's what works:

1. **Start with CI status.** If the tests failed, don't even look at the code yet. Check the failure, decide if it's an agent issue or a flaky test.
2. **Read the PR description.** Good background agents write detailed PR descriptions explaining what they did and why. Read this first for context.
3. **Scan the diff for red flags.** Security issues, deleted tests, changed configurations, new dependencies. These are the high-signal areas.

4. **Check test coverage.** Did the agent write tests? Do they test meaningful behavior or just cover lines?
5. **Run the code locally if needed.** For UI changes or complex integrations, pull the branch and verify manually.

Most well-specified background agent PRs take 5-10 minutes to review. If you're consistently spending 30+ minutes reviewing a single agent PR, the specs need work.

What Works Well for Background Agents

Background agents excel at specific categories of work. Understanding these categories helps you assign the right tasks and avoid frustration.

Well-Specified Implementation Tasks

"Add endpoint X with behavior Y, error handling Z, and tests A, B, C." When you've done the thinking and the agent does the typing, background agents are remarkably effective. The spec is the product. The code is just the mechanical output.

Repetitive Refactoring

"Convert all 15 repository classes from the old interface to the new interface." Humans hate this work. It's tedious, error-prone, and mind-numbing. Agents love it. They don't get bored, they don't skip classes, and they apply the pattern consistently.

Test Generation

"Add unit tests for all public methods in the PaymentService. Cover happy path, error cases, and edge cases. Use xUnit and NSubstitute. Follow the patterns in existing test files." Test generation is one of the highest-ROI uses of background agents. The work is well-defined, the quality is easy to verify (do the tests actually test meaningful behavior?), and the result directly improves your codebase.

Documentation

“Generate XML documentation for all public APIs in the `Services` namespace. Include parameter descriptions, return value descriptions, and example usage where appropriate.” Documentation is another task that’s easy to specify, easy to verify, and miserable for humans to do manually.

Dependency Updates

“Update `Microsoft.EntityFrameworkCore` from 8.x to 9.x. Fix all breaking changes. Ensure all tests pass.” Package updates with breaking changes are a perfect background agent task. The agent can read the migration guide, apply the changes, and iterate until the build passes.

What Doesn’t Work

Background agents fall apart on certain categories of tasks, and knowing these boundaries saves you from wasted cycles and frustrating mornings.

Ambiguous Requirements

“Improve the user experience of the dashboard.” Improve how? For whom? By what metric? Background agents can’t read your mind. They’ll make choices, and those choices will be different from what you wanted. Then you’ll spend more time understanding and undoing their work than it would have taken to do it yourself.

Tasks Requiring Clarification

If a task would make a human developer walk over to your desk and ask three questions before starting, a background agent will either guess wrong or get stuck. Background agents don’t (yet) have a good mechanism for asking clarifying questions mid-task. They make their best guess and ship it.

Complex Architectural Decisions

“Design the caching strategy for our microservices.” Designing is your job. Implementing a well-specified caching strategy is the agent’s job. Don’t confuse the two.

Tasks With Heavy External Dependencies

If the task requires access to a running database, a third-party sandbox API, or a specific hardware configuration, background agents in cloud sandboxes will struggle. They work best when the codebase is self-contained enough to build and test in isolation.

Deeply Interconnected Changes

Changes that touch many parts of the system simultaneously (cross-cutting concerns like changing your error handling strategy or your logging format) are risky for background agents. The agent might make the change correctly in 18 out of 20 places and subtly wrong in the other 2. Those 2 become bugs that are hard to find during review.

Setting Up Your Repo for Background Agent Success

Background agents clone your repo and try to build it. If that process is fragile, undocumented, or dependent on local state, the agent will fail before it writes a single line of code.

The README Test

Can a new developer clone your repo, follow the README, and have a running application with passing tests within 10 minutes? If no, fix that first. Background agents are effectively new developers joining your team every time they pick up a task.

Deterministic Builds

Use lock files (`packages.lock.json` in .NET, or enable central package management). Pin your SDK version with `global.json`. Ensure `dotnet build` and `dotnet test` work from a clean clone without any manual setup.

```
// global.json
{
  "sdk": {
    "version": "9.0.200",
    "rollForward": "latestPatch"
  }
}
```

Context Files

Background agents read your `AGENTS.md`, `.github/copilot-instructions.md`, or equivalent context files. These files are even more important for background agents than for interactive ones, because you can't course-correct in real time.

Your context files should include:

- Build and test commands
- Architecture overview (what's in each project/folder)
- Coding conventions (naming, patterns, error handling approach)
- What NOT to do (common mistakes, deprecated patterns)
- Test expectations (framework, mocking library, coverage requirements)

Comprehensive Test Suites

This is the most important factor. Background agents use tests as their feedback loop. When an agent writes code and runs the tests, failures tell it what to fix. Without tests, the agent has no signal. It writes code that compiles and calls it done.

The more comprehensive your tests, the higher quality output you'll get from background agents. This is one of those cases where investment in testing pays dividends in an entirely unexpected way: it makes your agents better.

CI Pipeline That Runs on PRs

Background agents, especially the GitHub Copilot Coding Agent, rely on your CI pipeline to validate their work. If your pipeline doesn't run on pull requests, or if it's slow (30+ minutes), or if it has flaky tests, the agent's feedback loop is broken.

Fast, reliable CI is a prerequisite for effective background agents. Aim for under 10 minutes from push to green (or red). Every minute of CI time is a minute the agent waits before it can iterate.

Real Workflow: Overnight Agent Tasks

Let's walk through a concrete weekly workflow using background agents.

Monday Evening

You've been planning the week's work. You have a feature (order export) that you'll build yourself, but there are several supporting tasks that are well-defined:

```
Issue #234: Add CSV export utility class with unit tests
Issue #235: Add PDF generation service using QuestPDF with unit tests
Issue #236: Update API documentation for the export endpoints
Issue #237: Add rate limiting to the export endpoint (100/hour/user)
```

You assign Issues #234 and #235 to the Copilot Coding Agent. You fire #236 to Codex Cloud from your phone while watching TV. #237 can wait until the export endpoint exists.

Tuesday Morning

Three PRs are waiting:

- **#234 (CSV utility):** Clean implementation, tests pass, one minor issue (used `StringBuilder` instead of `CsvHelper`). You leave a comment, the agent fixes it in 5 minutes. Merge.
- **#235 (PDF service):** Solid work. The agent even added a table of contents for exports with many orders. Tests pass. Merge.
- **#236 (API docs):** Accurate documentation based on the existing endpoint signatures. A few descriptions could be more specific. You edit them directly (faster than requesting changes for docs). Merge.

Total review time: 25 minutes. You now have three pieces of infrastructure ready for the export feature you're building today. You saved roughly 4-6 hours of implementation work.

Tuesday - Thursday

You continue this pattern. Each evening, review the backlog, pick 2-3 tasks, assign them to agents. Each morning, review PRs, merge, move on. By Thursday, you've completed roughly twice the throughput of a normal week, with the same number of working hours.

The Compounding Effect

Over a month, background agents contribute the equivalent of 40-80 hours of additional implementation work. That's not a rounding error. That's a full extra developer's output. The cost is subscription fees (covered in Chapter 22) and 30-45 minutes of review per morning.

The developers who get the most from background agents are the ones who build a pipeline. They maintain a backlog of well-specified, agent-ready tasks. They write specs as part of their planning process, not as an afterthought. They treat background agents as a permanent part of their team's capacity.

The Trust Calibration

Background agents require a different trust model than interactive agents. With an interactive agent, you're watching it work. You catch mistakes in real time. With a background agent, you're reviewing after the fact. The mistakes are already in the code when you see them.

This means your review process needs to be more rigorous, not less. Don't merge background agent PRs without:

1. Reading the full diff (not just the summary)
2. Checking that tests exist AND test meaningful behavior
3. Verifying security-sensitive code (auth, input validation, data access)
4. Running the test suite locally for complex changes

Over time, you'll calibrate your trust. You'll learn which categories of tasks your background agents handle reliably and which ones need extra scrutiny. You'll learn to write specs that produce consistently good results. You'll develop a sixth sense for which PRs need a deep review and which ones are rubber-stamp merges.

That calibration is earned, not assumed. Start with more review, not less. Relax as you build confidence. And never stop reading the diffs.

CHAPTER 20

Multi-Agent Coordination

Running three agents in parallel is one thing. Having those agents coordinate with each other is another thing entirely.

Parallel agents (Chapter 18) work on independent tasks. They don't know about each other. They don't need to. Multi-agent coordination is what happens when the tasks aren't independent, when Agent A's output is Agent B's input, when three agents need to work on the same feature without stepping on each other's code, when one agent needs to plan the work and distribute it to others.

This is harder. It's also where the real leverage lives.

The Specialization Model

The most common multi-agent pattern is specialization: different agents with different roles working on different aspects of the same feature. Think of it like a small team where each member has a clear responsibility.

The Classic Three-Agent Setup

For a full-stack feature, the setup looks like this:

API Agent: Builds the backend, the controllers, services, repositories, DTOs, and API tests. This agent has deep context about your backend architecture, your data access patterns, and your API conventions.

Frontend Agent: Builds the UI components, pages, state management, and frontend tests. This agent knows your component library, your routing patterns, and your styling conventions.

Test Agent: Writes integration tests that verify the API and frontend work together. This agent understands your test infrastructure, your test data setup, and your CI pipeline.

Each agent gets its own context file tailored to its specialty. The API agent's context file emphasizes your backend patterns. The frontend agent's context file emphasizes your component architecture. Specialization isn't just about dividing work; it's about giving each agent the right context for its role.

Why Specialization Works

The alternative to specialization is giving one agent the entire task: "Build the full-stack order export feature." This works for simple features but falls apart as complexity grows. A single agent trying to hold backend architecture, frontend patterns, and test infrastructure in context simultaneously will make compromises in all three areas.

Specialized agents produce better output because they have focused context. The API agent doesn't waste context tokens on React component patterns. The frontend agent doesn't waste context tokens on Entity Framework configurations. Each agent is an expert in its lane.

The tradeoff is coordination overhead. Three specialized agents need someone (or something) to make sure their outputs fit together. That's where orchestration comes in.

Agent Orchestrators

An orchestrator is an agent (or tool) that manages other agents. It takes a high-level goal, breaks it into tasks, assigns those tasks to specialized agents, monitors progress, handles failures, and integrates the results.

The Planning Phase

The orchestrator's first job is planning. Given "Build the order export feature," the orchestrator:

1. Reads the feature specification

2. Identifies the components needed (API endpoint, exporter services, frontend page, tests)
3. Determines dependencies (the frontend needs the API to exist first, integration tests need both)
4. Creates a task graph with ordering constraints
5. Assigns tasks to specialized agents in the right order

This planning step is crucial. Without it, you get agents working at cross-purposes or building against interfaces that don't exist yet.

The Coordination Phase

Once tasks are assigned, the orchestrator monitors execution:

- **Progress tracking:** Which agents are done? Which are stuck?
- **Dependency management:** Agent B can't start until Agent A delivers the API contract.
- **Conflict detection:** Are two agents trying to modify the same file?
- **Failure handling:** If Agent A fails, what happens to the agents that depend on its output?

File-Based Locking

When multiple agents work on the same repository, file conflicts are the primary coordination challenge. Several patterns have emerged:

Branch isolation: Each agent works on its own branch (same as the worktree pattern from Chapter 18). The orchestrator handles merging.

File ownership: The orchestrator assigns file-level ownership. Agent A owns `src/Services/`, Agent B owns `src/Controllers/`, Agent C owns `tests/`. No agent touches another agent's files.

Interface contracts: Agents communicate through shared interfaces defined before any agent starts working. The interfaces are committed to the main branch. Each agent implements against them in isolation.

Lock files: Some orchestrators use a simple lock file mechanism. Before an agent modifies a file, it claims a lock. Other agents skip or queue changes to locked files. This is the most granular approach but adds complexity.

In practice, branch isolation combined with interface contracts handles 90% of coordination needs. Lock files are overkill for most teams.

VS Code as Multi-Agent HQ

In February 2026, Microsoft announced that VS Code would become a unified interface for managing multiple agents simultaneously. This isn't just marketing. The practical implications are significant.

Before this announcement, running multiple agents meant multiple terminals, multiple browser tabs, and manual context-switching between them. VS Code's multi-agent support puts all your agents in one interface:

- **Side-by-side agent panels:** See Claude Code, Codex, and Copilot working simultaneously
- **Shared workspace context:** All agents see the same files, the same Git state, the same test results
- **Unified diff review:** Review output from multiple agents in a single diff view
- **Agent-to-agent handoffs:** Route the output of one agent as input to another

This is still early (as of this writing), but the direction is clear: the IDE becomes the command center for a team of agents, not just a text editor with AI features.

For .NET developers, this is particularly relevant because VS Code's C# support (via the C# Dev Kit) has reached parity with Visual Studio for most workflows. Running multiple agents in VS Code while maintaining full IntelliSense, debugging, and test runner integration makes the multi-agent workflow practical for day-to-day development.

Branch Strategy for Multi-Agent Work

When multiple agents contribute to the same feature, your Git strategy matters more than usual.

The Feature Branch Pattern

```
main
 └── feature/order-export           (integration branch)
      ├── agent/order-export-api     (API agent's branch)
      ├── agent/order-export-ui      (Frontend agent's branch)
      └── agent/order-export-tests   (Test agent's branch)
```

The feature branch is the integration point. Each agent works on its own sub-branch. When an agent completes its work:

1. You review the agent's PR against the feature branch
2. Merge if approved
3. The next agent rebases on the updated feature branch and continues

This pattern keeps main clean (no half-finished features) while allowing agents to work in parallel on separate branches.

Handling Merge Conflicts

Merge conflicts in multi-agent work usually fall into two categories:

Trivial conflicts: Two agents added import statements in the same file. These resolve automatically or with a quick manual fix.

Semantic conflicts: Two agents made incompatible assumptions about shared code. The API agent returned `OrderExportResult` but the frontend agent expected `ExportResponse`. These require you to make a decision and update one or both implementations.

Semantic conflicts are the reason you define interfaces upfront. With shared contracts, the only conflicts are trivial ones. Without them, every merge is a negotiation.

The Rebase-and-Continue Pattern

When Agent A's work is merged and Agent B needs to incorporate those changes:

```
cd ~/repos/my-app-agent-b
git fetch origin
git rebase origin/feature/order-export
# Agent B continues working with Agent A's code now visible
```

If you're using worktrees, this is seamless. The agent's worktree picks up the rebased code and continues. In practice, you often just re-launch the agent with an updated prompt: "The API is now merged. Implement the frontend against the actual endpoints in `src/Controllers/OrderExportController.cs`."

The Orchestrator Pattern

The most powerful (and most complex) multi-agent workflow uses a dedicated orchestrator agent that manages the others. This is the pattern used by tools like ComposioHQ's agent-orchestrator, and it's the pattern that scales to genuinely complex features.

How It Works

1. **You give the orchestrator a goal:** "Build the order export feature according to the spec in `docs/order-export.md`."
2. **The orchestrator reads the spec** and creates a task breakdown.
3. **The orchestrator spawns specialized agents** for each task, with appropriate context and instructions.
4. **The orchestrator monitors progress,** checking each agent's output, running tests, and intervening when agents get stuck.

5. **The orchestrator integrates the results,** merging branches, resolving conflicts, and running the full test suite.
6. **The orchestrator reports back** with a summary of what was built, what worked, and what needs your attention.

Personal AI Agents as Orchestrators

This is where tools like OpenClaw enter the picture. A personal AI agent running on your machine (or a server you control) has persistent context, access to your full development environment, and the ability to spawn and manage sub-agents.

The workflow looks like this:

```
You: "Build the order export feature. Spec is in the usual place."

OpenClaw (orchestrator):
  1. Reads docs/order-export.md
  2. Identifies three work streams: API, frontend, tests
  3. Creates three worktrees and branches
  4. Spawns Agent 1 on API implementation
  5. Spawns Agent 2 on frontend (queued until API contract is ready)
  6. Waits for Agent 1 to deliver the API contract
  7. Spawns Agent 2 with the actual API contract as context
  8. Spawns Agent 3 on integration tests (after both are done)
  9. Reviews all output, runs full test suite
  10. Presents you with a summary and three PRs to review
```

You went from “read the spec” to “review three PRs” with a single instruction. The orchestrator handled decomposition, sequencing, context management, and integration.

This is the high end of multi-agent coordination. It requires trust in the orchestrator, comprehensive tests (the orchestrator’s primary quality signal), and well-structured repositories. But when it works, the leverage is extraordinary.

Real Example: Building a Full-Stack Feature with Coordinated Agents

Let's walk through building a customer notification preferences feature using coordinated agents. Customers can choose which notifications they receive (email, SMS, push) and set quiet hours.

Phase 1: Contract Definition (You, 20 minutes)

You write the API contract and the shared types:

```
// Shared contract
public record NotificationPreferences
{
    public bool EmailEnabled { get; init; }
    public bool SmsEnabled { get; init; }
    public bool PushEnabled { get; init; }
    public TimeOnly QuietHoursStart { get; init; }
    public TimeOnly QuietHoursEnd { get; init; }
}

// API contract
// GET  /api/customers/{id}/notification-preferences
// PUT  /api/customers/{id}/notification-preferences
// Response: NotificationPreferences
// Errors: 404 (customer not found), 400 (invalid quiet hours range)
```

You commit this to the feature branch. All agents will see it.

Phase 2: Parallel Implementation (3 agents, ~1 hour)

Agent 1 (API): "Implement the notification preferences API endpoints. Use the contract in `src/Contracts/NotificationPreferences.cs`. Create

`NotificationPreferencesService` and `NotificationPreferencesRepository`. Use EF Core for persistence. Add a migration. Write unit and integration tests."

Agent 2 (Frontend): "Build a notification preferences page at `/settings/notifications`. Use the API contract types. Toggle switches for email/SMS/push. Time pickers for quiet hours. Optimistic UI updates. Component tests with React Testing Library."

Agent 3 (Infrastructure): "Add notification preferences to the database schema. Create the EF Core migration. Seed default preferences for existing customers (all enabled, no quiet hours). Add a health check for the notification preferences table."

Phase 3: Integration (You, 30 minutes)

Agent 1 and Agent 3 both touched the database schema. You resolve the migration ordering (Agent 3's seed data migration depends on Agent 1's schema migration). Quick fix.

Agent 2's frontend calls the API correctly because it coded against the shared contract. The only adjustment: you add CORS configuration for the new endpoint, which neither agent thought to do.

Run the full test suite. 47 tests pass, 2 fail (both are timezone-related edge cases in the quiet hours logic). You fix them manually. Takes 10 minutes.

Total elapsed time: about 2 hours, of which maybe 50 minutes was your active involvement. The feature is complete with API, frontend, database migration, seed data, and comprehensive tests.

When Multi-Agent Coordination Makes Sense

Multi-agent coordination has significant overhead. The planning, the interface definitions, the branch management, the integration work. This overhead is only worth paying when the feature is complex enough to justify it.

Use Multi-Agent Coordination When:

- The feature has clear architectural boundaries (API/frontend/infrastructure)
- Multiple people would work on it in a traditional team
- The interfaces between components can be defined upfront
- You have a comprehensive test suite to catch integration issues
- The feature would take more than a day to build solo

Stick to Single or Parallel Agents When:

- The feature is small enough to hold in one context
- The components are tightly coupled (changes in one require changes in another)
- You're still exploring the design (agents can't coordinate on a moving target)
- Your test coverage is thin (you won't catch integration bugs)

Multi-agent coordination is a power tool. It's not for every nail. But for the nails it fits, nothing else comes close.

The Future of Multi-Agent Coordination

We're in the early days. The tools are improving rapidly. VS Code's multi-agent support will mature. Orchestrator frameworks will get smarter about planning and conflict resolution. Agents will develop better protocols for communicating with each other (not just through files, but through structured messages).

The trajectory is clear: development teams will be hybrid, some human members, some agent members, coordinated through the same tools and processes. The developers who learn to orchestrate agents today will be the ones leading those hybrid teams tomorrow.

But the fundamentals won’t change. Clear specifications. Well-defined interfaces. Comprehensive tests. Good architecture. These have always been the foundation of effective teamwork, whether the team members are human or artificial.

CHAPTER 21

Agent + CI/CD Integration

Your CI pipeline just failed. A NuGet package update introduced a breaking change in the serialization layer. Twelve tests are red. In the old world, you'd spend 30 minutes diagnosing the failure, 45 minutes fixing the code, and 15 minutes waiting for CI to go green again.

In the new world, an agent reads the CI failure log, identifies the breaking change, fixes the serialization code, updates the affected tests, pushes a commit, and the pipeline goes green. You get a Slack notification: "CI failure on `main` auto-fixed. See commit `a3f7bc2`."

That's not science fiction. That's Tuesday.

Agent-integrated CI/CD pipelines represent one of the most practical and under-discussed applications of agentic development. While most of the conversation around coding agents focuses on writing new code, the real day-to-day value often comes from agents that maintain, fix, and optimize your existing delivery pipeline.

Agents That React to Failing Builds

The simplest and most immediately useful CI/CD integration is an agent that reacts to build failures. The pattern is straightforward:

1. CI pipeline fails
2. A webhook triggers the agent with the failure log
3. The agent analyzes the failure, reads the relevant code, and proposes a fix
4. The fix is either auto-committed (for low-risk changes) or submitted as a PR (for anything substantial)

What Agents Can Fix Automatically

Not all CI failures are created equal. Agents handle some categories reliably and others poorly.

High confidence (auto-commit candidates): - Compilation errors from updated dependencies (renamed methods, changed signatures) - Formatting/linting failures - Missing using statements after refactoring - Test failures caused by changed return types or renamed properties - Outdated snapshot tests

Medium confidence (PR candidates, human review required): - Test failures from changed business logic - Integration test failures from environment changes - Performance test regressions - Security scan findings

Low confidence (alert only, don't auto-fix): - Mysterious runtime failures with no clear cause - Intermittent/flaky test failures - Infrastructure failures (Docker build issues, network timeouts) - Authentication or permission errors

The categorization matters because the whole point of CI is trust. If your agent auto-commits a "fix" that papers over a real bug, you've undermined the pipeline's purpose. Start conservative (alert only), graduate to PRs (human review), and only auto-commit when you've built confidence in specific failure categories.

Implementation Pattern

Here's a concrete implementation using GitHub Actions:

```yaml
# .github/workflows/auto-fix.yml
name: Agent Auto-Fix

on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]

jobs:
  auto-fix:
    if: ${{ github.event.workflow_run.conclusion == 'failure' }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ github.event.workflow_run.head_branch }}

      - name: Download failure logs
        uses: actions/download-artifact@v4
        with:
          name: test-results
          run-id: ${{ github.event.workflow_run.id }}

      - name: Analyze and fix
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: |
          # Feed failure logs to the agent
          claude --print "Analyze the CI failure in test-results/.
            Fix the issue in the source code.
            Run 'dotnet test' to verify.
            Only modify files that are directly related to the failure.
            Do not change test assertions unless the test expectation
            is clearly outdated." \
            --allowedTools edit,bash
```

```
      - name: Create PR if changes exist
        if: ${{ steps.fix.outputs.changes == 'true' }}
        uses: peter-evans/create-pull-request@v6
        with:
          title: "fix: auto-remediation for CI failure"
          body: "Automated fix for CI failure. Review carefully."
          branch: auto-fix/${{ github.event.workflow_run.head_sha }}
```

This is a starting point. Production implementations need safeguards: rate limiting (don't let the agent create 50 PRs for the same flaky test), scope limiting (only touch files related to the failure), and approval gates (require human approval before merge).

Pipeline Generation with Agents

Writing CI/CD pipelines is one of those tasks that's tedious, error-prone, and well-suited to agents. The YAML is finicky. The documentation is sprawling. The debugging cycle is slow (push, wait for CI, read the error, fix, repeat).

GitHub Actions Generation

Give an agent your project structure and ask for a CI pipeline:

"Generate a GitHub Actions workflow for this .NET 9 solution. It should: build all projects, run unit tests with coverage reporting, run integration tests against a PostgreSQL service container, publish coverage to Codecov, and create a release on tag push. Use caching for NuGet packages. Target `ubuntu-latest`."

The agent produces a working `ci.yml` in seconds. But here's the important part: you review it. CI pipelines are security-sensitive. An agent that adds `permissions: write-all` or a dubious third-party action is a real risk. Review pipeline changes with the same rigor you'd apply to authentication code.

Azure DevOps Pipelines

The same pattern works for Azure DevOps, though the YAML schema is different and the documentation is more fragmented. Agents are particularly useful here because Azure DevOps pipeline syntax has subtle differences from GitHub Actions that trip up even experienced developers. Let the agent handle the syntax; you focus on the logic.

```yaml
# Agent-generated azure-pipelines.yml (review before committing!)
trigger:
  branches:
    include:
      - main
      - release/*

pool:
  vmImage: 'ubuntu-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.0.x'

stages:
  - stage: Build
    jobs:
      - job: BuildAndTest
        steps:
          - task: UseDotNet@2
            inputs:
              version: $(dotnetVersion)

          - task: DotNetCoreCLI@2
            displayName: 'Restore'
            inputs:
              command: restore
              projects: '**/*.csproj'

          - task: DotNetCoreCLI@2
            displayName: 'Build'
            inputs:
              command: build
              projects: '**/*.csproj'
              arguments: '--configuration $(buildConfiguration) --no-restore'
```

```
            - task: DotNetCoreCLI@2
              displayName: 'Test'
              inputs:
                command: test
                projects: '**/*Tests.csproj'
                arguments: '--configuration $(buildConfiguration) --collect:"XPlat Code Coverage"
```

Infrastructure as Code with Agents

Terraform, Bicep, and Pulumi configurations are another category where agents provide significant leverage. IaC files are declarative, well-documented, and follow predictable patterns. Agents excel at generating them.

Bicep for Azure Deployments

For .NET developers deploying to Azure, Bicep is the natural IaC choice. Agents can generate Bicep files from a description of your architecture:

“Generate Bicep modules for a .NET Aspire application deployment: an Azure Container Apps environment, two container apps (API and worker), an Azure SQL database, a Redis cache, and Application Insights. Use managed identity for all service-to-service communication. Output connection strings as Container Apps secrets.”

The agent produces modular Bicep files that follow Azure best practices. But IaC is another area where review is critical. A misconfigured network rule, an overly permissive role assignment, or a missing encryption setting can create security vulnerabilities. Review IaC changes with security in mind, not just “does it deploy.”

Terraform and Pulumi

The same pattern applies to Terraform (for multi-cloud or AWS/GCP deployments) and Pulumi (for developers who prefer C# over HCL). Agents handle the syntax and resource configuration; you handle the architecture decisions and security review.

Pulumi is particularly interesting for .NET developers because you write infrastructure in C#:

```
// Agent-generated Pulumi infrastructure
var resourceGroup = new ResourceGroup("rg-myapp");

var appServicePlan = new AppServicePlan("plan-myapp", new()
{
    ResourceGroupName = resourceGroup.Name,
    Sku = new SkuDescriptionArgs
    {
        Name = "B1",
        Tier = "Basic",
    },
});

var appService = new WebApp("app-myapp", new()
{
    ResourceGroupName = resourceGroup.Name,
    ServerFarmId = appServicePlan.Id,
    SiteConfig = new SiteConfigArgs
    {
        NetFrameworkVersion = "v9.0",
        AlwaysOn = true,
    },
});
```

An agent can generate, modify, and refactor Pulumi code just like application code. The same context engineering principles apply: give the agent your architecture decisions, your naming conventions, and your security requirements.

API Contract Testing and OpenAPI

Agents are remarkably good at generating and maintaining OpenAPI specifications. This matters because API contracts are the glue between services, and keeping them accurate is one of those tasks that teams consistently neglect.

Generating OpenAPI Specs from Code

"Generate an OpenAPI 3.1 specification for all controllers in `src/Api/Controllers/`. Include request/response schemas, error responses, authentication requirements, and example values. Use the XML documentation on the controller methods for descriptions."

The agent reads your controllers, extracts the routes, parameters, and return types, and produces a complete OpenAPI spec. You validate it against the actual API behavior (run the tests, hit the endpoints), and now you have accurate, up-to-date API documentation.

Contract Testing

Once you have an OpenAPI spec, agents can generate contract tests that verify your API actually matches the spec:

```
[Theory]
[InlineData("/api/customers", "GET", 200)]
[InlineData("/api/customers/999", "GET", 404)]
[InlineData("/api/customers", "POST", 201)]
[InlineData("/api/customers", "POST", 400)] // Invalid body
public async Task ApiEndpoint_MatchesOpenApiContract(
    string path, string method, int expectedStatus)
{
    // Agent generates these tests from the OpenAPI spec
    var request = CreateRequestFromSpec(path, method, expectedStatus);
    var response = await _client.SendAsync(request);

    Assert.Equal(expectedStatus, (int)response.StatusCode);
    await AssertResponseMatchesSchema(response, path, method, expectedStatus);
}
```

This is one of those areas where agents create a virtuous cycle: the agent generates the spec from code, generates tests from the spec, and the tests catch when code and spec diverge. The maintenance burden drops to near zero.

The Agent-in-the-Loop CI/CD Pipeline

Putting it all together, here’s what a fully agent-integrated CI/CD pipeline looks like:

```
Developer pushes code
    │
    ▼
CI Pipeline Runs
    │
    ├── Build ── Fails? ──► Agent analyzes, creates fix PR
    │
    ├── Unit Tests ── Fails? ──► Agent analyzes, creates fix PR
    │
    ├── Integration Tests ── Fails? ──► Agent analyzes, alerts team
    │
    ├── Security Scan ── Findings? ──► Agent creates remediation PR
    │
    ├── API Contract Check ── Drift? ──► Agent updates spec or code
    │
    ├── Code Coverage ── Below threshold? ──► Agent generates missing tests
    │
    └── All Green ──► Deploy to staging
                          │
                          ├── Smoke Tests ── Fails? ──► Agent rolls back, alerts
                          │
                          └── All Green ──► Deploy to production
```

Each failure point has an agent that can respond intelligently. Not blindly, not without oversight, but with targeted analysis and proposed fixes that a human can review and approve.

The key insight: agents in CI/CD aren't replacing your pipeline. They're reducing the time between "something broke" and "here's the fix." Instead of a developer context-switching away from their current task to investigate a CI failure, the agent handles triage and initial remediation. The developer reviews and approves.

Security Scanning with Agent Remediation

Security scanning in CI is common. Acting on the results is rare. Most teams have a security scan that produces findings, and those findings sit in a backlog until someone gets around to them (which is never).

Agents change this dynamic. When a security scan finds a vulnerability:

1. The agent reads the finding (CVE ID, affected package, severity)
2. The agent checks for an available fix (updated package version, patch)
3. The agent applies the fix, updates the package, and runs the tests
4. The agent creates a PR with the fix and a summary of the vulnerability

For dependency vulnerabilities (the most common category), this is nearly fully automatable. The fix is usually "update package X to version Y." The agent does it, the tests verify nothing broke, and you approve the PR.

For code-level vulnerabilities (SQL injection, XSS, insecure deserialization), the agent can still help. It may not produce a perfect fix, but it can identify the vulnerability, explain the risk, and propose a remediation approach. That's better than a finding sitting in a backlog for six months.

Real Example: Agent Fixes a Failing CI Build

Let's walk through a concrete scenario. Your team pushes a routine PR that updates `Microsoft.EntityFrameworkCore` from 9.0.1 to 9.0.2. CI runs. Eight tests fail.

Without Agent Integration

1. Developer gets notified (Slack, email). Maybe they see it 30 minutes later.

2. Developer opens the CI logs. Reads through the failure output. Identifies that the `SaveChangesAsync` behavior changed for batch operations.
3. Developer reads the EF Core 9.0.2 release notes. Finds the breaking change.
4. Developer updates the affected code. Runs tests locally. Pushes a fix.
5. CI runs again. Green. Total time: 60-90 minutes.

With Agent Integration

1. CI fails. The auto-fix workflow triggers.
2. Agent reads the test failure output. Identifies 8 failures, all in `RepositoryTests`.
3. Agent reads the EF Core 9.0.2 release notes (it fetches the changelog from NuGet).
4. Agent identifies the breaking change: batch `SaveChangesAsync` now throws `DbUpdateConcurrencyException` instead of returning 0 for concurrent modifications.
5. Agent updates the repository implementations to handle the new exception type.
6. Agent updates the affected tests to expect the new behavior.
7. Agent runs `dotnet test`. All green.
8. Agent creates a PR with a clear description: "Fix EF Core 9.0.2 breaking change in batch save behavior. Updated 3 repository classes and 8 tests. See: [link to release notes]."
9. Developer reviews the PR. Approves. Merges. Total developer time: 5-10 minutes.

That's the difference. Not "the developer didn't have to do anything." The developer still reviewed, still approved, still made the judgment call. But the grunt work of diagnosis and remediation happened automatically.

Setting Up Agent CI/CD Integration

If you want to add agent capabilities to your CI/CD pipeline, start small:

Step 1: Alert-Only Mode

Add an agent that reads CI failure logs and posts an analysis to your team channel. No auto-fixing. Just “here’s what broke and here’s my suggested fix.” This builds trust and helps you calibrate the agent’s analysis quality.

Step 2: PR Mode

Graduate to creating PRs for well-understood failure categories. Dependency updates, formatting fixes, snapshot test updates. Low-risk, high-frequency fixes that don’t require deep domain knowledge.

Step 3: Selective Auto-Commit

For the categories where the agent has proven reliable (formatting, linting, dependency version bumps), enable auto-commit on the branch (not on main). The changes still go through PR review before merging to main.

Step 4: Full Integration

Build out the full pipeline with agent checkpoints at each stage. Security scanning, contract testing, coverage enforcement, all with agent remediation capabilities.

The progression matters. Jumping straight to auto-commit for everything is a recipe for disaster. Build trust incrementally, the same way you would with a new team member.

The Cost-Benefit Reality

Agent CI/CD integration has real costs: API tokens for every CI run that triggers an agent, compute time for the agent’s analysis, and the maintenance burden of the integration itself.

For most teams, the math works out. A single CI failure that blocks a team of 5 developers for an hour costs 5 developer-hours. An agent that resolves the failure in 10 minutes (plus 5 minutes of review) costs a few dollars in API tokens. The ROI is clear.

But track it. Measure how often agents successfully fix failures vs. how often they produce noise. If the signal-to-noise ratio is poor, adjust the categories, tighten the scope, or improve your context files. Agent CI/CD integration is a tool that requires tuning, not a set-and-forget solution.

CHAPTER 22

Cost Management

Nobody talks about this.

Every blog post about AI-powered development mentions the productivity gains. Almost none of them mention the bill. And the bill is real. If you're running multiple agents, using background agents for async work, and integrating agents into your CI/CD pipeline, you're spending money. Potentially a lot of money.

This chapter is about understanding, managing, and justifying that spend.

The Real Cost of Running Multiple Agents

Let's start with honesty. Here's what a serious agentic development setup costs in early 2026:

Subscription Stacking

Most developers using agents seriously have multiple subscriptions:

Service	Monthly Cost	What You Get
GitHub Copilot Business	$19/user	Copilot Chat, Copilot Coding Agent, PR summaries
Claude Pro	$20/user	Claude Code with extended thinking, higher rate limits
Cursor Pro	$20/user	Cursor's agent mode, multi-file editing
ChatGPT Pro	$200/user	Codex Cloud, background agents, higher limits

That's potentially $259/month per developer, before you write a single line of agent-integrated CI/CD (which uses API tokens, not subscriptions).

Most developers don't need all four. But many have two or three, because different tools excel at different tasks. Claude Code for complex reasoning and large refactors. Copilot for inline completions and GitHub integration. Cursor for its multi-file editing UI. The tools aren't interchangeable, so the subscriptions stack.

API Token Costs

Subscription plans have rate limits. When you hit those limits, or when you need agents in CI/CD pipelines, you pay per token.

As of early 2026, representative API pricing:

Model	Input (per 1M tokens)	Output (per 1M tokens)
Claude Sonnet 4.5	$3	$15
Claude Opus 4	$15	$75
GPT-4o	$2.50	$10
GPT-5	$10	$40
Claude Haiku 4.5	$0.80	$4

A single complex coding session with Claude Code can consume 100K-500K tokens. At Sonnet 4.5 rates, that's $0.30-$1.50 per session for input, plus $1.50-$7.50 for output. Run 10 sessions a day and you're looking at $20-$90/day in API costs alone.

For CI/CD agents that trigger on every failed build, the costs scale with your commit frequency. A team of 10 developers pushing 50 commits/day with a 5% failure rate means 2-3 agent invocations per day. Manageable. But a team going through a major refactor with 30% CI failure rates? That's 15 agent invocations per day, each consuming 50K-200K tokens for analysis and remediation.

The Hidden Cost: Context Window Bloat

Here's something most developers don't realize: your context files cost money every time an agent loads them.

If your `AGENTS.md` is 5,000 tokens, and your agent loads it at the start of every session, and you run 20 sessions a day, that's 100,000 tokens per day just for context loading. At $3/million tokens, that's $0.30/day, or about $9/month. Not a lot on its own.

But add a bloated context file (20,000 tokens), three rule files (5,000 tokens each), and an overly detailed project README (10,000 tokens), and you're loading 45,000 tokens of context per session. Twenty sessions a day, that's 900,000 tokens/day, or $2.70/day, $81/month. Just for context. Before the agent does any actual work.

This is why context engineering isn't just about quality. It's about economy. Every token in your context files is a recurring cost, multiplied by every session, every developer, every day.

Context Window Efficiency

The most impactful cost reduction isn't switching to a cheaper model. It's making your context files smaller.

Audit Your Context Files

Pull up your `AGENTS.md` (or `.cursorrules`, or `copilot-instructions.md`). How long is it? If it's over 2,000 tokens (roughly 1,500 words), it's probably too long. Agents don't need your project's entire history. They need:

- Build commands (2-3 lines)
- Architecture overview (10-15 lines)
- Key conventions (10-15 lines)
- What not to do (5-10 lines)

That's about 500-800 tokens. Everything else should be in scoped rules that only load when relevant (e.g., C# rules only load when editing `.cs` files).

Use Scoped Rules

Instead of one massive context file, break it into scoped rules:

```
.claude/rules/
  csharp.md        # Only loads for .cs files
  testing.md       # Only loads for test files
  api.md           # Only loads for controller files
  infrastructure.md # Only loads for IaC files
```

Each rule file is small (200-500 tokens). Only the relevant rules load per session. A test-writing session loads `testing.md` but not `infrastructure.md`. An API implementation session loads `api.md` and `csharp.md` but not `testing.md`.

The savings compound. Instead of loading 2,000 tokens of context every session, you load 400-800 tokens of relevant context. Over hundreds of sessions per month, this adds up.

Trim Your Prompt Engineering

Verbose prompts cost money. "I would like you to please create a comprehensive service class that handles all aspects of..." is burning tokens on filler. "Create `OrderService` with these methods: ..." is cheaper and more effective.

This isn't about being terse. It's about being precise. Every word in a prompt should carry information. If removing a word doesn't change the output, remove it.

Model Selection Strategy

Not every task needs the most expensive model. This is the single biggest lever most developers ignore.

The Three-Tier Approach

Tier 1: Cheap and fast (Haiku, GPT-4o-mini) - Code formatting and linting fixes - Simple boilerplate generation - Documentation generation - Test scaffolding - Commit message generation - Context file maintenance

Cost: $0.80-$1.50 per million input tokens. Use this for 60-70% of your agent interactions.

Tier 2: Capable and balanced (Sonnet, GPT-4o) - Feature implementation from specs - Bug diagnosis and fixing - Code review assistance - Refactoring - CI failure analysis

Cost: $2.50-$3 per million input tokens. Use this for 25-35% of your interactions.

Tier 3: Maximum capability (Opus, GPT-5, o3) - Complex architectural decisions - Large-scale refactoring across many files - Debugging subtle, multi-component issues - Performance optimization - Security review

Cost: $10-$15 per million input tokens. Use this for 5-10% of your interactions, the problems that actually need it.

Automatic Model Routing

Some tools support automatic model selection based on task complexity. If yours doesn't, build the habit manually. Before launching an agent, ask: "Does this task need the expensive model?" If the answer is "probably not," use Tier 1 or Tier 2.

The difference in monthly cost between "always use Opus" and "use Opus only when needed" can be 5-10x. For a team of 10, that's the difference between a $500/month AI budget and a $5,000/month AI budget.

Cost Tracking and Budgeting for Teams

If you're a team lead or engineering manager, you need visibility into AI spending. Without tracking, costs creep up invisibly until someone notices the credit card bill.

What to Track

- **Per-developer spending:** Who's using the most tokens? Are they getting proportional value?
- **Per-task-type spending:** How much does CI/CD automation cost vs. feature development vs. code review?
- **Model usage distribution:** Are developers defaulting to expensive models for simple tasks?
- **Context file sizes:** Are they growing over time? (They usually are.)

- **Cost per PR:** Divide total AI spend by PRs merged. This gives you a rough cost-per-unit-of-output.

Setting Budgets

A reasonable starting budget for a team adopting agentic development:

- **Subscriptions:** $40-$60/developer/month (two tool subscriptions)
- **API tokens:** $20-$50/developer/month (for CLI agents and CI/CD)
- **Total:** $60-$110/developer/month

That's $720-$1,320/developer/year. For context, a developer's fully loaded cost is typically $150,000-$250,000/year. If AI tools provide even a 10% productivity improvement, the ROI is clear: $15,000-$25,000 of additional output for $720-$1,320 of tooling cost.

The math is overwhelmingly in favor of the tools. The question isn't whether to spend, it's whether you're spending efficiently.

ROI Calculation: Justifying the Spend

If you need to justify AI tooling spend to management (or to yourself), here's a framework.

The Conservative Calculation

Assume a modest 15% productivity improvement (well below the 26% GitHub measured, but accounting for the METR study's finding that naive usage can slow you down).

```
Developer cost:           $200,000/year (fully loaded)
15% productivity gain:    $30,000/year equivalent output
AI tooling cost:          $1,200/year ($100/month)
Net ROI:                  $28,800/year per developer
ROI percentage:           2,400%
```

Even if you cut the productivity gain to 5% (extremely conservative), the ROI is still 733%. The tooling cost is simply too small relative to developer compensation for the math to not work out.

The Honest Calculation

The conservative calculation assumes productivity gains are uniform. They’re not. Some tasks see 50%+ improvement (boilerplate, testing, documentation). Others see 0% or negative improvement (novel architecture, deep debugging, creative design).

A more honest calculation:

```
Tasks that benefit from agents:        60% of developer time
Average improvement on those tasks:    25%
Effective overall improvement:         15%
Tasks that don't benefit:              30% of developer time
Tasks where agents slow you down:      10% of developer time
Average slowdown:                      -10%
Net effective improvement:             14%
```

Even this more nuanced calculation produces massive ROI. The tools pay for themselves many times over, as long as you’re using them on the right tasks and not forcing them where they don’t help.

Tips for Reducing Costs Without Reducing Quality

1. Cache Your Context

If your agent tool supports it, cache compiled context between sessions. Some tools (Claude Code with its session memory, Cursor with its indexed codebase) do this automatically. The first session loads everything; subsequent sessions reuse the cache.

2. Batch Small Tasks

Instead of 10 separate agent sessions for 10 small fixes, batch them: “Fix all 10 items in this list: [list].” One session with one context load instead of ten sessions with ten context loads.

3. Write Better Specs

The tightest spec produces the fewest iterations. Every round of “that’s not what I meant, try again” costs tokens. Invest 5 minutes in a clear spec and save $5 in wasted iterations.

4. Use Checkpoints

For long-running tasks, use agents that support checkpointing. If the agent goes off-track at step 7 of 10, you can roll back to step 6 instead of starting over. Some tools support this natively; others require you to commit intermediate results to Git.

5. Kill Bad Sessions Early

If an agent is clearly going in the wrong direction, stop it. Don’t let it burn tokens trying to fix a fundamentally wrong approach. Reset, refine the spec, restart. The sunk cost fallacy applies to token spend too.

6. Monitor and Alert

Set up spending alerts with your API provider. Anthropic, OpenAI, and others support budget limits and email notifications. A runaway agent (stuck in a retry loop, for example) can burn through a month's budget in hours.

7. Compress Context Aggressively

Review your context files monthly. Remove outdated conventions. Consolidate redundant rules. Shorten verbose explanations. Every token you trim saves money on every future session.

8. Use the Right Tool for the Job

Don't use Claude Opus to generate a `.gitignore` file. Don't use a CI/CD agent for a one-time script. Match the tool to the task. This sounds obvious, but developers consistently default to their most powerful (and expensive) tool for everything.

The Team Economics

For engineering teams, the economics of agentic development extend beyond tool costs. Consider the second-order effects:

Reduced context-switching cost: When an agent handles CI failures, developers stay focused on their primary tasks. Context-switching costs are estimated at 15-20 minutes per interruption. Eliminating 2-3 CI-related interruptions per day saves 30-60 minutes per developer.

Reduced onboarding time: New developers with agent support can explore and understand codebases faster. The "ask the agent about the codebase" workflow (Chapter 18's codebase questions category) replaces hours of reading code or asking senior developers for explanations.

Reduced technical debt accumulation: Background agents can chip away at tech debt overnight (Chapter 19). Tasks that never make it to the top of a human's priority list can be assigned to agents during off-hours.

Increased deployment frequency: Agent-integrated CI/CD (Chapter 21) reduces the time from “something broke” to “it’s fixed.” Faster recovery means more confidence in deploying, which means more frequent deployments, which means faster feedback loops.

These second-order effects are harder to measure but often more valuable than the direct productivity gains. A team that deploys twice as often, onboards developers in half the time, and accumulates less tech debt is a fundamentally more effective team, regardless of what the line-item AI budget looks like.

The Bottom Line

AI tooling costs money. The costs are real, visible, and growing as you adopt more tools and more sophisticated workflows. Ignoring costs leads to waste. Obsessing over costs leads to under-utilization.

The right approach is informed management: know what you’re spending, know what you’re getting, and continuously optimize the ratio. Track costs per developer, per task type, and per model. Use cheap models for cheap tasks. Write tight specs and tight context files. Monitor for waste.

And when someone asks “is this worth the money?”, the answer is almost certainly yes. A $100/month tool that makes a $200,000/year developer 15% more effective pays for itself in the first two days of every month. The remaining 28 days are pure upside.

The real question isn’t whether to spend. It’s whether you’re spending wisely. This chapter gave you the framework to answer that question. Now go audit your context files. You’re probably burning tokens on instructions nobody reads.

PART VI

.NET-Specific Patterns

CHAPTER 23

Context Files for .NET

Why .NET Needs Specific Context

C# is a language that evolves fast. Between nullable reference types, file-scoped namespaces, primary constructors, global usings, and collection expressions, a codebase written in 2024 looks dramatically different from one written in 2021. Agents trained on years of Stack Overflow and GitHub data will happily generate code in any of these styles, often mixing them within the same file.

That's the problem. Without explicit guidance, your agent will:

- Use block-scoped namespaces in a project that uses file-scoped
- Ignore nullable reference types and litter your code with `null!` suppressions
- Use traditional constructors when you've adopted primary constructors
- Add `using` statements that your global usings already cover
- Generate `var` everywhere or nowhere, depending on which training data dominates

A good .NET context file eliminates all of this. One file, loaded automatically, and every agent interaction produces code that matches your project.

The .NET Conventions Section

Every .NET context file needs a section that locks down the language-level decisions. Here's the baseline:

```
## C# Conventions
- Target: .NET 9 / C# 13
- Use file-scoped namespaces (one namespace per file, no braces)
- Use primary constructors for dependency injection
- Enable nullable reference types (already in .csproj). Never use null! suppression.
- Use global usings (see GlobalUsings.cs). Don't add usings that are already global.
- Use collection expressions: [] not new List<T>(), [..existing] for spread
- Use raw string literals for multi-line strings (three quotes)
- Use pattern matching over type checks and casts
- Prefer records for immutable data types and DTOs
- Use required keyword for properties that must be set at construction
- Use init-only setters for immutable properties on classes
```

This section alone prevents 80% of the style inconsistencies agents introduce. Let's look at what each of these buys you.

Nullable Reference Types

This is the single most common source of agent-generated warnings. Agents will write code like:

```
public class OrderService
{
    private readonly IOrderRepository _repository; // CS8618 warning

    public OrderService(IOrderRepository repository)
    {
        _repository = repository;
    }
}
```

When your project uses primary constructors, the agent should write:

```
public class OrderService(IOrderRepository repository)
{
    public async Task<Order?> GetOrderAsync(int id) =>
        await repository.GetByIdAsync(id);
}
```

No field declarations. No null warnings. Cleaner code. But the agent won't do this unless you tell it to.

Global Usings

Every .NET project accumulates a set of namespaces used in nearly every file. If your `GlobalUsings.cs` already includes `System.Text.Json` and `Microsoft.EntityFrameworkCore`, an agent adding those usings to individual files creates noise. Call out your global usings explicitly:

```
## Global Usings (already configured, don't re-add)
- System.Text.Json
- Microsoft.EntityFrameworkCore
- Microsoft.AspNetCore.Mvc
- FluentValidation
- MediatR
```

Primary Constructors

Primary constructors are the clearest example of a feature where agents need explicit direction. The syntax is still new enough that many training examples use the old pattern. Be specific:

```
## Dependency Injection
Use primary constructors for all DI. Do NOT create private readonly fields manually.

Yes:
```csharp
public class OrderHandler(IOrderRepository repository, ILogger<OrderHandler> logger)
```
```

No:

```
public class OrderHandler
{
    private readonly IOrderRepository _repository;
    private readonly ILogger<OrderHandler> _logger;

    public OrderHandler(IOrderRepository repository, ILogger<OrderHandler> logger)
    {
        _repository = repository;
        _logger = logger;
    }
}
```

```
## ASP.NET Core Minimal API Template

Minimal APIs are the default for new ASP.NET Core projects. Here's a complete AGENTS.md templat

```markdown
# AGENTS.md

## Project Overview
REST API for [describe your domain]. Built with ASP.NET Core Minimal APIs.

## Tech Stack
- .NET 9 / ASP.NET Core
- EF Core 9 with PostgreSQL (Npgsql)
- FluentValidation for request validation
- Scalar for API documentation (not Swagger/Swashbuckle)
- xUnit + FluentAssertions + NSubstitute for testing
- Docker Compose for local dependencies

## Build & Run
```bash
dotnet build
dotnet test
dotnet run --project src/Api
```
```
```

Project Structure

```
src/Api/
  Features/
    Orders/
      CreateOrder.cs        # Endpoint + handler + request/response
      GetOrder.cs
      ListOrders.cs
    Products/
      ...
  Data/
    AppDbContext.cs
    Migrations/
  Common/
    Extensions/
    Middleware/
```

C# Conventions

- File-scoped namespaces
- Primary constructors for DI
- Nullable reference types enabled, no null! suppression
- Records for request/response DTOs
- Collection expressions where applicable

API Patterns

- One file per endpoint: request record, handler, endpoint mapping
- Group endpoints with MapGroup() and extension methods

- Return TypedResults (Results.Ok(), Results.NotFound(), etc.)
- Use .WithName(), .WithTags(), .Produces() for OpenAPI metadata
- Validate requests with FluentValidation endpoint filters
- All routes: /api/v1/{resource}

Endpoint Structure

Each endpoint file follows this pattern:

```
public static class CreateOrder
{
    public record Request(string CustomerName, List<OrderItem> Items);
    public record Response(int Id, string Status);

    public static void Map(RouteGroupBuilder group) =>
        group.MapPost("/", HandleAsync)
            .WithName(nameof(CreateOrder))
            .Produces<Response>(StatusCodes.Status201Created);

    private static async Task<IResult> HandleAsync(
        Request request,
        AppDbContext db,
        CancellationToken ct)
    {
        // implementation
        return TypedResults.Created($"/api/v1/orders/{order.Id}", new Response(order.Id, order.
    }
}
```

EF Core

- DbContext: AppDbContext in Data/ folder
- Entity configuration: IEntityTypeConfiguration in separate files
- Migrations via CLI: dotnet ef migrations add –project src/Api
- No lazy loading. Use explicit .Include() for related data.
- Use DateTimeOffset, not DateTime, for all timestamps

Testing

- Integration tests with WebApplicationFactory
- Use Testcontainers for PostgreSQL in tests
- Name: MethodName_Scenario_ExpectedResult
- Builder pattern for test data (see tests/Builders/)

What NOT to Do

- Do NOT use controllers (this is a Minimal API project)
- Do NOT use MediatR (endpoints ARE the handlers)
- Do NOT add Swashbuckle (we use Scalar)
- Do NOT use AutoMapper
- Do NOT modify Program.cs service registration without asking

ASP.NET Core MVC Template

For projects that use controllers (plenty of them still exist, and for good reason):

```markdown
# AGENTS.md

## Project Overview
Web application with API and MVC views for [describe domain].

## Tech Stack
- .NET 9 / ASP.NET Core MVC
- EF Core 9 with SQL Server
- ASP.NET Identity for authentication
- xUnit + WebApplicationFactory for testing

## C# Conventions
- File-scoped namespaces, primary constructors
- Nullable reference types enabled

## Controller Patterns
- One controller per resource/feature area
- Inherit from ControllerBase for APIs, Controller for MVC
- Use [ApiController] attribute on API controllers
- Use [FromRoute], [FromQuery], [FromBody] explicitly
- Return ActionResult<T> from API endpoints
- Keep controllers thin: delegate to services

## Service Layer
- One interface + implementation per feature area
- Register as scoped in DI
- Services handle business logic, controllers handle HTTP concerns
- Return Result<T> from services, not exceptions

## Views (MVC)
```

```
- Use Tag Helpers, not HTML helpers
- Partial views in Shared/ for reusable components
- ViewModels for every view (no passing entities to views)
- Client-side validation with jQuery Validation (already configured)
```

Blazor Template

Blazor projects need special attention because agents frequently confuse Blazor Server, Blazor WebAssembly, and the newer Blazor Web App model:

```
# AGENTS.md

## Project Overview
Blazor Web App with interactive server rendering for [describe domain].

## Tech Stack
- .NET 9 / Blazor Web App (Interactive Server)
- EF Core 9 with SQLite (development), PostgreSQL (production)
- MudBlazor component library
- bUnit for component testing

## Rendering Model
This is a Blazor Web App using Interactive Server rendering.
- Default render mode: InteractiveServer (set in App.razor)
- Static SSR for pages that don't need interactivity
- Do NOT use InteractiveWebAssembly or InteractiveAuto (we don't have a Client project)

## Component Patterns
- One component per file, file-scoped namespace
- Use @inject for DI, not [Inject] attribute
- Use CascadingParameter for shared state (auth, theme)
- Handle async loading with a loading state pattern:
```razor
@if (_items is null)
{
    <MudProgressCircular />
}
else
{
    @foreach (var item in _items) { ... }
}
```
```

- Dispose services properly: implement IAsyncDisposable when using timers or event subscriptions
- Use EventCallback for child-to-parent communication

State Management

- Scoped services for per-circuit state
- No Fluxor or other state management libraries
- Use cascading values for cross-component state

What NOT to Do

- Do NOT use JSInterop unless absolutely necessary
- Do NOT create WASM components (this is server-only)
- Do NOT use NavigationManager.NavigateTo() for same-page updates
- Do NOT put business logic in components (use services)

Class Library Template

For shared libraries and NuGet packages:

```markdown
# AGENTS.md

## Project Overview
Shared library providing [describe functionality]. Published as NuGet package.

## Tech Stack
- .NET 9 class library (targets net9.0)
- No framework dependencies (this is a pure library)

## Conventions
- File-scoped namespaces
- Primary constructors where appropriate
- XML documentation comments on all public members
- Internal by default. Only make types public if they're part of the API surface.
- Seal classes that aren't designed for inheritance

## Public API Design
- Use interfaces for extensibility points
- Use abstract base classes only when shared implementation is needed
- Provide extension methods for DI registration: services.Add{LibraryName}()
- Follow Microsoft's API design guidelines
- Include [Obsolete] attributes before removing public members

## Testing
- xUnit + FluentAssertions
- Test public API surface only
- 90%+ coverage on core logic

## What NOT to Do
- Do NOT add ASP.NET Core dependencies
```

```
- Do NOT add EF Core dependencies (this is a pure library)
- Do NOT use static state
- Do NOT throw exceptions for expected conditions (use Result<T>)
```

MAUI Template

.NET MAUI projects have a unique structure and agents need to understand the cross-platform constraints:

```
# AGENTS.md

## Project Overview
Cross-platform mobile/desktop app for [describe domain]. Targets Android and iOS.

## Tech Stack
- .NET 9 / .NET MAUI
- CommunityToolkit.Mvvm for MVVM pattern
- CommunityToolkit.Maui for UI helpers
- SQLite-net for local storage
- xUnit + device runners for testing

## Architecture
- MVVM pattern with CommunityToolkit.Mvvm source generators
- ViewModels use [ObservableProperty] and [RelayCommand] attributes
- Services registered in MauiProgram.cs
- Shell-based navigation

## Conventions
- One ViewModel per page
- Use [ObservableProperty] instead of manual INotifyPropertyChanged
- Use [RelayCommand] instead of manual ICommand implementations
- Async commands for all I/O operations
- Platform-specific code in Platforms/ folders only

## Navigation
- Use Shell routing: Shell.Current.GoToAsync()
- Register routes in AppShell.xaml.cs
- Pass parameters with query properties: [QueryProperty]

## What NOT to Do
- Do NOT use code-behind for business logic (keep XAML code-behind minimal)
- Do NOT use Device.BeginInvokeOnMainThread (use MainThread.InvokeOnMainThreadAsync)
- Do NOT add platform-specific code outside Platforms/ folders
- Do NOT use absolute layouts (use Grid or FlexLayout for responsive design)
```

Architecture-Specific Templates

Beyond project type, your architecture choice dramatically affects what agents need to know. Here are the additions for common .NET architectures.

Clean Architecture Addition

```
## Architecture: Clean Architecture
```

src/ Domain/ # Entities, value objects, domain events, interfaces Application/ # Use cases, DTOs, validators, interfaces for infra Infrastructure/ # EF Core, external APIs, file system, email Api/ # Controllers/endpoints, middleware, DI composition

```


### Dependency Rules (NEVER VIOLATE)
- Domain depends on nothing
- Application depends only on Domain
- Infrastructure depends on Application and Domain
- Api depends on all layers but only for DI composition
- NEVER reference Infrastructure from Application

### Where Things Go
- Business rules: Domain entities and domain services
- Use cases/orchestration: Application handlers
- Data access: Infrastructure repositories
- HTTP concerns: Api layer only
```

Vertical Slice Architecture Addition

```
## Architecture: Vertical Slice
Each feature is a self-contained folder with everything it needs.
```

src/Api/Features/ Orders/ CreateOrder.cs # Command + Handler + Validator + Endpoint GetOrder.cs # Query + Handler + Endpoint OrderEntity.cs # Entity (if feature-specific) OrderMappings.cs # EF Core entity configuration

```
### Rules
- Features don't reference other features directly
- Shared code goes in Common/ only if used by 3+ features
- Each file can contain the request, handler, and endpoint together
- Don't over-abstract: a little duplication between features is OK
```

EF Core Conventions

EF Core deserves its own section in every .NET context file. Agents make consistent mistakes with EF Core if you don’t specify your patterns:

```
## EF Core Conventions

### DbContext
- One DbContext for the application: AppDbContext
- Entity configurations in separate files: Data/Configurations/{Entity}Configuration.cs
- Use IEntityTypeConfiguration<T>, not Fluent API in OnModelCreating

### Entity Patterns
- Id property is always int (auto-increment) unless domain requires otherwise
- Use DateTimeOffset for all timestamps (CreatedAt, UpdatedAt)
- Soft delete: IsDeleted boolean + global query filter
- Owned types for value objects (Address, Money, etc.)

### Configuration Pattern
```csharp
public class OrderConfiguration : IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> builder)
    {
        builder.HasKey(o => o.Id);
        builder.Property(o => o.CustomerName).HasMaxLength(200).IsRequired();
        builder.HasMany(o => o.Items).WithOne().HasForeignKey(i => i.OrderId);
        builder.HasIndex(o => o.CreatedAt);
    }
}
```

Migrations

- Name migrations descriptively: Add_Order_CustomerName_Index
- Never edit existing migrations (create new ones)
- Run migrations: dotnet ef migrations add –project src/Api
- Test migrations against a real database before merging
```

What NOT to Do with EF Core

- Do NOT use lazy loading
- Do NOT call SaveChanges() in repository methods (call it in the handler/service)
- Do NOT use .Find() when you need to include related data (use .Include().FirstOrDefaultAsync())
- Do NOT use string-based Include() overloads
- Do NOT create circular navigation properties

```markdown
## Testing Conventions

Testing is where agents can save you enormous time, or create a mess. Be specific:

```markdown
## Testing

### Frameworks
- xUnit for test runner
- FluentAssertions for assertions
- NSubstitute for mocking
- WebApplicationFactory for integration tests
- Testcontainers for database tests
- Bogus for test data generation

### Unit Test Pattern
```csharp
public class CreateOrderHandlerTests
{
    private readonly IOrderRepository _repository = Substitute.For<IOrderRepository>();
    private readonly CreateOrderHandler _sut;

    public CreateOrderHandlerTests()
    {
        _sut = new CreateOrderHandler(_repository);
    }

    [Fact]
    public async Task Handle_ValidOrder_ReturnsCreatedOrder()
    {
        // Arrange
        var command = new CreateOrderCommand("John Doe", [new OrderItem("Widget", 2)]);
        _repository.AddAsync(Arg.Any<Order>()).Returns(new Order { Id = 1 });

        // Act
```

```
        var result = await _sut.Handle(command, CancellationToken.None);

        // Assert
        result.Should().NotBeNull();
        result.Id.Should().Be(1);
        await _repository.Received(1).AddAsync(Arg.Any<Order>());
    }
}
```

Integration Test Pattern

```
public class CreateOrderEndpointTests(WebApplicationFactory<Program> factory)
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client = factory.CreateClient();

    [Fact]
    public async Task CreateOrder_ValidRequest_Returns201()
    {
        // Arrange
        var request = new { CustomerName = "John", Items = new[] { new { Name = "Widget", Quant

        // Act
        var response = await _client.PostAsJsonAsync("/api/v1/orders", request);

        // Assert
        response.StatusCode.Should().Be(HttpStatusCode.Created);
        response.Headers.Location.Should().NotBeNull();
    }
}
```

Naming Convention

- MethodName_Scenario_ExpectedResult

- Example: Handle_EmptyOrderItems_ReturnsValidationError

What NOT to Do in Tests

- Do NOT use Thread.Sleep (use async delays if needed, but prefer event-based waiting)
- Do NOT share state between tests (each test is independent)
- Do NOT mock what you don't own (wrap external libraries behind interfaces)
- Do NOT test private methods (test through the public API) ```

Putting It All Together

The key insight is this: a generic AGENTS.md helps, but a .NET-specific one transforms your workflow. The templates above aren't theoretical. They're distilled from real projects where agents went from producing code that needed heavy editing to producing code that slotted in cleanly.

Start with the template closest to your project type. Copy it. Customize the sections that differ from your conventions. Remove anything that doesn't apply. Then use it for a week and iterate.

The best context files aren't written in one sitting. They grow. Every time an agent makes a mistake that your context file should have prevented, add a line. Every time you correct an agent's output, ask yourself: "Could I have prevented this with a convention in my AGENTS.md?" Usually, the answer is yes.

Your context file is a living document. Treat it like code: version it, review it, improve it. It's the single highest-leverage investment you can make in your agentic .NET development workflow.

CHAPTER 24

Agents and .NET Aspire

What .NET Aspire Actually Is

.NET Aspire is an opinionated stack for building cloud-native, distributed applications in .NET. If that sounds like marketing, here's what it means in practice: Aspire gives you a single project (the AppHost) that wires together all your services, databases, caches, and message brokers. You run one project and everything starts. You get a dashboard that shows logs, traces, and metrics across all services. You get service discovery so services find each other by name instead of hardcoded URLs.

For solo developers and small teams, Aspire eliminates the "how do I run all this locally" problem. For agent-assisted development, it eliminates something even more important: the gap between "the agent built a service" and "I can actually see if it works."

Why Aspire Matters for Agentic Development

When an agent creates a new service or modifies an existing one, you need to verify the work. Without Aspire, that means:

1. Figuring out which services to start
2. Making sure dependencies (databases, caches) are running
3. Checking logs across multiple terminal windows
4. Manually testing HTTP endpoints
5. Hoping the service discovery configuration is correct

With Aspire, you run `dotnet run --project src/AppHost` and everything comes up. The dashboard shows you every service, every log line, every HTTP trace. You can see immediately if the agent's code works, throws errors, or has performance issues.

This is the real value proposition: Aspire compresses the feedback loop between agent output and developer verification.

The Aspire Project Structure

An Aspire solution has a predictable structure that agents need to understand:

```
src/
  AppHost/                    # The orchestrator (you run this)
    Program.cs                # Wires everything together
    appsettings.json
  ServiceDefaults/            # Shared configuration for all services
    Extensions.cs             # OpenTelemetry, health checks, resilience
  Api/                        # Your API service
    Program.cs
  Worker/                     # Background worker service
    Program.cs
  Web/                        # Blazor frontend
    Program.cs
```

The AppHost is the entry point. It doesn't serve traffic. It orchestrates: it starts your services, provisions containers for dependencies, and configures service discovery. The ServiceDefaults project provides shared setup that every service references (telemetry, health checks, resilience policies).

The AppHost: Your Orchestration File

Here's what a typical AppHost Program.cs looks like:

```
var builder = DistributedApplication.CreateBuilder(args);

var postgres = builder.AddPostgres("postgres")
    .WithPgAdmin();

var ordersDb = postgres.AddDatabase("ordersdb");

var redis = builder.AddRedis("cache")
    .WithRedisInsight();

var messaging = builder.AddRabbitMQ("messaging")
    .WithManagementPlugin();

var api = builder.AddProject<Projects.Api>("api")
    .WithReference(ordersDb)
    .WithReference(redis)
    .WithReference(messaging);

builder.AddProject<Projects.Worker>("worker")
    .WithReference(ordersDb)
    .WithReference(messaging);

builder.AddProject<Projects.Web>("web")
    .WithReference(api);

builder.Build().Run();
```

This is the file agents need to modify when adding new services or dependencies. It's also the file agents most commonly get wrong.

The Aspire + Agent Workflow

Here's the workflow I recommend for building services with Aspire and coding agents:

Step 1: Scaffold with Aspire CLI

Never let agents create Aspire projects from scratch. Use the CLI:

```
dotnet new aspire-starter -n MyApp
```

Or for a blank slate:

```
dotnet new aspire -n MyApp
```

This gives you the correct project structure with AppHost, ServiceDefaults, and proper project references. Agents can add to this, but they shouldn't create it.

Step 2: Define the Service in AppHost First

Before asking an agent to build a service, define it in the AppHost. This is your spec:

```
var catalogDb = postgres.AddDatabase("catalogdb");

var catalogApi = builder.AddProject<Projects.CatalogApi>("catalog-api")
    .WithReference(catalogDb)
    .WithReference(redis);
```

Now the agent knows: build a project called CatalogApi that uses a PostgreSQL database called catalogdb and a Redis cache.

Step 3: Agent Builds the Service

Give the agent a prompt like:

> *"Create the CatalogApi project. It's registered in the AppHost with a PostgreSQL database (catalogdb) and Redis cache. Build CRUD endpoints for a Product entity with fields: Name (string, required, max 200), Price (decimal), Category (string). Follow the conventions in AGENTS.md."*

The agent creates the project, adds endpoints, sets up EF Core, and configures caching.

Step 4: Verify with the Dashboard

Run the AppHost:

```
dotnet run --project src/AppHost
```

Open the Aspire dashboard (usually at https://localhost:17027). You'll see:

- All services and their status (running, stopped, error)
- Logs from every service in one place
- Distributed traces showing HTTP calls between services
- Metrics for request rates, error rates, response times
- Health check status

If the agent's code has issues, you'll see them immediately. Red status indicators, error logs, failed health checks. No hunting through terminal windows.

Step 5: Iterate

If something's wrong, give the agent the error from the dashboard. Aspire's structured logging and distributed tracing give you precise error context that makes agent debugging dramatically faster.

Service Discovery: What Agents Need to Know

Aspire uses named references for service discovery. When you write `.WithReference(api)` in the AppHost, the consuming service can access the API using the name "api" instead of a hardcoded URL.

In the consuming service:

```
builder.Services.AddHttpClient<CatalogApiClient>(client =>
{
    client.BaseAddress = new Uri("https+http://catalog-api");
});
```

The `https+http://` scheme tells the service discovery system to resolve the address at runtime. Agents frequently get this wrong in two ways:

1. They hardcode `localhost:5001` instead of using the service name
2. They forget the `https+http://` scheme prefix

Your context file should call this out explicitly.

Configuration and Connection Strings

Aspire manages connection strings through the AppHost. When you write `.WithReference(ordersDb)`, Aspire injects a connection string into the consuming service's configuration. The service accesses it by name:

```
builder.AddNpgsqlDbContext<AppDbContext>("ordersdb");
```

Or for Redis:

```
builder.AddRedisDistributedCache("cache");
```

The name in `AddNpgsqlDbContext("ordersdb")` must match the name in `postgres.AddDatabase("ordersdb")`. Agents sometimes invent their own names. Your context file should list all resource names and their expected consumers.

Health Checks and Resilience

The ServiceDefaults project configures health checks and resilience policies for all services. Agents need to know this exists so they don't duplicate the configuration:

```
// In ServiceDefaults/Extensions.cs (already configured)
public static IHostApplicationBuilder AddServiceDefaults(this IHostApplicationBuilder builder)
{
    builder.ConfigureOpenTelemetry();
    builder.AddDefaultHealthChecks();
    builder.Services.AddServiceDiscovery();
    builder.Services.ConfigureHttpClientDefaults(http ⇒
    {
        http.AddStandardResilienceHandler();
        http.AddServiceDiscovery();
    });
    return builder;
}
```

Every new service should call `builder.AddServiceDefaults()` at the top of its `Program.cs`. Agents need to know this is required, not optional.

Context File Additions for Aspire Projects

Add these sections to your AGENTS.md for Aspire projects:

```
## .NET Aspire

### Project Structure
- AppHost: src/AppHost/ (orchestrator, run this to start everything)
- ServiceDefaults: src/ServiceDefaults/ (shared config, DO NOT MODIFY without asking)
- Services: src/{ServiceName}/ (individual services)

### Creating New Services
1. Create the project: dotnet new webapi -n {ServiceName} -o src/{ServiceName}
2. Add reference to ServiceDefaults: dotnet add src/{ServiceName} reference src/ServiceDefaults
3. Register in AppHost Program.cs with appropriate resource references
4. Call builder.AddServiceDefaults() at the top of the service's Program.cs
5. launchSettings.json is required for every service project

### Resource Names (must match exactly)
- "ordersdb" - PostgreSQL database for orders
- "cache" - Redis cache
- "messaging" - RabbitMQ message broker

### Service Discovery
- Use named references, never hardcoded URLs
- HTTP clients: new Uri("https+http://{service-name}")
- Connection strings: resolved by resource name

### What NOT to Do
- Do NOT modify ServiceDefaults without asking
- Do NOT hardcode connection strings (Aspire manages them)
- Do NOT hardcode service URLs (use service discovery)
- Do NOT remove launchSettings.json from any project
- Do NOT create Aspire projects by hand (use dotnet new aspire or aspire new)
```

Using the Dashboard for Agent Verification

The Aspire dashboard is your primary verification tool for agent-built services. Here's how to use it effectively.

Structured Logs

Click on any service to see its logs. When an agent adds a new endpoint and you test it, the logs show you exactly what happened: request received, database query executed, response sent. If the agent forgot to add error handling, you'll see unhandled exceptions in red.

Distributed Traces

When a request flows through multiple services (Web calls API, API queries database), the trace view shows the entire chain. This is invaluable for verifying that the agent wired up service communication correctly.

Click a trace to see each span: HTTP request, database query, cache lookup. You can see timing, status codes, and any errors. If the agent's code makes an N+1 query, you'll see it as dozens of database spans where there should be one.

Metrics

The metrics tab shows request rates, error rates, and response time percentiles. After an agent builds a new endpoint, hit it with a few requests and check: Are response times reasonable? Any errors? Is the database connection pool behaving?

Health Checks

The dashboard shows health check status for every service. Green means healthy, red means something's wrong. If the agent's code has a misconfigured database connection or a broken dependency, the health check catches it before you manually test.

Real Example: Building a Notification Service with Aspire + Agents

Let's walk through a complete example. You have an existing Aspire solution with an Orders API. You need a Notification service that sends emails when orders are placed.

Define in AppHost First

```
var smtp = builder.AddMailDev("smtp");

var notifications = builder.AddProject<Projects.NotificationService>("notifications")
    .WithReference(messaging)
    .WithReference(smtp);
```

You've declared: the Notification service needs RabbitMQ (for receiving order events) and an SMTP server (for sending emails). MailDev gives you a local SMTP server with a web UI for testing.

Prompt the Agent

> *"Create the NotificationService project. It consumes OrderPlaced messages from RabbitMQ (via MassTransit) and sends email notifications via SMTP. Use the resource references already defined in AppHost: 'messaging' for RabbitMQ, 'smtp' for mail. Follow AGENTS.md conventions."*

What the Agent Builds

A well-configured agent produces:

```
// src/NotificationService/Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.AddServiceDefaults();

builder.Services.AddMassTransit(x =>
{
    x.AddConsumer<OrderPlacedConsumer>();
    x.UsingRabbitMq((context, cfg) =>
    {
        cfg.Host(builder.Configuration.GetConnectionString("messaging"));
        cfg.ConfigureEndpoints(context);
    });
});

builder.Services.AddFluentEmail("noreply@myapp.com")
    .AddSmtpSender(builder.Configuration.GetConnectionString("smtp"));

var app = builder.Build();
app.MapDefaultEndpoints();
app.Run();
```

```
// src/NotificationService/Consumers/OrderPlacedConsumer.cs
namespace NotificationService.Consumers;

public class OrderPlacedConsumer(IFluentEmail email, ILogger<OrderPlacedConsumer> logger)
    : IConsumer<OrderPlaced>
{
    public async Task Consume(ConsumeContext<OrderPlaced> context)
    {
        var order = context.Message;
        logger.LogInformation("Sending notification for order {OrderId}", order.Id);

        await email
            .To(order.CustomerEmail)
            .Subject($"Order {order.Id} Confirmed")
            .Body($"Your order for {order.ItemCount} items has been confirmed.")
            .SendAsync();
    }
}
```

Verify in Dashboard

Run the AppHost. The dashboard shows five services now: API, Worker, Web, NotificationService, and MailDev. All green. Place an order through the API. The distributed trace shows:

1. POST /api/v1/orders hits the API
2. API publishes OrderPlaced to RabbitMQ
3. NotificationService consumes the message
4. SMTP call to MailDev

Open MailDev's web UI (linked from the dashboard) and confirm the email arrived. The entire verification took 30 seconds without writing a single test.

Common Agent Mistakes with Aspire

After months of building Aspire projects with agents, these are the recurring issues:

Missing ServiceDefaults reference. The agent creates a new project but forgets to add the ServiceDefaults project reference. Result: no telemetry, no health checks, no service discovery. Your context file must state this is required.

Hardcoded connection strings. The agent puts `"Host=localhost;Port=5432;Database=mydb"` in appsettings.json instead of using the Aspire-injected connection string. Everything works when you run standalone, breaks when you run through AppHost.

Missing launchSettings.json. Aspire AppHost needs launchSettings.json in each service project to set environment variables. Agents sometimes delete or skip this file, causing dashboard environment variable crashes.

Wrong Aspire NuGet packages. Aspire has specific client packages (like `Aspire.Npgsql.EntityFrameworkCore` instead of plain `Npgsql.EntityFrameworkCore.PostgreSQL`). The Aspire packages integrate with the dashboard, provide health checks, and work with service discovery. Agents use the non-Aspire versions unless told otherwise.

Not calling MapDefaultEndpoints(). The ServiceDefaults extension adds health check and alive endpoints. If the service doesn't call `app.MapDefaultEndpoints()`, the dashboard can't monitor its health.

Add all of these to your "What NOT to Do" section. Every one of them will save you debugging time.

The Aspire Advantage

.NET Aspire isn't just a development convenience. It's an observability layer that makes agent-assisted development dramatically more productive. Without it, you're reviewing

code in a vacuum, hoping the agent got things right. With it, you're verifying behavior in real time across your entire distributed system.

Every .NET developer using agents to build cloud-native applications should be using Aspire. It's not optional. It's the difference between trusting agent output and verifying it.

CHAPTER 25

EF Core Migrations with Agents

Why Migrations Are Different

When an agent writes bad C# code, you catch it in code review or tests. When an agent writes a bad migration, the damage can be permanent. A dropped column, a bad data transform, an irreversible rename. Migrations touch your production data. There's no "undo" button.

That's not a reason to avoid using agents for migrations. It's a reason to have a disciplined workflow around it.

The Migration Safety Workflow

Every agent-generated migration should follow this sequence:

Generate → Review → Test → Apply

No shortcuts. No skipping steps. Here's each one in detail.

Step 1: Generate

The agent makes changes to your entity classes and then runs the migration command:

```
dotnet ef migrations add AddOrderShippingAddress --project src/Api
```

The agent should name migrations descriptively. Not `Migration1` or `Update`. Names should describe what changed: `AddOrderShippingAddress`, `CreateProductCategoryIndex`, `RenameCustomerEmailToContactEmail`.

Step 2: Review

This is non-negotiable. Read the generated migration file. Every line. The agent created it, but you own it. Look for:

- **Destructive operations**: `DropColumn`, `DropTable`, `DropIndex`. Are they intentional?
- **Data loss risks**: Changing a column type (string to int) can fail or lose data
- **Missing data migrations**: Renaming a column without migrating existing data
- **Performance concerns**: Adding an index on a table with millions of rows without `CONCURRENTLY`
- **Default values**: New non-nullable columns need defaults for existing rows

Step 3: Test

Run the migration against a real database. Not SQLite (unless that's your production database). The same engine you use in production:

```
# Reset and apply all migrations to a test database
dotnet ef database drop --force --project src/Api
dotnet ef database update --project src/Api
```

If you're using Testcontainers in your test suite, even better. Your integration tests can apply migrations automatically and verify they work.

Step 4: Apply

Only after review and testing. In a CI/CD pipeline, migrations should apply automatically during deployment. In development, `dotnet ef database update` does the job.

Teaching Agents to Generate Migrations

The most productive pattern is entity-first: you describe what you want, the agent modifies entity classes and configurations, then generates the migration.

Adding a New Entity

Prompt:

> *"Add a ShippingAddress entity with Street, City, State, ZipCode (all strings, required), and a foreign key to Order. One Order has one ShippingAddress. Generate the migration."*

A well-configured agent produces:

```
public class ShippingAddress
{
    public int Id { get; set; }
    public required string Street { get; set; }
    public required string City { get; set; }
    public required string State { get; set; }
    public required string ZipCode { get; set; }
    public int OrderId { get; set; }
    public Order Order { get; set; } = null!;
}
```

```
public class ShippingAddressConfiguration : IEntityTypeConfiguration<ShippingAddress>
{
    public void Configure(EntityTypeBuilder<ShippingAddress> builder)
    {
        builder.HasKey(s => s.Id);
        builder.Property(s => s.Street).HasMaxLength(200).IsRequired();
        builder.Property(s => s.City).HasMaxLength(100).IsRequired();
        builder.Property(s => s.State).HasMaxLength(50).IsRequired();
        builder.Property(s => s.ZipCode).HasMaxLength(20).IsRequired();
    }
}
```

Then adds the `DbSet` to the context and runs `dotnet ef migrations add AddShippingAddress`.

Adding Relationships

Relationships are where agents start making mistakes. The most common issue: creating bidirectional navigation properties when you only need one direction. Your context file should specify your preference:

```
## EF Core Relationships
- One-to-many: navigation collection on the parent, foreign key on the child
- One-to-one: navigation property on the dependent, foreign key on the dependent
- Many-to-many: explicit join entity (no implicit join tables)
- Only add navigation properties you actually query through
```

Adding Indexes

Agents rarely add indexes on their own. You need to ask for them explicitly:

> *"Add an index on Order.CreatedAt. We query orders by date range frequently."*

```
builder.HasIndex(o => o.CreatedAt);
```

For composite indexes:

> *"Add a composite index on Order.CustomerId and Order.Status."*

```
builder.HasIndex(o => new { o.CustomerId, o.Status });
```

For unique constraints:

```
builder.HasIndex(o => o.OrderNumber).IsUnique();
```

Seed Data Generation

Agents are excellent at generating seed data. This is one area where you can delegate freely:

> *"Generate seed data for 10 product categories with realistic names and descriptions. Use HasData in the entity configuration."*

```
public class CategoryConfiguration : IEntityTypeConfiguration<Category>
{
    public void Configure(EntityTypeBuilder<Category> builder)
    {
        builder.HasKey(c => c.Id);
        builder.Property(c => c.Name).HasMaxLength(100).IsRequired();

        builder.HasData(
            new Category { Id = 1, Name = "Electronics", Description = "Computers, phones, and
            new Category { Id = 2, Name = "Clothing", Description = "Apparel and fashion items"
            new Category { Id = 3, Name = "Home & Garden", Description = "Furniture, decor, and
            // ... more categories
        );
    }
}
```

Important: `HasData` requires explicit primary key values. Agents sometimes forget this and leave `Id = 0`, which fails silently or creates duplicates. Call this out in your context file:

```
## Seed Data Rules
- Always specify explicit Id values in HasData
- Start seed data Ids at 1 and increment sequentially
- Keep seed data in the entity's configuration file
- For large seed data sets, use a separate static class
```

Handling Complex Migrations

Some schema changes can’t be expressed as simple add/remove operations. These require manual intervention, and agents need to know when to stop and ask.

Column Renames

EF Core's migration generator can't tell the difference between "rename a column" and "drop a column and add a new one." If you rename a property on your entity, the generated migration will contain:

```
migrationBuilder.DropColumn(name: "Email", table: "Customers");
migrationBuilder.AddColumn<string>(name: "ContactEmail", table: "Customers");
```

This destroys all existing email data. The correct migration is:

```
migrationBuilder.RenameColumn(
    name: "Email",
    table: "Customers",
    newName: "ContactEmail");
```

Your context file must warn about this:

```
## Migration Safety Rules
- Column renames: NEVER auto-generate. Always use RenameColumn manually.
- Table renames: Same rule. Use RenameTable.
- After generating any migration that involves renamed properties, STOP and flag it for review.
```

Data Transforms

Sometimes you need to transform data during a migration. Adding a new column that's computed from existing data, splitting a full name into first and last name, converting a string enum to an integer.

These require raw SQL in the migration:

```
migrationBuilder.AddColumn<string>(
    name: "FirstName",
    table: "Customers",
    nullable: true);

migrationBuilder.AddColumn<string>(
    name: "LastName",
    table: "Customers",
    nullable: true);

migrationBuilder.Sql("""
    UPDATE "Customers"
    SET "FirstName" = split_part("FullName", ' ', 1),
        "LastName" = split_part("FullName", ' ', 2)
    WHERE "FullName" IS NOT NULL
    """);

migrationBuilder.DropColumn(name: "FullName", table: "Customers");
```

Agents can write these SQL transforms, but they need explicit instruction. Don't assume the agent will know that a column rename requires data migration. Tell it:

> *"Rename the Customer.FullName property to FirstName and LastName (split on space). Generate a migration that preserves existing data."*

Changing Column Types

Converting a `string` column to `int` or changing a `varchar(50)` to `varchar(200)` can be safe or dangerous depending on the database engine and existing data. Widening a string column is usually fine. Narrowing it or changing types is risky.

Your context file should include:

```
## Type Changes
- Widening string columns (50 → 200): safe, generate normally
- Narrowing string columns (200 → 50): STOP and flag. Existing data may be truncated.
- String to numeric: STOP and flag. Requires data validation first.
- Numeric precision changes: STOP and flag.
```

Common Agent Mistakes with EF Core

These mistakes appear consistently across different agents and models. Document them in your context file and save yourself repeated corrections.

1. Forgetting to Add DbSet

The agent creates the entity and configuration but forgets to add `DbSet<NewEntity>` to the DbContext. The migration generates empty.

Prevention: Add to your context file: “After creating a new entity, always add a DbSet property to AppDbContext.”

2. Circular References

The agent creates bidirectional navigation properties that cause serialization loops:

```
public class Order
{
    public List<OrderItem> Items { get; set; }
}

public class OrderItem
{
    public Order Order { get; set; } // Circular reference when serializing
}
```

Prevention: “Only add navigation properties you query through. Use foreign key properties (OrderId) without navigation properties when you don’t need to load the related entity.”

3. Missing Cascade Delete Configuration

The agent adds relationships without thinking about what happens when a parent is deleted. EF Core defaults to cascade delete for required relationships, which might not be what you want.

Prevention: “Explicitly configure delete behavior for all relationships. Default to Restrict for production safety.”

```
builder.HasMany(o => o.Items)
    .WithOne()
    .HasForeignKey(i => i.OrderId)
    .OnDelete(DeleteBehavior.Cascade); // Explicit, not default
```

4. Using DateTime Instead of DateTimeOffset

Agents default to `DateTime` because it’s more common in training data. For anything stored in a database, `DateTimeOffset` is the safer choice because it preserves timezone information.

Prevention: “Use DateTimeOffset for all timestamp properties. Never use DateTime for database-stored dates.”

5. Not Using Async Methods

The agent writes `context.SaveChanges()` instead of `await context.SaveChangesAsync()`. Or `context.Products.ToList()` instead of `await context.Products.ToListAsync()`.

Prevention: “All EF Core operations must use async methods.
No .ToList(), .FirstOrDefault(), .SaveChanges(). Always use the Async suffix with await.”

6. Editing Existing Migrations

Sometimes an agent will modify an already-applied migration instead of creating a new one. This causes the migration history to go out of sync with the database.

Prevention: “NEVER edit an existing migration file. Always create a new migration to make changes.”

7. Missing Indexes on Foreign Keys

EF Core automatically creates indexes for foreign keys in some cases, but agents sometimes add redundant indexes or miss composite indexes that would help query performance.

Prevention: “Check that frequently queried columns have indexes. Don’t duplicate indexes that EF Core creates automatically for foreign keys.”

The Migration Review Checklist

Use this checklist every time an agent generates a migration:

```
## Migration Review Checklist
- [ ] Read the entire migration file (Up and Down methods)
- [ ] No accidental DropColumn or DropTable operations
- [ ] Column renames use RenameColumn, not drop+add
- [ ] New non-nullable columns have default values for existing rows
- [ ] Indexes are appropriate (not missing, not redundant)
- [ ] String columns have MaxLength configured
- [ ] Delete behaviors are explicitly set
- [ ] Down() method correctly reverses the Up() method
- [ ] Migration name is descriptive
- [ ] Tested against a real database instance
```

Print this out. Tape it next to your monitor. Or better yet, put it in your AGENTS.md so the agent itself checks these items before presenting the migration to you.

The Bigger Picture

EF Core migrations are a microcosm of the agentic development challenge: agents are fast, capable, and prone to subtle errors that compound over time. A wrong line of C# fails a test. A wrong line in a migration corrupts data.

The solution isn't to avoid agents for migrations. It's to build a workflow that captures the speed benefit while maintaining the safety guarantees. Generate, review, test, apply. Every time. No exceptions.

The developers who get burned by agent-generated migrations are the ones who skip the review step because "it's just adding a column." It's never just adding a column. There's always a default value question, an index question, or a cascade delete question hiding in there. The agent might get it right. But you need to verify.

That verification step is what separates professional agentic development from hoping for the best.

CHAPTER 26

Microsoft Agent Framework

The Microsoft Agent Framework

For over a year, the .NET ecosystem had a confusing split: Semantic Kernel for enterprise AI orchestration, AutoGen for multi-agent research. They had different APIs, different philosophies, and overlapping capabilities. Developers had to pick one and hope it was the right bet.

In late 2025, Microsoft unified them into the Microsoft Agent Framework. One set of APIs. One programming model. One NuGet metapackage. Semantic Kernel's production-grade kernel and plugin system, combined with AutoGen's multi-agent orchestration patterns. If you were using either one, you have a migration path. If you're starting fresh, you only need to learn one thing.

The core abstraction is the `Agent`. An agent has a name, instructions, tools it can call, and a model it uses for reasoning. You compose agents into workflows: sequential (one after another), concurrent (all at once), or orchestrated (a coordinator decides who runs when).

```
var writer = new ChatCompletionAgent
{
    Name = "Writer",
    Instructions = "You write clear, concise technical documentation.",
    Kernel = kernel
};

var reviewer = new ChatCompletionAgent
{
    Name = "Reviewer",
    Instructions = "You review documentation for accuracy and clarity. Be critical.",
    Kernel = kernel
};
```

That's it. Two agents. Each with a role, instructions, and access to the same kernel (which provides the AI model and any tools). Now you can orchestrate them.

Microsoft.Extensions.AI: The Foundation

Before diving into multi-agent patterns, you need to understand the abstraction layer underneath: `Microsoft.Extensions.AI`. This is the unified interface for AI services in .NET, following the same patterns as `Microsoft.Extensions.Http` and `Microsoft.Extensions.Caching`.

The key interfaces:

```
// Chat completion (conversational AI)
IChatClient chatClient = new OpenAIChatClient("gpt-4o", apiKey);

// Embeddings (vector representations)
IEmbeddingGenerator<string, Embedding<float>> embeddings = ...;
```

`IChatClient` is the interface that every AI provider implements. OpenAI, Azure OpenAI, Anthropic, Ollama, local models. You program against the interface, swap providers via DI configuration. No code changes.

```
// Registration in Program.cs
builder.Services.AddChatClient(new OpenAIChatClient("gpt-4o", apiKey));

// Or switch to Anthropic without changing any consuming code
builder.Services.AddChatClient(new AnthropicChatClient("claude-sonnet-4-20250514", apiKey));

// Or use a local model via Ollama
builder.Services.AddChatClient(new OllamaChatClient(new Uri("http://localhost:11434"), "llama3"
```

The middleware pipeline pattern works here too. You can add logging, caching, rate limiting, and telemetry as delegating handlers:

```
builder.Services.AddChatClient(pipeline => pipeline
    .UseOpenTelemetry()
    .UseDistributedCache()
    .UseRateLimiting()
    .Use(new OpenAIChatClient("gpt-4o", apiKey)));
```

This is the .NET way. If you've used `HttpClient` middleware, this feels identical. And it means your multi-agent apps get observability for free when running under Aspire.

Agent Patterns

The Microsoft Agent Framework provides three core orchestration patterns. Each solves a different problem.

Sequential Agents (AgentGroupChat with TerminationStrategy)

Agents take turns in a defined order. Agent A produces output, Agent B refines it, Agent C validates it.

```
var chat = new AgentGroupChat(writer, reviewer)
{
    ExecutionSettings = new()
    {
        TerminationStrategy = new MaximumIterationTerminationStrategy(4)
        {
            Agents = [reviewer] // Reviewer decides when to stop
        }
    }
};

await foreach (var message in chat.InvokeAsync())
{
    Console.WriteLine($"{message.AuthorName}: {message.Content}");
}
```

Use sequential agents when the task has a natural pipeline: draft, review, revise. The writer drafts, the reviewer critiques, the writer revises based on feedback, and the loop continues until the reviewer is satisfied (or you hit the iteration limit).

Concurrent Agents

Multiple agents work on the same input simultaneously. You collect all their outputs and merge the results.

```
var agents = new[] { securityReviewer, performanceReviewer, accessibilityReviewer };

var tasks = agents.Select(agent => agent.InvokeAsync(
    new ChatHistory([new ChatMessageContent(AuthorRole.User, codeToReview)])
));

var results = await Task.WhenAll(tasks);

foreach (var result in results)
{
    Console.WriteLine($"{result.AuthorName}: {result.Content}");
}
```

Use concurrent agents when you need multiple perspectives on the same input and the agents don't depend on each other's output. Code review is the classic example: security, performance, and accessibility are independent concerns.

Orchestrated Agents (Handoff Pattern)

A coordinator agent decides which specialist to invoke based on the current state of the conversation. This is the most flexible and most complex pattern.

```
var triage = new ChatCompletionAgent
{
    Name = "Triage",
    Instructions = """
        You are a customer support triage agent.
        Route to BillingAgent for payment issues.
        Route to TechAgent for technical problems.
        Route to GeneralAgent for everything else.
        """,
    Kernel = kernel
};

// Configure handoff behavior
var orchestrator = new HandoffOrchestrator([triage, billingAgent, techAgent, generalAgent]);
```

The orchestrator pattern works well for customer support, IT help desks, and any scenario where the right specialist depends on the user's request.

MCP C# SDK: Connecting to External Tools

The Model Context Protocol (MCP) is Anthropic's open standard for connecting AI models to external tools and data sources. The MCP C# SDK lets you build MCP servers (expose your .NET APIs as tools) and MCP clients (consume external MCP servers) in C#.

Building an MCP Server

An MCP server exposes tools that any MCP-compatible agent can call. Here's a server that provides access to your product catalog:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddMcpServer()
    .WithTools<ProductTools>();

var app = builder.Build();
app.MapMcp();
app.Run();

public class ProductTools(AppDbContext db)
{
    [McpTool("search_products", "Search products by name or category")]
    public async Task<List<ProductDto>> SearchProducts(
        [Description("Search query")] string query,
        [Description("Maximum results")] int limit = 10)
    {
        return await db.Products
            .Where(p => p.Name.Contains(query) || p.Category.Contains(query))
            .Take(limit)
            .Select(p => new ProductDto(p.Id, p.Name, p.Price, p.Category))
            .ToListAsync();
    }

    [McpTool("get_product", "Get product details by ID")]
    public async Task<ProductDto?> GetProduct(
        [Description("Product ID")] int id)
    {
        var product = await db.Products.FindAsync(id);
        return product is null ? null : new ProductDto(product.Id, product.Name, product.Price,
    }
}
```

The `[McpTool]` attribute exposes methods as tools. The `[Description]` attributes on parameters help the AI model understand what each parameter does. That's the entire server. Any MCP client (Claude Desktop, VS Code agents, other .NET apps) can connect and use these tools.

Consuming an MCP Server

On the client side, you connect to an MCP server and make its tools available to your agents:

```
var mcpClient = await McpClientFactory.CreateAsync(new()
{
    Id = "product-catalog",
    Name = "Product Catalog",
    TransportType = TransportType.Sse,
    Location = "http://localhost:5050/mcp"
});

// Add MCP tools to your kernel
var tools = await mcpClient.ListToolsAsync();
kernel.Plugins.AddFromMcpTools(tools);
```

Now your agents can search products, look up details, and interact with your catalog, all through the standardized MCP protocol.

Real Example: Building a Support Agent System

Let's build something real: a customer support system with three agents (triage, billing, and technical), backed by your product catalog via MCP, and deployed with Aspire.

Project Structure

```
src/
  AppHost/
  ServiceDefaults/
  SupportApi/              # HTTP API for chat interface
  AgentOrchestrator/       # The multi-agent system
  ProductCatalogMcp/       # MCP server for product data
```

The Agent Orchestrator Service

```
// src/AgentOrchestrator/Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.AddServiceDefaults();

// AI model via Microsoft.Extensions.AI
builder.Services.AddChatClient(new AzureOpenAIChatClient(
    new Uri(builder.Configuration["AzureOpenAI:Endpoint"]!),
    "gpt-4o",
    new DefaultAzureCredential()));

builder.Services.AddSingleton<SupportAgentService>();

var app = builder.Build();
app.MapDefaultEndpoints();

app.MapPost("/api/support/chat", async (
    ChatRequest request,
    SupportAgentService agents,
    CancellationToken ct) =>
{
    var response = await agents.HandleMessageAsync(request.Message, request.SessionId, ct);
    return TypedResults.Ok(new ChatResponse(response));
});

app.Run();
```

The Agent Service

```
public class SupportAgentService
{
    private readonly Kernel _kernel;
    private readonly ConcurrentDictionary<string, AgentGroupChat> _sessions = new();

    public SupportAgentService(IChatClient chatClient)
    {
        var kernelBuilder = Kernel.CreateBuilder();
        kernelBuilder.Services.AddSingleton(chatClient);
        _kernel = kernelBuilder.Build();
    }

    public async Task<string> HandleMessageAsync(string message, string sessionId, Cancellation
    {
        var chat = _sessions.GetOrAdd(sessionId, _ => CreateAgentChat());

        chat.AddChatMessage(new ChatMessageContent(AuthorRole.User, message));

        string lastResponse = "";
        await foreach (var response in chat.InvokeAsync(ct))
        {
            lastResponse = response.Content ?? "";
        }

        return lastResponse;
    }

    private AgentGroupChat CreateAgentChat()
    {
        var triage = new ChatCompletionAgent
        {
            Name = "Triage",
            Instructions = """
                You handle customer support requests.
                For billing questions (payments, refunds, invoices): hand off to BillingSpecial
                For technical issues (bugs, errors, setup): hand off to TechSpecialist.
```

```
            For general questions: answer directly using the product catalog tools.
            """,
        Kernel = _kernel
    };

    var billing = new ChatCompletionAgent
    {
        Name = "BillingSpecialist",
        Instructions = """
            You handle billing and payment questions.
            You can look up orders, process refunds, and explain invoices.
            Be empathetic but efficient.
            """,
        Kernel = _kernel
    };

    var tech = new ChatCompletionAgent
    {
        Name = "TechSpecialist",
        Instructions = """
            You handle technical support questions.
            You can look up product specifications and known issues.
            Provide step-by-step troubleshooting.
            """,
        Kernel = _kernel
    };

    return new AgentGroupChat(triage, billing, tech)
    {
        ExecutionSettings = new()
        {
            SelectionStrategy = new KernelFunctionSelectionStrategy(
                KernelFunctionFactory.CreateFromPrompt("""
                    Determine which agent should respond next based on the conversation.
                    Agents: Triage, BillingSpecialist, TechSpecialist
                    Return only the agent name.
```

```
                        """),
                    _kernel),
                TerminationStrategy = new MaximumIterationTerminationStrategy(6)
            }
        };
    }
}
```

The AppHost

```
var builder = DistributedApplication.CreateBuilder(args);

var postgres = builder.AddPostgres("postgres");
var catalogDb = postgres.AddDatabase("catalogdb");

var productMcp = builder.AddProject<Projects.ProductCatalogMcp>("product-mcp")
    .WithReference(catalogDb);

var orchestrator = builder.AddProject<Projects.AgentOrchestrator>("agent-orchestrator")
    .WithReference(productMcp);

var supportApi = builder.AddProject<Projects.SupportApi>("support-api")
    .WithReference(orchestrator);

builder.Build().Run();
```

What the Aspire Dashboard Shows You

When you run this and send a support message, the dashboard reveals the entire flow:

1. HTTP POST to SupportApi
2. SupportApi calls AgentOrchestrator
3. Triage agent evaluates the message
4. If it’s a billing question, the selection strategy routes to BillingSpecialist

5. BillingSpecialist calls the ProductCatalogMcp tools to look up order data
6. Response flows back through the chain

Every AI model call shows up as a span in the trace: the prompt, the response, token counts, latency. Every MCP tool call is visible. You can see exactly which agent handled the request and why.

This observability is why Aspire integration matters for multi-agent apps. Without it, debugging “why did the agent give a wrong answer” is guesswork. With it, you trace the exact decision chain.

Agent Design Principles for .NET

After building several multi-agent systems, these principles consistently produce better results:

Keep agents focused. An agent that does one thing well is better than an agent that does five things poorly. The triage/specialist pattern works because each agent has a narrow scope.

Use tools, not knowledge. Don’t put product details in the agent’s instructions. Give the agent a tool to look up product details. Tools are always current. Instructions get stale.

Set iteration limits. Multi-agent conversations can loop forever if two agents keep handing work back and forth. Always set `MaximumIterationTerminationStrategy` with a reasonable cap.

Log everything. Use `Microsoft.Extensions.AI`’s telemetry middleware to capture every model call. When a multi-agent system behaves unexpectedly, the logs are the only way to understand why.

Test agent behavior, not implementation. Don’t unit test that Agent A hands off to Agent B. Test that a billing question gets a billing answer. The routing is an implementation detail.

Testing Multi-Agent Apps

Testing agent-powered applications requires a different approach than traditional unit tests. You're testing probabilistic behavior, not deterministic functions.

```
public class SupportAgentTests(WebApplicationFactory<Program> factory)
    : IClassFixture<WebApplicationFactory<Program>>
{
    [Theory]
    [InlineData("I need a refund for order #123", "billing")]
    [InlineData("The app crashes when I click save", "technical")]
    [InlineData("What products do you have?", "catalog")]
    public async Task HandleMessage_RoutesToCorrectDomain(string message, string expectedDomain
    {
        var client = factory.CreateClient();
        var response = await client.PostAsJsonAsync("/api/support/chat",
            new { Message = message, SessionId = Guid.NewGuid().ToString() });

        var result = await response.Content.ReadFromJsonAsync<ChatResponse>();

        // Assert the response contains domain-relevant content
        // Not exact string matching (LLM output varies), but domain verification
        result.Should().NotBeNull();
        result!.Response.Should().NotBeNullOrEmpty();
    }
}
```

For more rigorous testing, use `Microsoft.Extensions.AI.Evaluations` to score agent responses against expected criteria:

```
var evaluator = new RelevanceEvaluator(chatClient);
var score = await evaluator.EvaluateAsync(
    question: "I need a refund",
    answer: agentResponse,
    context: "Customer support billing domain");

score.Score.Should().BeGreaterThan(0.7);
```

Where This Is Going

The Microsoft Agent Framework is in its early days. The APIs will evolve. But the patterns are stable: focused agents, tool-based knowledge, orchestrated workflows, observable execution. These patterns work regardless of which framework version you're running.

The combination of Microsoft.Extensions.AI (unified model access), MCP (standardized tool integration), the Agent Framework (multi-agent orchestration), and Aspire (deployment and observability) gives .NET developers a complete stack for building production agent systems. No other ecosystem has this level of integration between the AI layer and the application platform.

Whether you're building a customer support bot, an internal knowledge assistant, or an automated code review pipeline, the building blocks are the same. Define your agents. Give them tools. Orchestrate their interactions. Observe the results. Iterate.

That's multi-agent development in .NET. Not hype. Just engineering.

PART VII

The Discipline

CHAPTER 27

Security Review with Agents

Security review is the task most developers skip and most teams underinvest in. Not because they don't care, but because it's tedious, specialized, and easy to defer. "We'll do a security pass before launch" becomes "we'll fix it if something breaks."

Agents change the economics of security review. Not because they replace security experts (they don't), but because they make first-pass security scanning nearly free. The effort drops from "schedule a meeting with the security team" to "run the agent before you push." That's a meaningful shift.

This chapter covers how to use agents for systematic security review: what they catch, what they miss, and how to build a workflow that makes security a continuous habit instead of a quarterly event.

The OWASP Top 10 with Agent Eyes

The OWASP Top 10 is the standard checklist for web application security. Let's walk through each category and how well agents handle it.

What Agents Catch Well

Injection (A03:2021). Agents are excellent at spotting SQL injection, command injection, and LDAP injection vulnerabilities. They can trace data flow from user input to database queries and flag unsanitized usage. This is pattern matching at scale, exactly what LLMs excel at.

```
// An agent will flag this immediately
public async Task<User> GetUser(string username)
{
    var query = $"SELECT * FROM Users WHERE Username = '{username}'";
    return await _db.QueryFirstAsync<User>(query);
}
```

The fix is obvious to the agent too: parameterized queries, or better yet, an ORM. Agents catch these with near-100% accuracy because the pattern is unambiguous.

Security Misconfiguration (A05:2021). Agents are surprisingly good at catching misconfigurations in startup code, middleware ordering, CORS policies, and header settings. They know the correct patterns and can spot deviations.

```
// Agent will flag: CORS allows any origin
builder.Services.AddCors(options =>
{
    options.AddDefaultPolicy(policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyMethod()
              .AllowAnyHeader();
    });
});
```

Vulnerable and Outdated Components (A06:2021). This is where agents shine brightest. They can scan your `.csproj` files, `package.json`, `requirements.txt`, or any dependency manifest and cross-reference against known vulnerability databases. More on this in the dependency auditing section below.

Identification and Authentication Failures (A07:2021). Agents catch weak password policies, missing rate limiting on login endpoints, tokens stored in local storage, and missing HTTPS enforcement. These are well-documented anti-patterns that agents have seen thousands of times in training data.

What Agents Catch Sometimes

Broken Access Control (A01:2021). Agents can spot missing `[Authorize]` attributes and obvious authorization gaps. But they struggle with business logic authorization, the kind where “managers can only see their own team’s data” is buried in a requirements document the agent has never read.

Cryptographic Failures (A02:2021). Agents know that MD5 is broken and that you shouldn’t roll your own crypto. They’ll flag weak hashing algorithms and hardcoded encryption keys. But subtle cryptographic issues (timing attacks, improper IV reuse, weak random number generation in security contexts) get missed more often than not.

Server-Side Request Forgery (A10:2021). Agents can flag obvious SSRF patterns where user input directly controls a URL being fetched. They miss the subtle cases where URLs are constructed through multiple layers of indirection.

What Agents Miss

Insecure Design (A04:2021). This is the category where agents fail most consistently. Insecure design is about architectural decisions: missing threat modeling, insufficient separation of concerns, no defense in depth. An agent reviewing code can’t tell you that your architecture lacks a proper trust boundary. That requires understanding the system holistically, something that needs a human security architect.

Software and Data Integrity Failures (A08:2021). CI/CD pipeline security, unsigned updates, deserialization attacks in complex chains. Agents might catch `BinaryFormatter` usage (a known .NET vulnerability), but they won’t evaluate whether your deployment pipeline has proper integrity checks.

Security Logging and Monitoring Failures (A09:2021). Agents can check if you’re logging authentication events, but they can’t evaluate whether your monitoring is sufficient to detect an active breach. This requires operational context agents don’t have.

Dependency Vulnerability Auditing

This is the lowest-effort, highest-value security task you can hand to an agent. Here's the workflow:

The Prompt

```
Audit all NuGet dependencies in this solution for known vulnerabilities.

For each project:
1. List all direct dependencies and their versions
2. Check if any have known CVEs
3. Flag any packages that are more than 2 major versions behind
4. Identify packages that are deprecated or unmaintained
5. Check for packages with known supply chain concerns

Output a table: Package | Current Version | Latest Version | Vulnerabilities | Risk Level
```

Combining with CLI Tools

Agents work even better when they can run `dotnet list package --vulnerable --include-transitive`:

```
Run `dotnet list package --vulnerable --include-transitive` on the solution.
Parse the output and for each vulnerable package:
1. Explain the vulnerability in plain language
2. Assess the impact on our specific application
3. Recommend the upgrade path
4. Flag any breaking changes in the upgrade
```

This is the kind of task that would take a developer an hour of research per vulnerability. The agent does it in minutes, with reasonable accuracy. You still need to verify the upgrade paths, but the research phase is nearly eliminated.

Transitive Dependencies Matter

Direct dependencies are easy. Transitive dependencies are where the real risk lives. Your app uses PackageA, which depends on PackageB v2.1, which has a critical vulnerability. You'd never know without scanning the full dependency tree.

Agents handle this well because they can parse the `dotnet list package --include-transitive` output and cross-reference each package. The key insight: tell the agent to focus on transitive vulnerabilities specifically, because those are the ones humans consistently miss.

Secrets Detection

Hardcoded secrets in code and configuration files remain one of the most common security failures. Agents are excellent at catching them.

The Prompt

```
Scan the entire repository for hardcoded secrets, API keys, connection strings,
passwords, tokens, and credentials.

Check:
- Source code files (*.cs, *.js, *.ts, *.py)
- Configuration files (appsettings*.json, *.config, *.yaml, *.env)
- Infrastructure files (*.tf, *.bicep, docker-compose*.yml)
- Test files (often overlooked but still risky)
- Documentation (README files sometimes contain real credentials)

For each finding:
1. File and line number
2. Type of secret (API key, password, connection string, etc.)
3. Risk level (is this a production credential or a test value?)
4. Recommended remediation (environment variable, key vault, user secrets)
```

What Agents Catch

Agents reliably catch:

- Connection strings with passwords: `Server=prod-db;Password=hunter2`
- API keys in config: `"ApiKey": "sk-proj-abc123..."`
- Hardcoded JWTs and bearer tokens
- AWS/Azure/GCP credentials in any format
- Private keys (PEM format, XML format)
- Passwords in comments ("// TODO: remove this password: admin123")

The False Positive Problem

Agents over-report. They'll flag placeholder values (`"YOUR_API_KEY_HERE"`), test fixtures, and example configurations. This is annoying but preferable to under-reporting. Build a simple workflow: the agent flags everything, you triage quickly, and the few false positives take seconds to dismiss.

A good follow-up prompt:

```
From the secrets you found, separate them into three categories:
1. CRITICAL: Looks like a real production credential
2. REVIEW: Could be real, needs human verification
3. LIKELY SAFE: Test values, placeholders, or examples
```

This reduces your triage time significantly.

Security-Focused Code Review Prompts

Beyond scanning for specific vulnerability categories, you can use agents for general security-focused code review. Here are prompts that work well.

For API Endpoints

```
Review this API endpoint for security concerns:

1. Input validation: Is all user input validated before processing?
2. Authorization: Are there proper access controls? Can users access
   other users' data?
3. Rate limiting: Is this endpoint protected against abuse?
4. Data exposure: Does the response include more data than necessary?
5. Error handling: Do error messages leak internal details?
6. Logging: Are security-relevant events logged?

Be specific. Point to exact lines of code.
```

For Authentication Flows

```
Review this authentication implementation:

1. Password handling: Proper hashing? Salt? Work factor?
2. Token management: Secure generation? Proper expiration? Refresh flow?
3. Session handling: Secure cookies? CSRF protection? Session fixation?
4. Account enumeration: Can an attacker determine if an account exists?
5. Brute force protection: Rate limiting? Account lockout? CAPTCHA?
6. MFA implementation: Proper TOTP? Recovery codes?
```

For Data Access Layers

```
Review this data access code for security:

1. SQL injection: Any raw queries with string concatenation?
2. Mass assignment: Can users set fields they shouldn't (isAdmin, price)?
3. IDOR: Can users access resources by manipulating IDs?
4. Data filtering: Is tenant isolation enforced at the query level?
5. Sensitive data: Are passwords, tokens, or PII properly handled?
```

The Systematic Security Review Workflow

Here's the workflow I recommend. It runs in under 30 minutes for most codebases and catches the vast majority of low-hanging fruit.

Phase 1: Automated Scanning (5 minutes)

Run the agent with this prompt:

```
Perform a security scan of this codebase:

1. Run `dotnet list package --vulnerable --include-transitive`
2. Scan all files for hardcoded secrets
3. Check all API endpoints for missing [Authorize] attributes
4. Review middleware ordering in Program.cs for security issues
5. Check CORS, HTTPS, and header security configuration

Produce a findings report sorted by severity.
```

The agent runs the CLI commands, scans the codebase, and produces a report. You skim the report, noting anything critical.

Phase 2: Targeted Review (15 minutes)

Pick the highest-risk areas identified in Phase 1 and run focused reviews:

```
Deep review the authentication controller at src/Api/Controllers/AuthController.cs.
Check for all OWASP authentication failures.
```

```
Review the payment processing service at src/Domain/Payments/PaymentService.cs.
Focus on input validation, error handling, and data exposure.
```

Phase 3: Threat Modeling Assist (10 minutes)

This is where agents help you think, not just scan:

```
Based on the codebase structure, identify the top 5 attack vectors
an external attacker would target. For each:
1. The attack vector
2. Current defenses (if any)
3. Recommended mitigations
```

Agents produce surprisingly useful threat models for well-structured codebases. They won't match a dedicated security architect's analysis, but they'll identify the obvious attack surfaces you might overlook when you're deep in feature work.

Real Example: Security Review of an API Endpoint

Let's walk through a concrete example. Here's a payment endpoint that an agent might review:

```
[ApiController]
[Route("api/v1/[controller]")]
public class PaymentsController : ControllerBase
{
    private readonly IPaymentService _paymentService;
    private readonly ILogger<PaymentsController> _logger;

    public PaymentsController(
        IPaymentService paymentService,
        ILogger<PaymentsController> logger)
    {
        _paymentService = paymentService;
        _logger = logger;
    }

    [HttpPost]
    public async Task<IActionResult> ProcessPayment(PaymentRequest request)
    {
        _logger.LogInformation(
            "Processing payment for card {CardNumber}, amount {Amount}",
            request.CardNumber, request.Amount);

        var result = await _paymentService.ProcessAsync(request);

        if (!result.Success)
            return BadRequest(new { error = result.ErrorMessage, details = result.Exception?.To

        return Ok(result);
    }
}

public record PaymentRequest(
    string CardNumber,
    string Cvv,
    int ExpiryMonth,
    int ExpiryYear,
```

```
    decimal Amount,
    string Currency);
```

Agent Findings

A well-prompted agent will identify these issues:

CRITICAL: PCI violation in logging. The `LogInformation` call logs the full card number. This violates PCI DSS requirements. Card numbers must never appear in logs. The agent flags this immediately because it's a well-known anti-pattern.

CRITICAL: Missing authorization. No `[Authorize]` attribute on the controller or action. Anyone can call this endpoint. The agent catches missing authorization attributes reliably.

HIGH: Exception details in error response. `result.Exception?.ToString()` returns the full stack trace to the caller. This leaks internal implementation details that help attackers. The agent recommends returning a generic error message and logging the exception server-side.

HIGH: No input validation. No validation on `CardNumber` format, `Amount` range (negative amounts?), `Currency` whitelist, or `ExpiryMonth` / `ExpiryYear` sanity checks. The agent suggests adding `[Required]`, `[Range]`, `[RegularExpression]`, and a FluentValidation validator.

MEDIUM: No rate limiting. Payment endpoints are high-value targets for card testing attacks. The agent recommends `[EnableRateLimiting("payment")]` or equivalent middleware.

MEDIUM: CVV in the request model. If this request model is logged or serialized anywhere, the CVV is exposed. The agent suggests a `[JsonIgnore]` attribute on CVV for serialization, or better yet, tokenizing the card data before it reaches your API.

LOW: No idempotency key. Without an idempotency mechanism, retried requests could process duplicate payments. The agent suggests adding an `IdempotencyKey` header.

The Fixed Version

After the agent applies its recommendations:

```
[ApiController]
[Route("api/v1/[controller]")]
[Authorize]
[EnableRateLimiting("payment")]
public class PaymentsController : ControllerBase
{
    private readonly IPaymentService _paymentService;
    private readonly ILogger<PaymentsController> _logger;

    public PaymentsController(
        IPaymentService paymentService,
        ILogger<PaymentsController> logger)
    {
        _paymentService = paymentService;
        _logger = logger;
    }

    [HttpPost]
    public async Task<IActionResult> ProcessPayment(
        [FromBody] PaymentRequest request,
        [FromHeader(Name = "Idempotency-Key")] string? idempotencyKey)
    {
        _logger.LogInformation(
            "Processing payment for card ending {LastFour}, amount {Amount} {Currency}",
            request.CardNumber[^4..], request.Amount, request.Currency);

        var result = await _paymentService.ProcessAsync(request, idempotencyKey);

        if (!result.Success)
        {
            _logger.LogWarning("Payment failed: {Error}", result.ErrorMessage);
            return BadRequest(new { error = "Payment processing failed." });
        }

        return Ok(new { transactionId = result.TransactionId, status = "approved" });
    }
```

```
}

public record PaymentRequest
{
    [Required]
    [CreditCard]
    public required string CardNumber { get; init; }

    [Required]
    [JsonIgnore(Condition = JsonIgnoreCondition.WhenWritingDefault)]
    public required string Cvv { get; init; }

    [Range(1, 12)]
    public required int ExpiryMonth { get; init; }

    [Range(2024, 2040)]
    public required int ExpiryYear { get; init; }

    [Range(0.01, 999999.99)]
    public required decimal Amount { get; init; }

    [Required]
    [RegularExpression("^[A-Z]{3}$")]
    public required string Currency { get; init; }
}
```

Six of the seven findings are mechanical fixes. The agent handled them all correctly. The seventh (CVV tokenization) is an architectural decision that requires human judgment about your payment provider integration.

What Agents Catch Well vs. What They Miss

Let me be direct about this, because overselling agent security capabilities is dangerous.

Agents Are Reliable For

- **Known vulnerability patterns.** SQL injection, XSS, CSRF, hardcoded secrets, weak hashing. These are well-documented, pattern-based, and agents have seen millions of examples.
- **Configuration errors.** Missing HTTPS, overly permissive CORS, incorrect middleware ordering, debug settings in production configs.
- **Dependency vulnerabilities.** Cross-referencing package versions against CVE databases.
- **Missing security controls.** No authorization, no rate limiting, no input validation, no CSRF tokens.
- **Data exposure.** Logging sensitive data, returning too much in API responses, verbose error messages.

Agents Are Unreliable For

- **Business logic authorization.** “Can this user see this data?” depends on business rules the agent doesn’t know.
- **Architectural security.** Trust boundaries, defense in depth, network segmentation. These require system-level thinking.
- **Novel attack vectors.** Zero-day patterns, creative exploitation chains, attacks specific to your domain.
- **Compliance.** HIPAA, PCI DSS, GDPR requirements beyond obvious data handling. Agents know the regulations exist but can’t audit compliance reliably.
- **Timing and race conditions.** TOCTOU bugs, race conditions in concurrent code, timing side channels. These are subtle and agents miss them consistently.

- **Supply chain attacks.** Typosquatting packages, compromised maintainers, malicious build scripts. Agents can check known vulnerabilities but can't evaluate trust.

The First-Pass Filter Model

The right mental model: agents are a first-pass security filter. They catch the obvious stuff quickly and cheaply. This raises the floor of your security posture dramatically, because "obvious stuff" is what causes most breaches.

But they don't raise the ceiling. For that, you need security experts, penetration testing, formal threat modeling, and architectural review. The agent handles the 80% of findings that are mechanical. Humans handle the 20% that require judgment.

The mistake is treating agent security review as sufficient. It's not. It's necessary but not sufficient. The discipline is knowing the difference: let agents handle the scanning, flagging, and obvious fixes. Reserve your (or your security team's) attention for the architectural decisions, business logic authorization, and novel threats that agents can't evaluate.

Building the Habit

The best security review is the one that actually happens. Agents make it happen by reducing the cost to near zero.

Add security review to your agent workflow: 1. Before every PR, run a security-focused agent review on changed files 2. Weekly, run a full dependency audit 3. Monthly, run a comprehensive codebase security scan 4. Quarterly, have a human security expert review the agent's findings and your threat model

The agent handles steps 1-3. Step 4 is where human expertise compounds the value. The agent's continuous scanning keeps the codebase clean between expert reviews. The expert reviews catch what agents miss and improve your agent prompts for next time.

That’s the security review workflow. Agents as a continuous, cheap, reliable first pass. Humans for the judgment calls. Together, they create a security posture that neither achieves alone.

CHAPTER 28

When NOT to Use Agents

Every chapter in this book has been about when and how to use agents effectively. This chapter is about when to turn them off.

This is the chapter most agentic development advocates won't write. The toolmakers won't write it because it's bad for business. The enthusiasts won't write it because it contradicts the narrative. But if you're going to be genuinely productive with agents, you need to know when they're the wrong tool. Using agents everywhere, on everything, all the time, will make you slower. Not theoretically slower. Measurably slower.

The METR study proved it. Experienced developers on familiar codebases were 19% slower with AI tools. And they *believed* they were 20% faster. That gap between perception and reality is the danger zone. This chapter is your map out of it.

Security-Critical Code Paths

Chapter 27 covered how agents can help with security review. Here's the flip side: agents should not be the primary author of security-critical code.

Authentication flows, cryptographic implementations, authorization logic, token handling, certificate validation, key management. These are areas where subtle bugs have catastrophic consequences and where agents make subtle bugs.

An agent will happily generate a JWT validation routine that looks correct, passes your tests, and has a timing vulnerability that leaks token validity information to attackers. It'll write a password hashing function that uses bcrypt (good) with a work factor of 4 (dangerously low). It'll implement OAuth correctly for 95% of cases and miss the state parameter validation that prevents CSRF attacks.

The problem isn't that agents write *bad* security code. They write *almost correct* security code. And in security, almost correct is worse than obviously broken, because you'll ship it.

The rule: Use agents to review security code. Use agents to write tests for security code. Write the security code yourself, or use battle-tested libraries (which is what you should do anyway).

Performance-Sensitive Hot Paths

Agents optimize for correctness and readability. Those are usually the right priorities. But in hot paths (code that executes thousands or millions of times per second) performance dominates, and agents consistently make the wrong tradeoffs.

Consider a serialization hot path in a high-throughput API:

```
// What the agent writes (correct, readable, slow)
public string Serialize(Order order)
{
    return JsonSerializer.Serialize(order);
}

// What you need (correct, less readable, 10x faster)
public void Serialize(Order order, IBufferWriter<byte> buffer)
{
    using var writer = new Utf8JsonWriter(buffer, new JsonWriterOptions { SkipValidation = true
    writer.WriteStartObject();
    writer.WriteString("id"u8, order.Id);
    writer.WriteNumber("total"u8, order.Total);
    // ... manual serialization for each field
    writer.WriteEndObject();
}
```

Agents don't know your performance requirements. They don't know that this method is called 50,000 times per second. They'll use LINQ where a for loop saves allocations. They'll create new objects where pooling is essential. They'll use string concatenation where `StringBuilder` or `Span<T>` is required.

You *can* prompt agents for performance-optimized code. But you'll spend more time specifying the constraints, reviewing the output, and benchmarking the result than you'd spend writing it yourself. The "faster by hand" test (more on this below) almost always triggers for hot path code.

The rule: Identify your hot paths with profiling first. Write those paths by hand. Use agents for everything else. That's probably 95-99% of your codebase.

Deep Domain Logic

Every codebase has logic that requires business context no agent possesses. Insurance rate calculation tables. Regulatory compliance rules. Medical dosage algorithms. Financial instrument pricing models. Tax calculation edge cases for specific jurisdictions.

These are areas where:

1. The requirements are complex, nuanced, and often contradictory
2. The source of truth is a human expert, a legal document, or a spreadsheet, not code
3. Getting it wrong has real-world consequences (financial, legal, medical)
4. The logic changes based on context that isn't in the codebase

An agent can implement a tax calculation if you give it a complete, unambiguous specification. But creating that specification is the hard part. And if the specification is complete enough for an agent to implement correctly, it's complete enough that the implementation is trivial. You're not saving time; you're adding a translation layer between your spec and the code.

The rule: When the domain expert needs to be in the room while you code, the agent doesn't help. Write it yourself, with the expert, and add comprehensive tests. Then use agents to maintain and refactor it later, when the tests protect the business rules.

The METR Paradox Revisited

The METR study deserves deeper examination here because it reveals something important about when agents fail.

The study measured experienced open-source developers working on codebases they maintained. These developers knew every file, every pattern, every quirk. When they used AI tools:

- Tasks took 19% longer on average
- Developers believed they were 20% faster
- The perception gap was roughly 40 percentage points

Why? Several factors:

Context switching cost. Prompting the agent, waiting for output, reviewing the output, correcting the output. For code you could write in 30 seconds from muscle memory, this overhead adds 2-3 minutes. Multiply by dozens of small changes per day.

Review burden on familiar code. When you know the codebase intimately, you can write correct code on the first try. But you still have to review agent output line by line, because the agent doesn't know the codebase intimately. The review takes longer than writing.

The illusion of delegation. "I'll let the agent handle this" feels productive. You're delegating! But if the delegation cost (prompting + reviewing + correcting) exceeds the implementation cost, you've paid more for less.

Overcorrection loops. The agent produces something 90% right. You correct the 10%. The agent "fixes" it but breaks something in the original 90%. You correct again. Two iterations later, you've spent triple the time.

The METR finding doesn't mean agents are useless. It means agents are useless *when you already know what to type.* The value of agents scales inversely with your familiarity with the code:

- **Unfamiliar codebase:** Agents save massive time (exploration, understanding, boilerplate)
- **Somewhat familiar:** Agents save moderate time (complex features, cross-cutting concerns)
- **Deeply familiar:** Agents often cost time (simple changes, routine maintenance)

The discipline is recognizing which category you're in for each task.

Tasks Where Agents Consistently Struggle

Beyond the categories above, here are specific task types where agents reliably underperform:

Large-scale refactoring across many files. Agents handle single-file refactoring well. But when you need to rename a concept across 50 files, update all the tests, fix all the DI registrations, and adjust the database migrations, agents lose coherence. They'll miss files, introduce inconsistencies, and create merge conflicts with themselves. Your IDE's refactoring tools (Rename Symbol, Find All References) are faster and more reliable.

UI polish and visual design. Agents can build functional UI. They cannot make it look good. They'll produce a form that works, with the wrong spacing, inconsistent margins, and colors that clash. Visual quality requires human eyes.

Debugging production issues from logs. Agents can help parse logs and suggest hypotheses. But production debugging requires understanding the system's runtime behavior, infrastructure quirks, and deployment history. "The service crashed at 3 AM" requires context about traffic patterns, recent deployments, and infrastructure changes that agents don't have.

Writing meaningful commit messages and documentation. Agents write technically accurate but soulless documentation. They'll describe what the code does without explaining why it exists or how it fits the larger picture. Documentation that actually helps future developers requires human judgment about what matters.

Complex database query optimization. Agents can write queries. They can't evaluate whether a query will perform well against your specific data distribution, indexes, and access patterns. They'll suggest indexes without knowing your write-to-read ratio. They'll recommend denormalization without understanding your consistency requirements.

The "Faster by Hand" Test

Before starting any agent task, ask yourself: "Would I finish this faster by just writing it myself?"

Be honest. Factor in:

- **Prompting time.** How long to write the prompt with enough context?
- **Waiting time.** Agent response time (seconds to minutes depending on the tool and task complexity).
- **Review time.** How long to verify the output is correct?
- **Correction time.** How many iterations before it's right?
- **Your typing speed.** For simple code you've written before, your fingers are fast.

If the total agent time (prompt + wait + review + correct) exceeds your direct implementation time by more than 20%, skip the agent. The 20% buffer accounts for the cognitive benefit of reviewing code versus writing it, but beyond that buffer, you're wasting time.

Tasks that almost always pass the "faster by hand" test:

- Simple bug fixes where you already know the root cause
- Adding a field to a model, DTO, and mapping

- Writing a one-line configuration change
- Fixing a typo or renaming a variable
- Adding a log statement
- Updating a version number

Tasks that almost always fail it (use the agent):

- Implementing a new feature with multiple files
- Writing a comprehensive test suite
- Building boilerplate (controllers, DTOs, mappers, DI registration)
- Exploring an unfamiliar part of the codebase
- Migrating code from one pattern to another
- Generating documentation from code

The middle ground is where judgment matters. And that judgment improves with practice.

Recognizing Diminishing Returns

Agent-assisted development follows a curve of diminishing returns within a single task. The first 80% of the implementation comes fast. The next 15% takes as long as the first 80%. The final 5% takes longer than everything else combined.

This happens because:

- The easy parts (boilerplate, standard patterns, happy path) are what agents do best
- The hard parts (edge cases, integration points, business-specific logic) are what agents do worst
- Each correction cycle takes longer as the code gets more complex and intertwined

Recognize the inflection point. When you’re on your third correction cycle and the agent keeps oscillating between two broken states, stop. Take over. Write the remaining code yourself. You’ve extracted the value the agent can provide.

A practical signal: if you've prompted the agent three times for the same issue and it's still not right, you'll finish faster by hand. This isn't a hard rule, but it's a reliable heuristic.

The Discipline of Turning Off the Tool

The hardest part of agent-assisted development isn't learning when to use agents. It's learning when to stop.

There's a psychological pull toward using agents for everything. You've invested in learning the tools. You've set up context files. You've built workflows. The sunk cost makes you want to use agents even when they're not helping. Add the perception gap from the METR study (feeling faster while being slower) and you have a recipe for persistent misuse.

Build the habit of checking in with yourself:

- Am I actually making progress, or am I prompting and re-prompting?
- Is the agent adding value here, or am I doing the work twice (once to specify, once to review)?
- Would I be done already if I'd just started typing?
- Am I using the agent because it's the right tool, or because it's the tool I have open?

The best agentic developers I know switch fluidly between agent-assisted and manual coding, sometimes within the same feature. They'll use an agent for the boilerplate, the tests, and the standard patterns. Then they'll close the agent panel, put on headphones, and write the core logic by hand. Then they'll bring the agent back for the cleanup, documentation, and review.

That's the discipline. Not "always use agents" or "never use agents." It's knowing which mode you're in for each task, each file, each function. And being honest with yourself about when the tool is serving you versus when you're serving the tool.

The goal was never to use agents as much as possible. The goal is to build great software as efficiently as possible. Sometimes agents get you there. Sometimes your own keyboard does. The professional knows the difference.

CHAPTER 29

The 30-Day Agentic Transformation

You've read the theory. You've seen the workflows. Now it's time to actually do this.

This chapter is a 30-day plan to transform how you build software. Not a gradual maybe-someday shift. A structured, day-by-day plan with specific tasks, checkpoints, and measurements. By day 30, you'll either be working in a fundamentally different way, or you'll have concrete data on why the agentic approach doesn't work for your context. Either outcome is valuable.

One rule: follow the plan in order. Each day builds on the previous one. Skipping ahead is how people end up with powerful tools they use poorly.

Before You Start

Measure your baseline. Pick a representative task you'll do in the next few days (a feature, a bug fix, a refactoring) and track:

- **Time to complete** (wall clock, not just coding time)
- **Lines of code written/changed**
- **Number of bugs found during testing**
- **How many files you touched**

Write these numbers down. You'll compare against them in Week 4.

Week 1: Foundation (Days 1-7)

This week is about setup and first contact. You're building the infrastructure that makes everything else work.

Day 1: Install and Configure Your Agent Tools

Pick your primary agent. I recommend starting with one, not three:

- **Claude Code** (CLI agent, terminal-native, strong for .NET)
- **GitHub Copilot Agent Mode** (IDE-integrated, familiar for VS Code users)
- **Cursor** (IDE with deep agent integration)

Install it. Configure it. Run your first "hello world" prompt on your actual project:

```
Explain the high-level architecture of this codebase.
What are the main projects and how do they relate to each other?
```

Don't ask it to write code yet. Just verify it can read your codebase and produce coherent output.

Day 1 checkpoint: Agent installed, configured, and can read your codebase. You've had one conversation with it about your project.

Day 2: Create AGENTS.md for Your Main Project

This is the most important file you'll create during the 30 days. Open your project root and create `AGENTS.md` (or `CLAUDE.md`, `.cursorrules`, or `copilot-instructions.md`, depending on your tool).

Start minimal. Include only what the agent needs to know right now:

```
# Project: MyApp

## Build & Run
- `dotnet build` to build
- `dotnet test` to run all tests
- `dotnet run --project src/Api` to start the API

## Architecture
- src/Api: ASP.NET Core Web API
- src/Domain: Business logic, no framework dependencies
- src/Infrastructure: EF Core, external service clients
- tests/: xUnit + FluentAssertions

## Conventions
- Use records for DTOs
- Use MediatR for CQRS
- Async all the way (suffix with Async)
- Nullable reference types enabled

## Do NOT
- Add packages without asking
- Change the database schema directly (use migrations)
- Modify Program.cs without explaining why
```

This takes 15-20 minutes and saves hours over the next month.

Day 2 checkpoint: AGENTS.md committed to your repository. You've listed build commands, architecture, core conventions, and anti-patterns.

Day 3: First Agent Task with Context File

Pick a small, low-risk task. A bug fix. A new DTO. Adding validation to an existing endpoint. Something that takes you 20-30 minutes by hand.

Do it with the agent instead. Use this prompt structure:

```
[Task description in 2-3 sentences]

Relevant files:
- src/Api/Controllers/OrdersController.cs
- src/Domain/Orders/Order.cs
- tests/Api.Tests/OrdersControllerTests.cs

[Specific instructions or constraints]
```

Time yourself. Note what went well and what went poorly. Was the agent faster? Did it follow your conventions from AGENTS.md?

Day 3 checkpoint: One real task completed with agent assistance. You have a gut feel for whether the context file helped.

Days 4-5: Add Path-Scoped Rules

Based on your Day 3 experience, the agent probably did some things wrong. Wrong naming conventions in tests. Wrong patterns in the API layer. Wrong import style.

Create path-scoped rules for your most common file types:

For Claude Code, create files like `.claude/rules/tests.md`:

```
# Test Conventions

- Use xUnit with FluentAssertions
- Test class name: {ClassName}Tests
- Method name: {Method}_{Scenario}_{ExpectedResult}
- Use Arrange/Act/Assert comments
- Use NSubstitute for mocking, not Moq
- Always test edge cases and error conditions
```

For Copilot, add sections to `copilot-instructions.md`. For Cursor, use `.cursorrules`.

Create at least 3 path-scoped rules: 1. Test file conventions 2. API/Controller conventions 3. Domain/business logic conventions

Days 4-5 checkpoint: Path-scoped rules created. Run another small task and verify the agent follows them.

Days 6-7: Your First TDD Agent Loop

This is the workflow from Chapter 5. Time to try it for real.

Pick a task with clear business logic. Write the tests first. All of them. Then hand it to the agent:

```
Make all tests in [TestFile] pass.
Create the necessary types and implementation in [TargetDirectory].
Run the tests after implementation to verify they pass.
```

Don't help the agent. Let it iterate. Watch how it handles test failures and self-corrects.

Days 6-7 checkpoint: One complete TDD Agent Loop task done. Tests pass. Code reviewed and committed. You've experienced the full cycle.

Week 1 Retrospective

At the end of Week 1, update your AGENTS.md with anything you learned. Add conventions the agent missed. Remove instructions that were unnecessary. This file is a living document.

Ask yourself: - Is the agent saving me time? On what types of tasks? - What's my biggest friction point? - What do I keep correcting?

Write your answers down. They shape Week 2.

Week 2: Core Workflows (Days 8-14)

This week is about repetition and range. You're building muscle memory for the three core agentic workflows.

Days 8-10: TDD Agent Loop Variations

Run the TDD Agent Loop on three different task types:

Day 8: Business logic. Pure domain code. A service, calculator, or validator. No database, no HTTP. This is the easiest type.

Day 9: API endpoint. Integration test with `WebApplicationFactory`. The agent builds the full vertical slice: controller, handler, service, and DI registration. This is moderately complex.

Day 10: Refactoring. Write tests that capture existing behavior. Ask the agent to refactor the implementation while keeping tests green. This is the trust exercise.

By Day 10, the TDD loop should feel natural. If it doesn't, repeat Days 8-10 with different tasks before moving on.

Days 8-10 checkpoint: Three different TDD loop tasks completed. You can do the workflow without referencing the book.

Days 11-12: Error-Driven Debugging

The debugging workflow: paste an error message, let the agent diagnose and fix.

Pick a real bug (or introduce one deliberately). Give the agent the error message and relevant context:

```
This test is failing with the following error:

[paste full error output]

The test is in [file]. The code under test is in [file].
Diagnose the root cause and fix it. Run the tests to confirm.
```

Practice this workflow on: - A failing unit test (Day 11) - A runtime exception from a log (Day 12)

Days 11-12 checkpoint: Two bugs fixed using error-driven agent debugging.

Days 13-14: Codebase Understanding

Use agents to explore unfamiliar code. If your project doesn't have unfamiliar parts, use an open-source project you've never seen before.

```
I need to understand how [feature] works in this codebase.

Trace the flow from the API endpoint to the database for [operation].
Include: which files are involved, what patterns are used,
and where the business logic lives.
```

Then go deeper:

```
Based on the [feature] flow you traced:
1. What are the key design decisions?
2. Are there any code smells or potential issues?
3. How would you add [new capability] to this flow?
```

Use agents for codebase archaeology at least twice this week. It's one of the highest-value use cases and one that most developers underuse.

Days 13-14 checkpoint: You've used agents to understand two unfamiliar code areas and can explain them to a colleague.

Week 3: Scale Up (Days 15-21)

This week you push beyond single-agent, single-task workflows.

Days 15-16: Parallel Agent Sessions

Run two agents simultaneously on different tasks. This requires git worktrees or separate branches:

```
# Create a worktree for the second agent
git worktree add ../myapp-agent2 -b feature/agent2-task
```

Assign each agent a different, non-overlapping task. While Agent 1 works on a new endpoint, Agent 2 writes tests for an existing module. Practice context-switching between the two: check on Agent 1, prompt Agent 2, review Agent 1's output, correct Agent 2.

The key insight from parallel work: your bottleneck shifts from coding speed to review speed. The agents produce faster than you can verify. Get comfortable with that tension.

Days 15-16 checkpoint: Two parallel tasks completed. You've experienced the review bottleneck.

Days 17-18: CI/CD Agent Integration

Set up an agent that reacts to CI failures. The specifics depend on your CI platform, but the pattern is universal:

1. CI run fails
2. Agent receives the failure log
3. Agent diagnoses the issue
4. Agent creates a fix
5. Agent pushes the fix (or creates a PR for your review)

For GitHub, Copilot Coding Agent and Codex can be assigned issues directly. For other platforms, use a CLI agent triggered by a webhook or manual step.

Start simple: when a test fails in CI, paste the failure log to your agent and ask it to fix it. You'll automate the trigger later.

Days 17-18 checkpoint: At least one CI failure diagnosed and fixed with agent assistance.

Days 19-21: Background Agent for an Overnight Task

Assign an agent a larger task (2-4 hours of work) and let it run while you do something else. Good candidates:

- Generate comprehensive tests for an untested module
- Migrate a set of endpoints from one pattern to another
- Add structured logging throughout a service
- Create API documentation from code

Use a background agent tool (Codex Cloud, GitHub Copilot Coding Agent, or a long-running Claude Code session). Write a detailed spec as a markdown file and reference it in your prompt:

```
Implement the spec in docs/logging-migration-spec.md.
Work through each file listed in the spec.
Run tests after each file to ensure nothing breaks.
Create a PR when complete.
```

Review the result the next morning. Note what was done well and what needs correction.

Days 19-21 checkpoint: One background agent task completed. You've experienced the async workflow.

Week 4: Full Agentic Workflow (Days 22-30)

This is the capstone. You're building a complete feature using everything you've learned.

Days 22-24: Plan the Feature

Pick a real feature from your backlog. Something meaningful: a new module, a significant enhancement, or a complex bug fix. Not a toy example.

Write the spec as a markdown document: - What the feature does (user stories or requirements) - Acceptance criteria - Affected files and modules - Test strategy - Any constraints or considerations

Break it into tasks. Each task should be one agent session (30-60 minutes of agent work). A typical feature breaks into 4-8 tasks.

Days 22-24 checkpoint: Feature spec written. Tasks broken down. Ready for implementation.

Days 25-28: Build It

Implement the feature using only agentic methods:

- TDD Agent Loop for each task
- Parallel agents when tasks don't overlap
- Error-driven debugging when things break
- Background agents for boilerplate and tests
- Security review agent on the completed code (Chapter 27)

Track everything: - Time per task - Number of agent iterations - Manual corrections required - Bugs found in review

Be disciplined about Chapter 28 too. If you hit a task where the agent is clearly the wrong tool, do it by hand. Note the decision and why.

Days 25-28 checkpoint: Feature complete. All tests passing. Code reviewed and ready for PR.

Days 29-30: Measure and Refine

Compare your Week 4 feature build against your baseline measurement from before Day 1:

- **Time:** Faster or slower? By how much?
- **Quality:** More or fewer bugs? Better or worse code structure?
- **Coverage:** More or fewer tests? Better edge case handling?
- **Confidence:** Do you trust the code more or less?

Update your AGENTS.md one final time. After a month of use, you know exactly what the agent needs to know. Your context file should be tight, specific, and battle-tested.

Set up your ongoing rhythm: - **Daily:** Use the TDD Agent Loop for feature work. Review and update path-scoped rules when the agent drifts. - **Weekly:** Run a dependency audit and security scan. - **Monthly:** Review your AGENTS.md. Remove stale instructions. Add new patterns you've adopted. - **Quarterly:** Measure your productivity again. Are you still improving?

How to Measure Your Progress

Subjective feeling is unreliable (remember METR: developers felt 20% faster while being 19% slower). Use objective measurements:

Task completion time. Pick 3-5 representative task types and track time consistently. Compare month-over-month.

PR turnaround. From task start to merged PR. This captures the full cycle including review and correction.

Test coverage delta. Are you writing more tests with agents? (You should be. Tests are cheap when agents help implement against them.)

Bug escape rate. Bugs found after merge. This should decrease as your TDD and security review workflows mature.

Context file quality. Subjective, but track it: how often does the agent need correction? Fewer corrections means better context.

Don't measure lines of code. More lines isn't better. Don't measure prompts per day. More prompts isn't better. Measure outcomes: features shipped, bugs prevented, time saved.

Common Pitfalls and How to Avoid Them

Week 1 Pitfalls

Pitfall: Writing a massive AGENTS.md on Day 2. You don't know what the agent needs yet. Start small. Add instructions when the agent does something wrong, not preemptively. A 20-line AGENTS.md beats a 200-line one that's full of irrelevant instructions.

Pitfall: Picking a complex first task on Day 3. Your first agent task should be embarrassingly simple. A DTO addition. A validation rule. You're learning the tool, not testing its limits. Save the complex tasks for Week 2.

Pitfall: Abandoning the plan after a bad Day 3. Your first agent task will be slower than doing it by hand. That's expected. The investment pays off in Week 2 when the context file starts working for you.

Week 2 Pitfalls

Pitfall: Skipping the TDD loop because "it works without tests." It does work without tests. It works worse. The TDD loop isn't about making agents functional. It's about making agents reliable. Without tests, you're reviewing code hoping it works. With tests, you know it works and you're reviewing for quality.

Pitfall: Not updating context files after corrections. Every time you correct the agent, ask: “Should this correction be in AGENTS.md so I never make it again?” If yes, add it immediately. Context files grow from corrections, not from imagination.

Week 3 Pitfalls

Pitfall: Running too many parallel agents. Start with two. Seriously. Two parallel agents already strain your review capacity. Three is manageable for experienced users. Five is for when you have months of practice and well-tested context files. The bottleneck is always your review speed, not the agent count.

Pitfall: Expecting the background agent to produce perfect output. Background agents produce first-draft output. Plan 30-60 minutes of review and correction for every 2-4 hours of background agent work. If you’re not budgeting review time, you’ll be disappointed.

Week 4 Pitfalls

Pitfall: Forcing the agent on every task. Some parts of your feature will be faster by hand. Do them by hand. The goal is building the feature efficiently, not proving that agents can do everything.

Pitfall: Not measuring. If you don’t compare against your baseline, you have opinions instead of data. Opinions are unreliable (METR proved this). Measure. Compare. Adjust.

Pitfall: Declaring victory (or defeat) too early. Day 30 isn’t the end. It’s the beginning of a practice that improves over months. Your context files get better. Your prompts get sharper. Your judgment about when to use agents gets more accurate. Measure again at Day 60 and Day 90.

The Day 31 Question

On Day 31, ask yourself: “Would I go back to working without agents?”

If the answer is “no,” you’ve transformed. Not because agents are magic, but because you’ve built the discipline, the context, and the judgment to use them well. You’ve internalized when they help and when they don’t. You’ve built the infrastructure (context files, test habits, review workflows) that makes agent-assisted development genuinely productive instead of performatively productive.

If the answer is “yes” or “maybe,” that’s valid too. Review your measurements. Where did agents help? Where did they hurt? Maybe the transformation is narrower than a full workflow shift. Maybe agents work for your test writing but not your feature building. Maybe they work for new code but not maintenance. Partial adoption based on evidence is better than full adoption based on hype.

Either way, you have 30 days of data, a refined context file, and a clear understanding of what agentic development means for your specific work. That’s worth more than any amount of theory.

Now go build something.